



Object Oriented Programming Lab Manual

Faculty Name(s)

The Practice of Knowledge

Department of Computer Science
COMSATS University, Islamabad
(Abbottabad Campus)

Object Oriented Programming

Lab Manual



Dr. Rab Nawaz Jadoon

Assistant Professor

Department of Computer Sciences

COMSATS University, Islamabad

Abbottabad Campus

Jadoon956.wordpress.com

Jadoon.rabnawaz@gmail.com

Contents

Preface	6
Lab No. 1: Object Oriented Modeling.....	8
Lab No. 2: Control Structures.....	11
Lab No. 3: Classes, Objects, and Methods	19
Lab No. 4: Methods.....	26
Lab No. 5: Method Overloading.....	32
Lab No. 6: Arrays Application - I.....	39
Lab No. 7: Arrays Application - II.....	44
Lab No. 8: String handling.....	50
Lab No. 9: Inheritance.....	57
Lab No. 10: Inheritance, Overriding and Constructor Chaining.....	68
Lab No. 11: Polymorphism-I	79
Lab No. 12: Polymorphism-II.....	85
Lab No. 13: Abstract classes, interface, and inner classes-I.....	95
Lab No. 14: Abstract classes, interface, and inner classes-II.....	98
Lab No. 15: Abstract classes, interface, and inner classes-III.....	102
Lab No. 16: Static Members and Functions	107
Lab No. 17: File Handling.	114
Lab No. 18: Exception Handling-I	118
Lab No. 19: Exception Handling-II	125
Lab No. 20: GUI-Programming-I.....	134
Lab No. 21: GUI Programming-II	146
Lab No. 22: JDBC-I	153



Lab No. 23: JDBC-II	159
Lab No. 24: JDBC-III	165
Lab No. 25: Multithreading-I	171
Lab No. 26: Multithreading-II	177
Lab No. 27: Special Lab Task - I	182
Lab No. 28: Special Lab Task - II	185
Lab No. 29: Special Lab Task - III	188
Lab No. 30: Special Lab Task - IV	185



Preface

The indispensable intention for writing the manual for Object Oriented Programming using JAVA is to impart inclusive guidance for the students to learn object oriented programming technique for developing software using JAVA. The inspiration at the rear is to spotlight on the practical phase of programming. To implement this objective it has been included couple of examples regarding each topic.

The manual provides complete programming guide for the students from beginning to some advanced level. The language is simple and stress-free to understand.

Each lab ends with some exercises for best practice at lab and for home too. This manual is basically intended for the students but it can also be used by any person who wants to get the knowledge of Object Oriented Programming.



Lab No. 1

Object Oriented Modeling

Lab No. 1: Object Oriented Modeling

Objective:

To explore the concept behind Object Oriented Modeling.

Scope:

In completing this exercise, the students will be able to:

- Identify entities in a simple “personal banking” application domain.
- Identify attributes required to model these entities.
- Identify methods required to model these entities.
- Identify relationships among these entities.

A Case Study:

1. The Personal Banking Domain

In this exercise the students will investigate the different kinds of objects required to model the banking domain. To make it simple, the scope of the model is limited to personal banking activities, i.e. excluding business banking.

2. Brain-Storming

Consider a world in which people can save money into a bank. There may be different banks (e.g. Bank Al-Habib, Allied Bank etc) and customers may have different activities with their banks.

- Write down the different kinds of objects which are involved in this “Personal banking” domain.
- Look at the objects that you put down above. Is/are there any object(s) that you chose not to include in your model? What are they? Why?

3. Customer

Depending on your brain-storming result in exercise above, you may have some Customer objects. Consider a customer who can open bank accounts in a bank. Write down the list of attributes required to model the data components of a customer, (e.g. the name of the customer)

For each of these attributes:

- Would you model it as a class-level/object-level attribute and why?
- What will be the appropriate data type? (e.g. integer, floating point number, a string, etc.)
- Would you make this attribute visible to other objects and why?
- Would you hide this attribute from other objects and why?
- Write down the list of methods required to model the procedural component of a customer, i.e. Operations/ behavior that a customer can perform. For each of these methods:
- Would you model it as a class-level/object-level method and why?
- Does the method need any input argument/ parameter? If yes, what is/are the data type(s) of this/these parameter(s)?
- Does the method return any value? If yes, what is the appropriate return data type?



- Would you make the method visible/invisible to other objects? Why?

4. Bank Account

A bank account stores data about the amount of money that a customer saves in a bank. Write down the list of attributes required to model a bank account. For each of these attributes:

- Should it be object/class-level and why?
- What is the appropriate data type? (E.g. integer, floating point number, a string, etc.)
- Would you make the attribute visible to the outside world and why?
- Write down the list of methods required to model a bank account. For each of these methods:
- Should it be object/class-level and why?
- Does the method need any input argument/ parameter? If yes, what is/are the data type(s) of this/these parameter(s)?
- Does the method return any value? If yes, what is the appropriate return data type?
- Would you make the method visible/invisible to other objects? Why?

Consider the relationship between a bank account and a customer.

- Do you think this is a one-to-one, one-to-many, or many-to-many relationship? Why?
- If you need to model this relationship between a bank account and a customer, would you model it as an attribute or method? Why and How?
- Would you make it an attribute/method of the bank account or the customer?
- What would be the data type of this attribute/method?

5. The Bank

A bank account stores data about the amount of money that a customer saves in a bank. Write down the list of attributes and methods required to model a Bank. For each of this attributes/method:

- Would you model it as a class/object-level attribute/method? Why?

For each attribute:

- What is the appropriate data type?
- For each method:
- What is/are the required input argument(s)/parameter(s)?
- What is/are its/their data type(s)?
- What is the return data type (if any)?
- Consider the bank and bank accounts:
- Is there any relationship between a bank and bank accounts?
- Is it a one-to-one, one-to-many, or many-to-many relationship?
- How would you model this relationship?
- Consider the bank and customer:
- Is there any relationship between banks a customer?

Will you model this relationship? If yes, how? If you are going to have multiple banks in our domain (e.g. Bank Al-Habib, Allied Bank etc), will it make a difference to your domain model? If you need to make any change, how will you modify it?



Lab No. 2

Control structures

Lab No. 2: Control Structures

Objective:

Implementation of Control Structures (Selection, and Iterative statements)

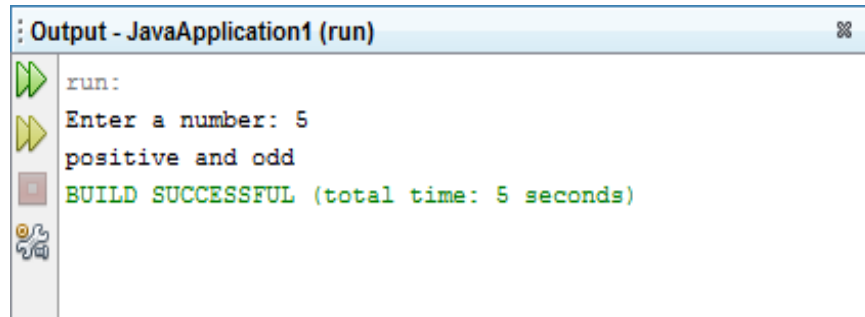
Scope:

Sometimes we need to make a decision between two choices. The statements in by which we make this decision are called conditional statements. We use loops to make one or more statements repeat for specified number of times. Clearly this has an importance when we wish to take more than one input from user or do some calculations.

Example 1: This program is used to demonstrate the uses if-else condition.

```
import java.io.*;
public class Numbers{
    public static void main(String[] args)
        throws IOException{
        BufferedReader in;
        in = new BufferedReader(new InputStreamReader(System.in));
        int x;
        String userInput;
        System.out.print("Enter a number: ");
        userInput = in.readLine();
        x = Integer.parseInt(userInput);
        if(x == 0){
            System.out.println("zero");} //x is positive
        else if(x > 0){ //x is even
            if(x % 2 == 0){
                System.out.println("positive and even");
            } //x is odd
            else{
                System.out.println("positive and odd");
            } //x is negative
        } else{ //x is even
            if(x % 2 == 0){
                System.out.println("negative and even");
            } //x is odd
            else{
                System.out.println("negative and odd");
            }
        }
    }
}
```

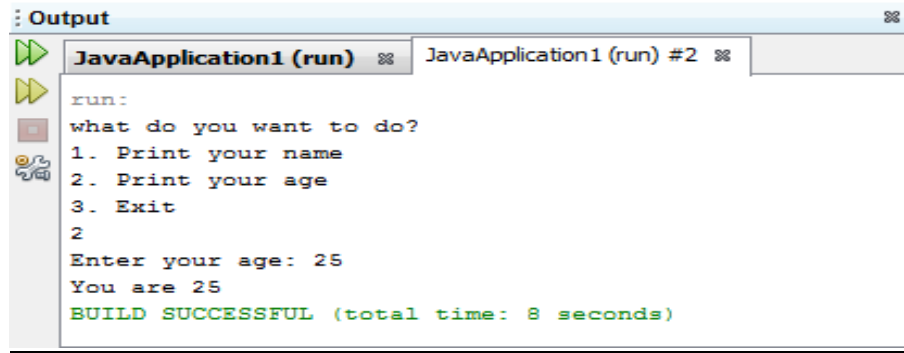




Example 2: This program is used to demonstrate the uses switch-case.

```
import java.io.*;
public class SwitchCase{
public static void main(String[] args)
throws IOException{
BufferedReader in;
in = new BufferedReader(new InputStreamReader(System.in));
System.out.println("what do you want to do?");
System.out.println("1. Print your name");
System.out.println("2. Print your age");
System.out.println("3. Exit");
int choice;
choice = Integer.parseInt(in.readLine());
switch(choice){
case 1:
System.out.print("Enter your name: ");
String name = in.readLine();
System.out.println("Welcome " + name);
break;
case 2:
System.out.print("Enter your age: ");
String age = in.readLine();
System.out.println("You are " + age);
break;
case 3:
System.exit(0);} //End switch
} //End main
} //End class
```



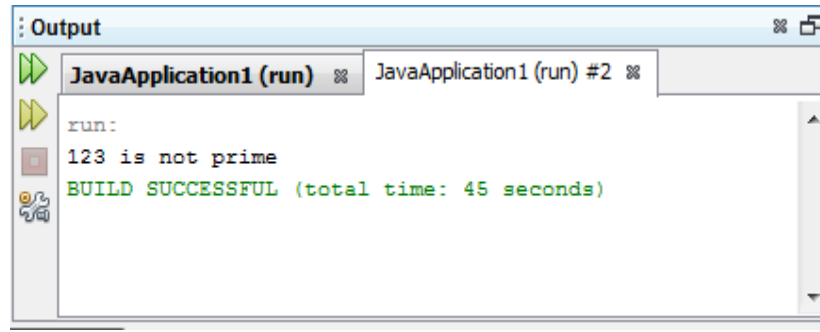
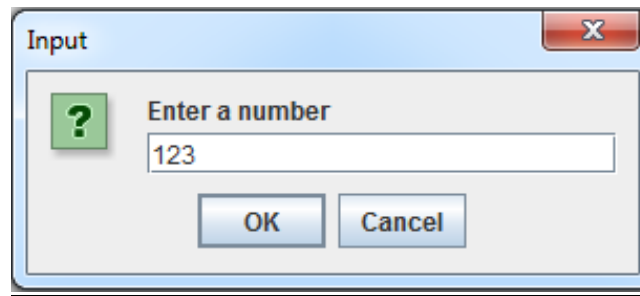


```
run:
what do you want to do?
1. Print your name
2. Print your age
3. Exit
2
Enter your age: 25
You are 25
BUILD SUCCESSFUL (total time: 8 seconds)
```

Example 3: This program demonstrates either an input number is prime or not

```
import javax.swing.*;
class Prime{
public static void main(String[] args){
int x;
String userInput;
userInput=JOptionPane.showInputDialog("Enter a number");
x = Integer.parseInt(userInput);
//1, 2 and 3 are prime numbers
if(x == 1 || x == 2 || x == 3){
System.out.println(x + " is prime");
}
//All even numbers > 2 are not prime
else if(x % 2 == 0){
System.out.println(x + " is not prime");
}else{
//We have to check using division remainder
//we initially suppose that x is prime
boolean prime = true;
//No number has factor greater than it's half
for(int i = 3; i <= x / 2; i++){
if(x % i == 0){
//We have found a factor
//so x is not prime
prime = false;
break;}}
if(prime){
System.out.println(x + " is prime");}
else{
System.out.println(x + " is not prime");}}}}
```





Example 4: Following program that keeps reading numbers (integers) from user until input is 0, then prints average, sum, maximum and minimum.

```

import java.io.*;
class Numbers1{
public static void main(String[] args) throws IOException{
int min=0, max=0, x, sum = 0, count = 0;
boolean firstNumber = true;
BufferedReader in =new BufferedReader(new InputStreamReader(System.in));
while(true){
System.out.print("Enter a number: ");
x = Integer.parseInt(in.readLine());
if(x == 0){ break; }
if(firstNumber){
min = max = x;
firstNumber = false;
}
sum += x;
if(x > max){ max = x; }
if(x < min){ min = x; }
count++;
}
System.out.println("Sum = " + sum);
System.out.println("Average = " + sum / count);
System.out.println("Minimum = " + min);
System.out.println("Maximum = " + max);}}

```

```

Output
JavaApplication1 (run)  JavaApplication1 (run) #2
run:
Enter a number: 1
Enter a number: 2
Enter a number: 3
Enter a number: 0
Sum = 6
Average = 2
Minimum = 1
Maximum = 3
BUILD SUCCESSFUL (total time: 10 seconds)

```

Exercises

Exercise 1:

Write a program that specifies whether a given number (x) falls in one of the following categories (give x a value from the code, don't read from user):



- 0 to 9
- 10 to 19
- 20 to 29
- None of the categories

For example, if $x = 5$, program should print "0 to 10", and if $x = 44$ it should print "None".

Hint: use if statements with Boolean expressions combined using &&.

Exercise 2:

Write a program that reads two integers and prints their sum like the code below (text shown in boldface is supposed to be user input).

Enter the first number: 3

Enter the second number: 4

The sum is 7

Exercise 3:

Write a program that asks the user to enter two numerical values (integers) and then select an operation (addition, subtraction, multiplication and division) then prints the result based on operation selected. The code below shows examples of the output (text shown in boldface is supposed to be user input).

Enter first number: 4

Enter second number: 2

1. Addition (+).
2. Subtraction (-).
3. Multiplication (*).
4. Division (/).

Enter operation number: 3

The result is 8

Exercise 4:

Write a program that reads 10 numbers from the user then prints out how many positive numbers and negative numbers user has entered (consider 0 a positive number).

Homework

1. Modify calculation program in session 4 exercise 3 by adding the following question at the end of the program:

Do you want to make another calculation?

1. Yes
2. No



Enter your option:

If user selects yes (by entering 1), program will ask him again to enter new two numbers and select operation, if user selects no (by entering 2), program exits. Use appropriate loop to accomplish this.

2. Write a program that asks the user to enter certain number, after that asks him to enter another 20 numbers, after entering them all, it prints out the number of occurrences of the first number. See the below example (text shown in boldface is supposed to be user input).

Enter number to search for: 2

Enter a number: (20 times)

2 3 56 7 9 2 4 5 5 6 2 21 33 19 32 88 0 32 100 20

The number (2) occurred 3 times in your input.



Lab No. 3

Classes, Objects, and

Methods

Lab No. 3: Classes, Objects, and Methods

Objective:

- Understanding the concept of Object-Oriented programming.
- Introducing classes and objects and differences between them.
- Understanding class members and their properties.
- Calling methods.

Scope:

In JAVA, programmers concentrate on creating their own user defined classes and objects. Each class contains data and data members and methods to facilitate the user how to kept data encapsulated manner for the protection from the unauthorized access.

Example 1: This program demonstrates a simple class implementation.

```
class Circle{
    //Radius is enough to determine all circle attributes
    public double radius;
    //Constructor
    public Circle(double r){
        radius = r;
    }
    //Methods
    public double getRadius(){
        return radius;
    }
    public double getArea(){
        return (radius * radius) * Math.PI;
    }
    public double getCircumference(){
        return 2 * Math.PI * radius;
    }
    public double getDiameter(){
        return radius * 2;
    }
}
```

```
class circle1{
    Public static void main(String[] a){
```



```

Circle c1 = new Circle(10);
Circle c2 = new Circle(12);
System.out.println("c1 Area is " + c1.getArea());
System.out.println("c2 Diameter is " + c2.getDiameter());

Circle c1 = new Circle(10);
System.out.println("c1 Area is " + c1.getArea());
c1.radius = 15;
System.out.println("c1 Area is now " + c1.getArea());
} }

```

Example 2: This program demonstrates the private member of a class can't be accessed from the outside of the class which reports compile-time error..

```

class A{
private int value = 10;
public int getValue(){ return value; }
public void setValue(int newValue){ value = newValue; }
}

class B{
public static void main(String[] args){
A a = new A();
System.out.println(a.value); //Error, value is private
}
}

```

```

class B{
public static void main(String[] args){
A a = new A();
a.setValue(33); //Changes the value by mutator method
System.out.println(a.getValue()); //prints 33
}
}

```



Example 3: The following example shows the difference between static and non-static variables.

```
class StaticTest{
private static int a;
private int b;
public StaticTest(){
a = 0; b = 0;
}
//Accessors
public static int getA(){ return a; }
public int getB(){ return b; }
//Mutators
public static void setA(int newA){ a = newA; }
public void setB(int newB){ b = newB; }
}
class Main{
public static void main(String[] args){
StaticTest s1 = new StaticTest();
StaticTest s2 = new StaticTest();
System.out.println("s1 values:");
System.out.println("a =" + s1.getA() + ", b =" + s2.getB());
System.out.println("s2 values:");
System.out.println("a =" + s2.getA() + ", b =" + s2.getB());
s1.setA(5);
s1.setB(3);
System.out.println("\ns1 values:");
System.out.println("a =" + s1.getA() + ", b =" + s2.getB());
System.out.println("s2 values:");
System.out.println("a =" + s2.getA() + ", b =" + s2.getB());
}
}
```

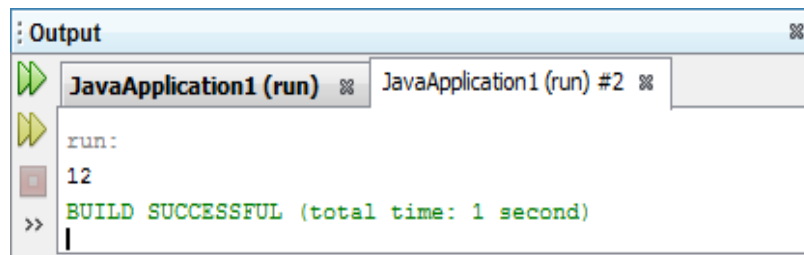
```
s1 values:
a =0, b =0
s2 values:
a =0, b =0

s1 values:
a =5, b =0
s2 values:
a =5, b =0
BUILD SUCCESSFUL (total time: 1 second)
```



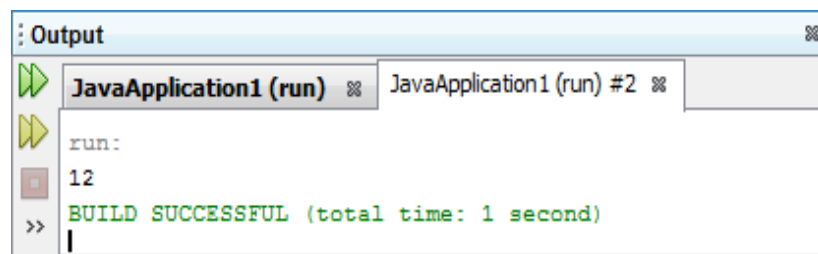
Example 4: The following example shows how to add two numbers using method calling.

```
class Addition{
public static void main(String[] args){
int x = 5;
int y = 7;
sum(x, y);
}
static void sum(int a, int b){
int c = a + b;
System.out.println(c);
}
```



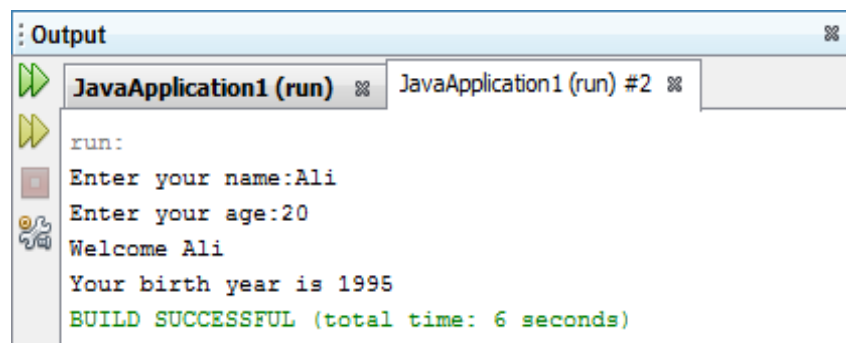
Example 5: Method returns integer that contains the sum of two numbers.

```
class Addition{
public static void main(String[] args){
int x = 5;
int y = 7;
int z = sum(x, y);
System.out.println(z);
}
static int sum(int a, int b){
int c = a + b;
return c;
}
}
```



Example 6: Demonstration of methods with BufferedReader class

```
import java.io.*;
class ReadingSimplifier{
public static void main(String[] args){
String name = readString("Enter your name:");
int age = readInt("Enter your age:");
System.out.println("Welcome " + name);
System.out.println("Your birth year is " + (2007-age));
}
static String readString(String message){
System.out.print(message);
BufferedReader in = new BufferedReader(
new InputStreamReader(System.in));
try{
String input = in.readLine();
return input;
}catch(Exception e){}
return "";
}
static int readInt(String message){
String input = readString(message);
return Integer.parseInt(input);
}
}
```



Exercises:

Exercise 1:

Write a class Book with three data members BookId, Pages and Price. It also contains the following member function:

- The get () method is used to input values
- The show () method is used to display values



- The `set()` method is used to set the values of data members using parameters
- The `getPrice()` method is used to return the value of price

Exercise 2:

Create a class `building` that has the public member `floors`, `area`, and `occupants` and a method `areaperperson()` respectively that display the area per person for building. In the `main()` method create two instance of `building` called `house` and `office` and display the area per person by division of `area/occupants`.

Homework:

- 1) Write a class `Result` that contains `roll no`, `name` and `marks` of three subjects. The marks are stored in an array of integers. The class also contains the following member functions.
 - The `input()` method is used to input values
 - The `show()` method is used to display values
 - The `total()` returns the total marks a student
 - The `avg()` method returns the average marks of a student
- 2) Create a class `rectangle`. The class has attributes `length` and `width` each of which defaults to 1. It has member function that calculates the perimeter and the area of the rectangle. It has `set()` and `get()` functions for both `length` and `width`. The `set()` function should verify that `length` and `width` are each floating point numbers larger than 0.0 and less than 20.0.



Lab No. 4

Methods

Lab No. 4: Methods

Objective:

To implements the concepts of Methods.

Scope:

A Java method is a collection of statements that are grouped together to perform an operation. When you call the System.out.println method, for example, the system actually executes several statements in order to display a message on the console. Now you will learn how to create your own methods with or without return values, invoke a method with or without parameters, overload methods using the same names, and apply method abstraction in the program design.

Example 1: This program demonstrates the use of methods.

```
package methods;
class A{
    int b_id, price;
    String b_name;
    void get(int id, String name, int p){
        b_id = id;
        b_name = name;
        price = p;
    }
    void show(){
        System.out.println("Book ID :"+b_id);
        System.out.println("Book Name :"+b_name);
        System.out.println("Book Price :"+price);
        System.out.println("\n");
    }
    int pricing(){
        return(price);
    }
}
//main...
public class Methods {
    public static void main(String[] args) {
        A ob1 = new A();
        A ob2 = new A();
        ob1.get(99,"C++ Programming",1350);
        ob2.get(83,"JAVA Programming",1583);
        ob1.show();      ob2.show();
    }
}
```



```

        System.out.println("\nMost costly book details are");
        if(ob1.pricing() > ob2.pricing())
            ob1.show();
        else
            ob2.show();
    }
}

```

Output - methods (run)

```

run:
Book ID :99
Book Name :C++ Programming
Book Price :1350
Book ID :83
Book Name :JAVA Programming
Book Price :1583

Most costly book details are
Book ID :83
Book Name :JAVA Programming
Book Price :1583
BUILD SUCCESSFUL (total time: 3 seconds)

```

Example 2: This program demonstrates the use of methods

```
package methods;
import javax.swing.*;
class studs{
    int id;
    String name;
    int[]marks = new int[3];
    int sum=0;
    void get(){
        id = Integer.parseInt(JOptionPane.showInputDialog("Enter ID:"));
        name = JOptionPane.showInputDialog("Enter Name:");
        for(int i=0; i<marks.length; i++){
            marks[i] = Integer.parseInt(JOptionPane.showInputDialog
                ("Marks for subject ["+i+"]"));
            sum+=marks[i];
        }
    }
    void total(){
        for(int i=0; i<3; i++)
            sum+=marks[i];
    }
    void show(){
        System.out.println("ID..." +id);
        System.out.println("Name..." +name);
        System.out.println("Obtained Marks in 3 subjects....." +sum);
        System.out.println("\n");
    }
}

//main...
public class Methods {
    public static void main(String[] args) {
        studs[] ob = new studs[3];
        for(int i=0; i<3; i++){
            ob[i]=new studs();
            ob[i].get();
            ob[i].total();
            ob[i].show();
        }
    }
}
```



```
Output - methods (run)

run:
ID...100
Name...Mohammad Tariq
Obtained Marks in 3 subjects.....526

ID...101
Name...Mohammad Umar
Obtained Marks in 3 subjects.....500

ID...102
Name...Aamir Khan
Obtained Marks in 3 subjects.....428

BUILD SUCCESSFUL (total time: 1 minute 17 seconds)
```

Example 3: This program demonstrates the use of methods.

```
package methods;
class Ob_pass_ret{
    public int x,y;
    public void get(int a, int b){
        x=a;
        y=b;
    }
    public Ob_pass_ret update(Ob_pass_ret obj){
        obj.x+=10;
        obj.y+=10;
        Ob_pass_ret temp=new Ob_pass_ret();
        temp.x=obj.x+this.x;
        temp.y=obj.y+this.y;
        return temp;
    }
    public void show(){
        System.out.println(x+"\t"+y);
    }
}

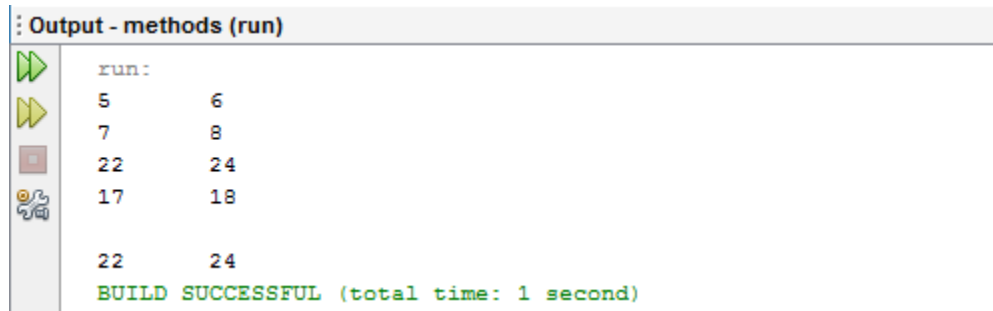
//main...
public class Methods {
    public static void main(String[] args) {
        Ob_pass_ret ob1=new Ob_pass_ret();
        Ob_pass_ret ob2=new Ob_pass_ret();
        ob1.get(5,6);          ob1.show();
        ob2.get(7,8);          ob2.show();
    }
}
```



```

    ob1=ob1.update (ob2) ;
    ob1.show() ;      ob2.show() ;
    System.out.println ("\n"+ob1.x+"\t"+ob1.y) ;
}
}

```



```

Output - methods (run)
run:
5      6
7      8
22     24
17     18

22     24
BUILD SUCCESSFUL (total time: 1 second)

```

Exercises:

Exercise 1:

Write a program to generate a Fibonacci series by creating `fibonacci` class. The result become like

1 1 2 3 5 8 and so on

Homework:

1) Write a class `Car` that contains the following attributes:

The name of car

The direction of car (E, W, N, S)

The position of car (from imaginary zero point)

The class has the following methods:

- A constructor to initialize the attributes
- `Turn()` method to change the direction of car to one step right side (i.e. if the direction is to E, it should be changed to S and so on)
- Overload the `turn()` method to change the direction to any side directly. It should accept the direction as parameter.
- `Move()` method to change the position of car away from zero point. It should accept the distance as parameter.



Lab No. 5

Method Overloading

Lab No. 4: Method Overloading

Objective:

- Defining constructors
- Understanding method / constructor overloading.
- Creating and using packages

Scope:

Overloading means to have more than one constructors or methods having same name. There are more than one way to discriminate between methods, including name and number and type of parameters. To see how parameters can help in overloading, consider the following examples.

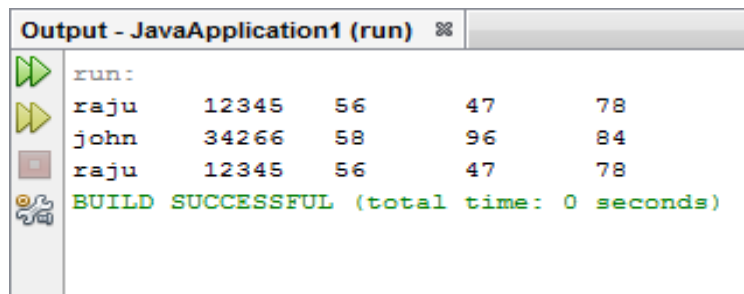
Packaging is a way to organize files when a project consists of multiple modules. It also helps resolve naming conflicts when different packages have classes with the same names.

Packages access level also allows you to protect data from being used by the non-authorized classes.

Example 1: Write a Java Program to define a class, describe its constructor, overload the Constructors and instantiate its object

```
import java.lang.*;
class student{
    String name;
    int regno, marks1,marks2,marks3;
    student(){ // null constructor
        name="raju";
        regno=12345;
        marks1=56;
        marks2=47;
        marks3=78;}
    student(String n,int r,int m1,int m2,int m3){// parameterized constructor
        name=n;
        regno=r;
        marks1=m1;
        marks2=m2;
        marks3=m3;}
    student(student s){// copy constructor
        name=s.name;
        regno=s.regno;
        marks1=s.marks1;
        marks2=s.marks2;
        marks3=s.marks3;}
    void display(){
        System.out.println(name + "\t" +regno+ "\t" +marks1+ "\t" +marks2+ "\t" + marks3);}}
class studentdemo{
    public static void main(String arg[]){
        student s1=new student();
        student s2=new student("john",34266,58,96,84);
        student s3=new student(s1);
        s1.display();
        s2.display();
        s3.display();}}
```

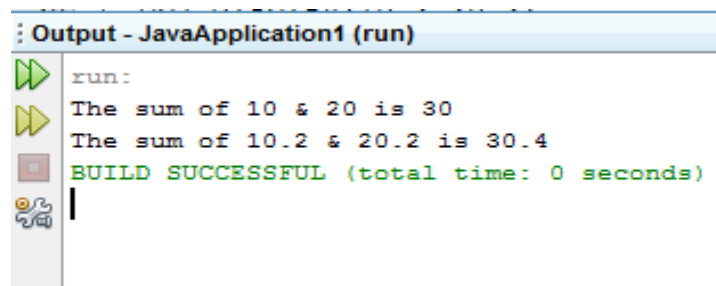




```
run:
raju 12345 56 47 78
john 34266 58 96 84
raju 12345 56 47 78
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example 2: Write a Java Program to define a class, define instance methods and overload them and use them for dynamic method invocation.

```
import java.lang.*;
class add
{
void display(int a,int b)
{
int c=a+b;
System.out.println("The sum of " + a + " & " + b + " is " + c);
}
void display(double a,double b)
{ double c=a+b;
System.out.println("The sum of " + a + " & " + b + " is " + c);
}
}
class add_demo
{
public static void main(String arg[])
{
add obj=new add();
obj.display(10,20);
obj.display(10.2,20.2);
}
}
```



```
run:
The sum of 10 & 20 is 30
The sum of 10.2 & 20.2 is 30.4
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example 3: Implementation of constructor overloading

```
class Rectangle{
private double height, width;
public Rectangle(double h, double w){
height = h;
width = w;
}
public Rectangle(double d){
height = width = d;
}
public Rectangle(){
height = width = 1;
}
}
```

Example 4: This code generates error message.

```
class A{
private int value;
public void setValue(int t){
value = t;
}
public void setValue(int s){
value = s;
}
}
```

To better understand classes and objects, let's analyze a well-known statement we've been using repeatedly:

```
System.out.println("Hello World");
```

System is a name of a class, inside System class, there is a public and static member (object) of class PrintWriter, the name of this object is out. Inside PrintWriter class, there is a public, non-static method called println that has several overloads.

Exercises:

Exercise 1:

- Write a class that represents triangle named Triangle, the class must have the following-members:
 - private double height;//Height
 - private double base;//Base length
 - public Triangle(double h, double b);//Constructor
 - public void setHeight(double x);//Sets height
 - public double getHeight();//Gets height
 - public void setBase(double x);//Sets base length



- `public double getBase();`//Gets base length
- `public double getArea();`//Returns the area of the triangle

Exercise 2:

Write a class `Employee` that represents an employee of some organization, the class should contain the following members:

- `private int id;`//Employee id
- `private String name;`//Employee name
- `private int type;`//1 = employee, 2 = manager
- `private double baseSalary;`//Base salary
- `public Employee(int _id, String _name);`//Constructor
- `public void setID(int x);`//id mutator
- `public void setName(int x);`//name mutator
- `public int getID();`//id accessor
- `public String getName();`//name accessor
- `public double getSalary();`
- `//if manager, add 10% to base salary`
- `public void setBaseSalary(double bs);`//sets base salary.

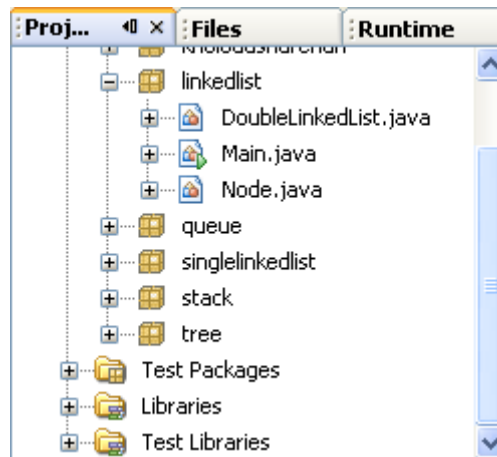
Creating and using Packages:

A package is the collection of interfaces and classes. It is an important aspect one should be aware of, packaging means putting two or more classes/interfaces in the same package by stating package name before declaring class (and also exceptions). When two classes are in the same package, it is possible to access any one of them from the other, for example, if we have a class named `Student` that represents a student, another class named `Main` to run a program that manages student information, we can access `Student` class from `Main` class and define several objects of it if we use the following statement before each class declaration:

```
package studentinfo;
```

In NetBeans IDE, there is no need to manually write the above statement, by adding new class to any package of your project, that class is automatically declared as a part of that package. Following Figure shows packages and classes in NetBeans IDE project explorer.





To access a class from its package we simply provide package name followed by dot '.' then desired class name. For example, we can access Main class show in above Figure by calling it in the following statement:
linkedList.Main

Example 5: Write a Java program to implement the concept of importing classes from user defined package.

```
package p1;
class Student
{
    int regno;
    String name;
    public void getdata(int r,String s)
    {
        regno=r;
        name=s;
    }
    public void putdata()
    {
        System.out.println("regno = " + regno);
        System.out.println("name = " + name);
    }
}
```

```

class StudentTest
{
public static void main(String arg[])
{
Student s=new Student();
s.getdata(123,"xyz");
s.putdata();
}
}

```

```

class StudentTest
{
public static void main(String arg[])
{
Student s=new Student();
s.getdata(123,"xyz");
s.putdata();
}
}

```

```

: Output - JavaApplication1 (run)
run:
regno = 123
name = xyz
BUILD SUCCESSFUL (total time: 0 seconds)

```



Lab No. 6

Arrays Application-I

Lab No. 6: Arrays Application-I

Objective:

To implements the concept of one dimensional Array.

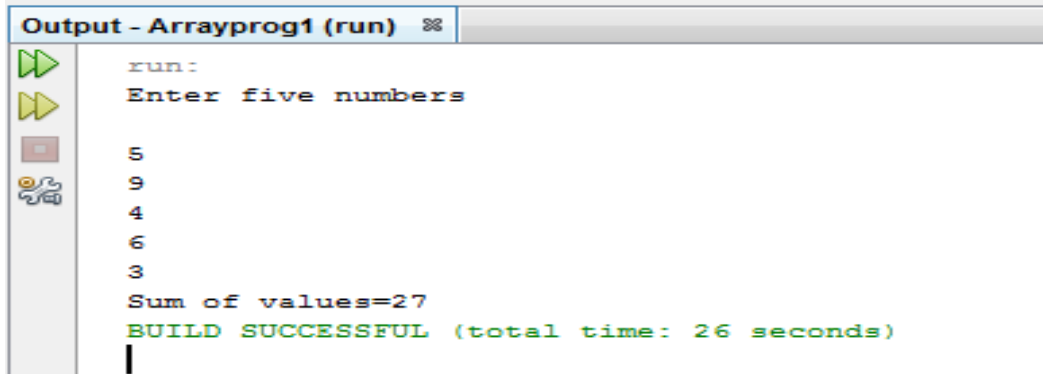
Scope:

Arrays are special data types that let us store specified number of variables from the same type using one variable name. Arrays are an indexed data type, which means they are storing different elements discriminating between them using unique index for each one. Declaring and using an array in Java is similar to declaration of any variable; you have to specify data type and name, in addition to this, you have also to specify the length of the array before using it.

Example 1: This program is used to calculate the sum of array values.

```
public class Arrayprog1 {  
    public static void main(String[] args) {  
        int[] values = new int[5];  
        int i, sum = 0;  
        Scanner input = new Scanner(System.in);  
        System.out.println("Enter five numbers");  
        System.out.println();  
        for (i = 0; i < values.length; i++) {  
            values[i] = input.nextInt();//Entering five numbers  
        }  
        //calculating sum of values  
        for (i = 0; i < values.length; i++) {  
            sum = sum + values[i];  
        }  
        System.out.println("Sum of values=" + sum);  
    }  
}
```





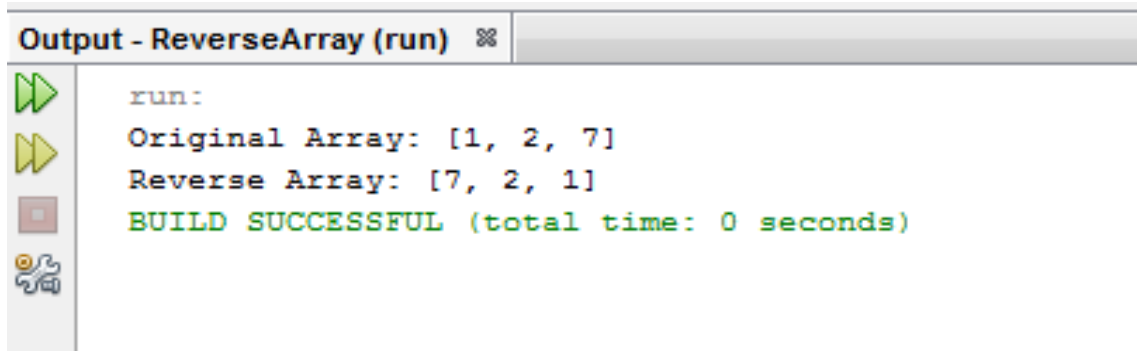
```
Output - Arrayprog1 (run)
run:
Enter five numbers
5
9
4
6
3
Sum of values=27
BUILD SUCCESSFUL (total time: 26 seconds)
```

Example 2: This program demonstrates how to reverse the array values.

```
import java.util.Arrays;
public class ReverseArray {

    public static void main(String[] args)
    {
        int[] data = {1, 2, 7};
        int[] dataR = reverseArray(data);
        System.out.println("Original Array: " + Arrays.toString(data));
        System.out.println("Reverse Array: " + Arrays.toString(dataR));
    }
    public static int[] reverseArray(int[] data)
    {
        int[] reversedData = new int[data.length];
        int i;

        for(i=0; i < data.length; i++)
        {
            reversedData[i] = data[(data.length)-1 - i];
        }
        return reversedData;
    }
}
```



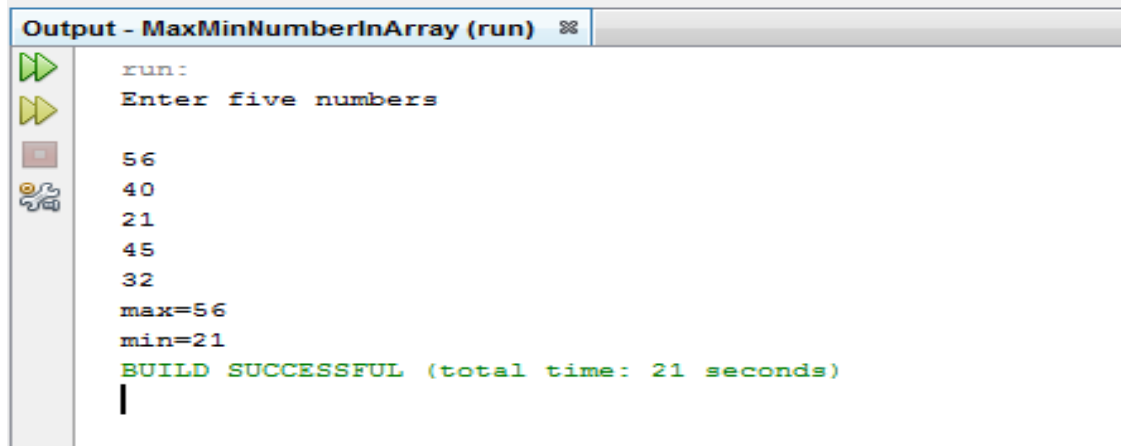
```
Output - ReverseArray (run) %
run:
Original Array: [1, 2, 7]
Reverse Array: [7, 2, 1]
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example 3: This program demonstrates how to find Maximum and minimum value in arrays.

```
import java.util.Scanner;
public class MaxMinNumberInArray {

    public static void main(String[] args) {
        int [] values=new int[5];
        int i,max, min;
        Scanner input=new Scanner(System.in);
        System.out.println("Enter five numbers");
        System.out.println();
        for(i=0;i<values.length;i++)
            values[i]=input.nextInt();//Entering five numbers
        max=values[0];
        min=values[0];
        for(i=0;i<values.length;i++){
            if(max<values[i])
                max=values[i];
            if(min>values[i])
                min=values[i];
        }

        System.out.println("max="+max);
        System.out.println("min="+min);
    }
}
```



```
run:
Enter five numbers
56
40
21
45
32
max=56
min=21
BUILD SUCCESSFUL (total time: 21 seconds)
```

Exercises

Exercise 1:

Write a program using arrays to solve the problem. The problem is to read 10 numbers, get the average of these numbers, and find the number of the items greater than the average.

Exercise 2:

Count the occurrences of each letter in the array. To do so, create an array, say counts, of 26 int values, each of which counts the occurrences of a letter. That is counts[0] counts the number of a's, counts[1] counts the number of b's, and so on.

Exercise 3:

Write a program that reads 10 numbers in an array, write a method which will return the sort array.

Homework:

1. (*Eliminate duplicates*) Write a method that returns a new array by eliminating the duplicate values in the array. Write a JAVA program that reads in at least ten integers, invokes the method, and displays the results.

Lab No. 7

Arrays Application-II

Lab No. 7: Arrays Application - II

Objective:

To implements the concepts of Multidimensional dimensional Arrays.

Scope:

Up to this point, we were storing elements in linear manner, that is, we provide one index to locate the element in one-dimensional structure. Java also provides *arrays of arrays*, this means we create an array, and each element in that array is an array itself. These arrays are also called multidimensional arrays.

Example 1: Two dimensional arrays are common in representing matrices, for example, we can write a program that adds two matrices (remember that matrices must have same number of rows and same number of columns when it comes to addition).

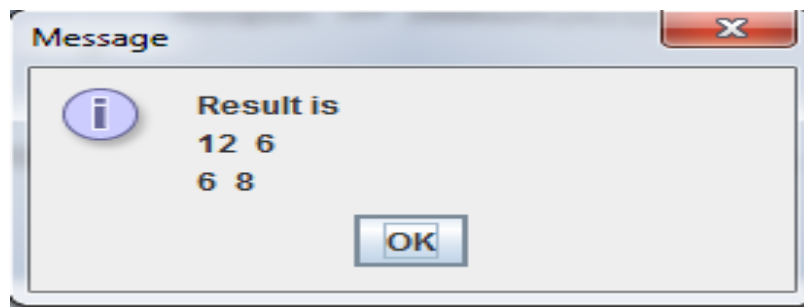
```
public class Matrices {  
  
    public static void main(String[] args) {  
        String input;  
        int rows, columns;  
        input = JOptionPane.showInputDialog("Enter number of rows");  
        rows = Integer.parseInt(input);  
        input = JOptionPane.showInputDialog("Enter number of columns");  
        columns = Integer.parseInt(input);  
        int[][] matrix1 = new int[rows][columns];  
        int[][] matrix2 = new int[rows][columns];  
        String m1 = "Enter first matrix elements\n";  
        String m2 = "Enter second matrix elements\n";  
        for (int i = 0; i < rows; i++) {  
            for (int j = 0; j < columns; j++) {  
                input  
                    = JOptionPane.showInputDialog(m1 + "[" + i + "]" + j + "]");  
                matrix1[i][j] = Integer.parseInt(input);  
            }  
        }  
        for (int i = 0; i < rows; i++) {  
            for (int j = 0; j < columns; j++) {  
                input  
                    = JOptionPane.showInputDialog(m2 + "[" + i + "]" + j + "]");  
                matrix2[i][j] = Integer.parseInt(input);  
            }  
        }  
    }  
}
```



```

    /
    int[][] result = new int[rows][columns];
    String output = "";
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < columns; j++) {
            result[i][j] = matrix1[i][j] + matrix2[i][j];
            output += result[i][j] + " ";
        }
        output += "\n";
    }
    JOptionPane.showMessageDialog(null, "Result is\n" + output);
}
}

```



Example 2: This program demonstrates how to pass Two Dimensional arrays to methods.

```
import java.util.Scanner;

public class PassingMultiArray {

    public static void main(String[] args) {
        int[][] m = getArray(); // Get an array
        // Display sum of elements
        System.out.println("\nSum of all elements is " + sum(m));
    }

    public static int[][] getArray() {
        // Create a Scanner
        Scanner input = new Scanner(System.in);
        // Enter array values
        int[][] m = new int[3][4];
        System.out.println("Enter " + m.length + " rows and " + m[0].length + " columns: ");
        for (int i = 0; i < m.length; i++) {
            for (int j = 0; j < m[i].length; j++) {
                m[i][j] = input.nextInt();
            }
        }
        return m;
    }

    public static int sum(int[][] m) {
        int total = 0;
        for (int row = 0; row < m.length; row++) {
            for (int column = 0; column < m[row].length; column++) {
                total += m[row][column];
            }
        }
        return total;
    }
}
```



```
Output - PassingMultiArray (run) ✖
run:
Enter 3 rows and 4 columns:
5
4
6
7
9
6
5
4
6
8
9
9

Sum of all elements is 78
BUILD SUCCESSFUL (total time: 35 seconds)
```

Exercises

Exercise 1:

Write a program that reads 16 integers from the user, stores them in 5 by 4 matrix, the last row of the matrix should contain the sums of the columns (see figure 9.2) calculated by your program. Then the whole matrix is printed.

User input	4	7	1	8
	3	8	9	5
	1	3	4	5
	8	1	5	6
Sum	16	19	19	24

Exercise 2:

Write a Java program to multiply two matrices of 5x5. Suggest the data.

Exercise 3:

(Largest row and column) Write a program that randomly fills in 0s and 1s into a 4-by-4 matrix, prints the matrix, and finds the first row and column with the most 1's. Here is a sample input/output of the program:



0011

0011

1101

1010

The largest row index: 2

The largest column index: 2

Homework:

1. (Locate the largest element) Write the following method that returns the location of the largest element in a two-dimensional array.

The return value is a one-dimensional array that contains two elements. These two elements indicate the row and column indices of the largest element in the two-dimensional array. Write a test program that prompts the user to enter a two dimensional array and displays the location of the largest element in the array.

Enter the number of rows and columns of the array: 3 4

Enter the array:

23.5 35 2 10

4.5 3 45 3.5

35 44 5.5 9.6

The location of the largest element is at (1, 2)



Lab No. 8

String Handling

Lab No. 8: String Handling

Objective:

To implements the concepts of Strings in Java.

Scope:

Strings, which are widely used in Java programming, are a sequence of characters. In the Java strings are objects and provide the String class to create and manipulate Strings. String class is immutable; so that once it is created a String object cannot be changed.

Example 1: This program demonstrates either an input string is Palindrome or not.

```
import java.util.Scanner;

public class PalindromeClass {

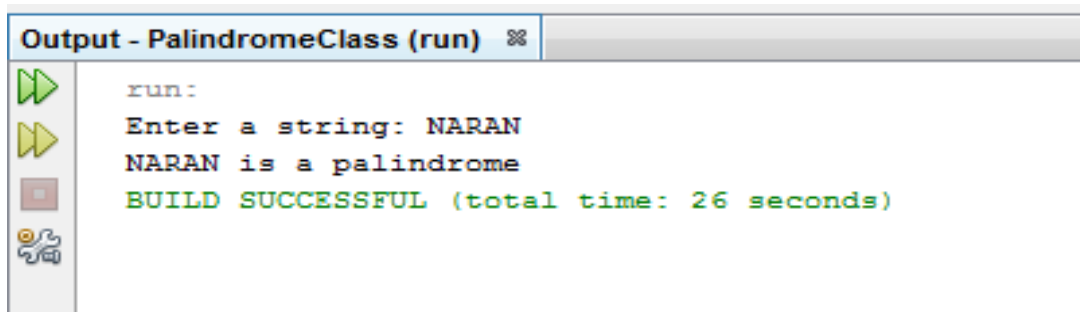
    public static void main(String[] args) {
        // Create a Scanner
        Scanner input = new Scanner(System.in);
        // Prompt the user to enter a string
        System.out.print("Enter a string: ");
        String s = input.nextLine();
        if (isPalindrome(s)) {
            System.out.println(s + " is a palindrome");
        } else {
            System.out.println(s + " is not a palindrome");
        }
    }
}
```



```

        public static boolean isPalindrome(String s) {
// The index of the first character in the string
        int low = 0;
// The index of the last character in the string
        int high = - 1;
        while (low < high) {
            if (s.charAt(low) != s.charAt(high)) {
                return false; // Not a palindrome
            }
            low++;
            high--;
        }
        return true; // The string is a palindrome
    }
}

```



```

run:
Enter a string: NARAN
NARAN is a palindrome
BUILD SUCCESSFUL (total time: 26 seconds)

```

Example 2: The following class, Filename, illustrates the use of `lastIndexOf()` and `substring()` to isolate different parts of a file name.

```

public class FileName {
    private String fullPath;
    private char pathSeparator, extensionSeparator;

    public FileName(String str, char sep, char ext) {
        fullPath = str;
        pathSeparator = sep;
        extensionSeparator = ext;
    }

    public String extension() {
        int dot = fullPath.lastIndexOf(extensionSeparator);
        return fullPath.substring(dot + 1);
    }

    // gets filename without extension
    public String filename() {
        int dot = fullPath.lastIndexOf(extensionSeparator);
        int sep = fullPath.lastIndexOf(pathSeparator);
        return fullPath.substring(sep + 1, dot);
    }

    public String path() {
        int sep = fullPath.lastIndexOf(pathSeparator);
        return fullPath.substring(0, sep);
    }
}

```

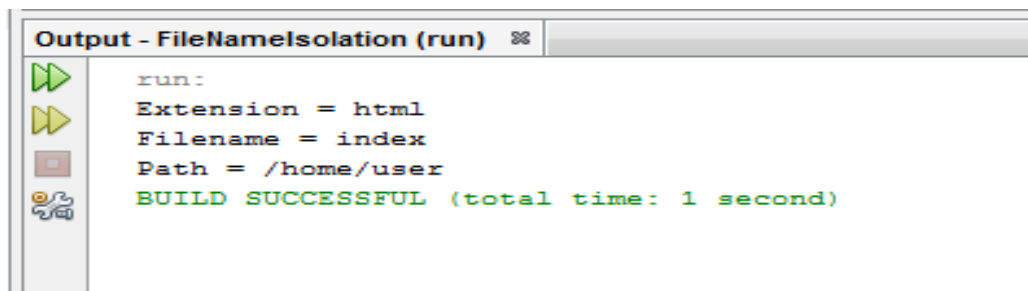


```

public class FileNameIsolation {

    public static void main(String[] args) {
        final String FPATH = "/home/user/index.html";
        FileName myHomePage = new FileName(FPATH, '/', '.');
        System.out.println("Extension = " + myHomePage.extension());
        System.out.println("Filename = " + myHomePage.filename());
        System.out.println("Path = " + myHomePage.path());
    }
}

```



Output - FileNameIsolation (run)

```

run:
Extension = html
Filename = index
Path = /home/user
BUILD SUCCESSFUL (total time: 1 second)

```

Example 3: Following program will sort the names in ascending order

```

public class NameSort {
    public static void main(String[] args) {
        Scanner in = new Scanner (System.in);
        System.out.print ("Enter the number of names you wish to enter: ");
        int n = in.nextInt();
        String names[] = new String [n];
        for (int i = 0; i<names.length; i++)
        {
            System.out.print("Please enter the name: ");
            names[i]= in.nextLine();
        }
        String temp;
        for (int b = n-1; b>=0; b--)
        {
            for (int j=0; j<b; j++)
            {
                if (names[j].compareToIgnoreCase (names[j+1])>0)
                {
                    temp = names[j];
                    names[j] = names[j+1];
                    names[j+1] = temp;
                }
            }
        }
        System.out.println ("The names sorted in alphabetical order are: ");
        for (int a = 0; a<n; a++)
            System.out.print (names[a]+"\\t");
    }
}

```

Output - NameSort (run) ☒

```

run:
Enter the number of names you wish to enter: 5
Please enter the name: Please enter the name: swati
Please enter the name: zubair
Please enter the name: ali
Please enter the name: mazhar
The names sorted in alphabetical order are:
    ali    mazhar  swati  zubair  BUILD SUCCESSFUL (total time: 24 seconds)

```

Exercises:

Exercise 1:

The problem is to write a program that prompts the user to enter a string and counts the number of occurrences of each letter in the string regardless of case.



Exercise 2:

(Checking substrings) You can check whether a string is a substring of another string by using the `indexOf` method in the `String` class. Write your own method for this function. Write a program that prompts the user to enter two strings, and check whether the first string is a substring of the second.

Exercise 3:

(Counting the letters, digits, consonants and vowels in a string) Write a method that counts the number of letters, digits, consonants and vowels in a string.

Write a test program that prompts the user to enter a string and displays the number of letters, digits, consonants and vowels in the string.

Homework:

(Checking password) Some Websites impose certain rules for passwords. Write a method that checks whether a string is a valid password. Suppose the password rule is as follows:

- A password must have at least eight characters.
- A password consists of only letters and digits.
- A password must contain at least two digits.

Write a program that prompts the user to enter a password and displays "Valid Password" if the rule is followed or "Invalid Password" otherwise.



Lab No. 9

Inheritance

Lab No. 9: Inheritance

Objective:

To implements the concepts of Inheritance.

.Scope:

Inheritance enables you to define a general class (i.e., a superclass) and later extend it to more specialized classes (i.e., subclasses). You use a class to model objects of the same type. Different classes may have some common properties and behaviors, which can be generalized in a class that can be shared by other Classes. You can define a specialized class that extends the generalized class. The specialized classes inherit the properties and methods from the general class.

Example 1: This program demonstrates a generalized super class i.e. SimpleGeometricObject and a Specialized subclass CircleFromGeometricObject. All geometric objects such as a circle or a rectangle may have the property of color but only circles has radius. The sub class has therefore a data member radius but also accesses the data member color of the superclass. Furthermore the setColor() and getColor() methods of the superclass are also accessed by the subclass.



```

public class TestCircleRectangle {
    public static void main(String[] args) {
        CircleFromSimpleGeometricObject circle= new CircleFromSimpleGeometricObject(2.5, "Red");
        System.out.println(circle.getcolor());
    }
}

class SimpleGeometricObject {
    private String color = "white";
    public SimpleGeometricObject(){
    }
    /* Set color */
    public void setColor(String color) {
        this.color=color;
    }
    /* Return Color */
    public String getcolor() {
        return color;
    }
}

class CircleFromSimpleGeometricObject extends SimpleGeometricObject{
    private double radius;
    public CircleFromSimpleGeometricObject(double radius,
    String color) {
        this.radius = radius;
        setColor(color);
    }
}

```

Example 2: This program demonstrates inheritance with four classes i.e. EmployeeTest, BasePlusCommissionEmployee, CommissionEmployee and Employee respectively. EmployeeTest is the main class with the main method. BasePlusCommissionEmployee is the subclass of CommissionEmployee. CommissionEmployee is the subclass of Employee class. An object of the BasePlusCommissionEmployee subclass is made in the main method of EmployeeTest class. Since BasePlusCommissionEmployee is the base class of all classes in this example therefore its object can access its members as well as members of its super classes.



```

public class EmployeeTest {
    public static void main(String[] args) {
        // instantiate BasePlusCommissionEmployee object
        BasePlusCommissionEmployee employee =
            new BasePlusCommissionEmployee(
                "Ali", "Khan", "333-33-3333", 5000, .01, 100 );

        // get base-salaried commission employee data
        System.out.println(
            "Employee information obtained by get methods: \n" );
        System.out.printf( "%s %s\n", "First name is",
            employee.getFirstName() );
        System.out.printf( "%s %s\n", "Last name is",
            employee.getLastName() );
        System.out.printf( "%s %s\n", "Social security number is",
            employee.getSocialSecurityNumber() );
        System.out.printf( "%s %.2f\n", "Gross sales is",
            employee.getGrossSales() );
        System.out.printf( "%s %.2f\n", "Commission rate is",
            employee.getCommissionRate() );
        System.out.printf( "%s %.2f\n", "Base salary is",
            employee.getBaseSalary() );
        System.out.printf( "%s %.2f\n", "Earning is",
            employee.earnings() );

        //////////////////////////////////////
    }
}

```

```

try{
    employee.setBaseSalary( 200 );
    employee.setFirstName("Waqas");
    employee.setLastName("Jadoon");
    employee.setSocialSecurityNumber("13101-0883993-5");
    employee.setBaseSalary( -200 );// set base salary
}
catch(IllegalArgumentException e)
{
    System.out.printf("%s :%S \n","Please Enter valid value:",e.getMessage());
}

System.out.printf( "\n%s:\n\n%s\n",
    "Updated employee information obtained by toString",
    employee.toString() );
System.out.printf( "%s %.2f", "Earning is",
    employee.earnings());
} // end main

```

```

class BasePlusCommissionEmployee extends commissionEmployee
{
    private double baseSalary; // base salary per week

    // six-argument constructor
    public BasePlusCommissionEmployee( String first, String last,
        String ssn, double sales, double rate, double salary )
    {
        super( first, last, ssn, sales, rate );
        setBaseSalary( salary );
        // validate and store base salary
    } // end six-argument BasePlusCommissionEmployee constructor

    // set base salary
    public void setBaseSalary( double salary )
    {
        if ( salary >= 0.0 )
            baseSalary = salary;
        else
            throw new IllegalArgumentException(
                "Base salary must be >= 0.0" );
    } // end method setBaseSalary

    // return base salary
    public double getBaseSalary()
    {
        return baseSalary;
    } // end method getBaseSalary

    // calculate earnings
    @Override // indicates that this method overrides a superclass method
    public double earnings()
    {
        //return getBaseSalary()+grossSales*commissionRate;
        return getBaseSalary() + super.earnings();
    } // end method earnings

    // return String representation of BasePlusCommissionEmployee
    @Override // indicates that this method overrides a superclass method
    public String toString()
    {
        return String.format( "%s %s\n%s: %.2f", "base-salaried",
            super.toString(), "base salary", getBaseSalary() );
    } // end method toString
} // end class BasePlusCommissionEmployee

```



```

class commissionEmployee extends Employee
{
    private double grossSales; // gross weekly sales
    private double commissionRate; // commission percentage

    // five-argument constructor
    public commissionEmployee( String first, String last, String ssn,
        double sales, double rate )
    {
        super(first,last,ssn);
        setGrossSales( sales ); // validate and store gross sales
        setCommissionRate( rate ); // validate and store commission rate
    } // end five-argument CommissionEmployee constructor

    // set gross sales amount
    public void setGrossSales( double sales )
    {
        if ( sales >= 0.0 )
            grossSales = sales;
        else
            throw new IllegalArgumentException(
                "Gross sales must be >= 0.0" );
    } // end method setGrossSales

    // return gross sales amount

```



```

public double getGrossSales()
{
    return grossSales;
} // end method getGrossSales

// set commission rate
public void setCommissionRate( double rate )
{
    if ( rate > 0.0 && rate < 1.0 )
        commissionRate = rate;
    else
        throw new IllegalArgumentException(
            "Commission rate must be > 0.0 and < 1.0" );
} // end method setCommissionRate

// return commission rate
public double getCommissionRate()
{
    return commissionRate;
} // end method getCommissionRate

// calculate earnings
@Override
public double earnings()
{
    return getCommissionRate() * getGrossSales();
} // end method earnings

```

```

// return String representation of CommissionEmployee object
@Override // indicates that this method overrides a superclass method
public String toString()
{
    return String.format( "%s: %s\n%s: %.2f\n%s: %.2f",
        "commission employee", super.toString(),
        "gross sales", getGrossSales(),
        "commission rate", getCommissionRate());
} // end method toString
} // end class CommissionEmployee

abstract class Employee {
    private String firstName;
    private String lastName;
    private String socialSecurityNumber;
    public Employee(String fname,String lname, String ssn)
    {
        firstName=fname;
        lastName=lname;
        socialSecurityNumber=ssn;
    }
    // set first name
    public void setFirstName( String first )
    {
        firstName = first; // should validate
    } // end method setFirstName
}

```

```

// return first name
public String getFirstName()
{
    return firstName;
} // end method getFirstName

// set last name
public void setLastName( String last )
{
    lastName = last; // should validate
} // end method setLastName

// return last name
public String getLastName()
{
    return lastName;
} // end method getLastName

// set social security number
public void setSocialSecurityNumber( String ssn )
{
    socialSecurityNumber = ssn; // should validate
} // end method setSocialSecurityNumber

// return social security number
public String getSocialSecurityNumber()
{
    return socialSecurityNumber;
} // end method getSocialSecurityNumber

// return String representation of CommissionEmployee object
@Override // indicates that this method overrides a superclass method
public String toString()
{
    return String.format( "%s %s\n%s:",
        getFirstName(), getLastName(),
        "social security number", getSocialSecurityNumber());
} // end method

public abstract double earnings();
}

```

The outputs are as follows



```
Output - EmployeeTest (run)

run:
Employee information obtained by get methods:

First name is Ali
Last name is Khan
Social security number is 333-33-3333
Gross sales is 5000.00
Commission rate is 0.01
Base salary is 100.00
Earning is 150.00
Please Enter valid value: :BASE SALARY MUST BE >= 0.0

Updated employee information obtained by toString:

base-salaried commission employee: Waqas Jadoon
social security number:
gross sales: 5000.00
commission rate: 0.01
base salary: 200.00
Earning is 250.00BUILD SUCCESSFUL (total time: 0 seconds)
```

Exercises:

Exercise 1:

Create a new class `RectangleFromSimpleGeometricObject` in example 1 and extend it with `SimpleGeometricObject` class. Create an object for this class in the `TestCircleRectangle` class with a constructor receiving one parameter of color . Add a method `setWidthHeight()` which accepts two double parameters width and height. Add another method `getWidthHeight` which returns width and height. Also call the `setColor()` method of the superclass from the constructor and call the `getColor()` method in `TestCircleRectangle` class for `RectangleFromSimpleGeometricObject` class.

Exercise 2:

Create a new method `CalculateCommission` in the `CommissionEmployee` class. This method calculates the commission based on the value of `commissionRate` data member.

Homework

Create a new class `SalariedEmployee` that extends from class `Employee` and overrides the method `earnings()` of class `Employee` to calculate the fixed monthly salary of each employee. Create object of `SalariedEmployee` class in `EmployeeTest` class(i.e, main class) to set the record for new employee and print it using `toString()` method.



Lab No. 10

Inheritance, overriding and constructor chaining

Lab No. 10: Inheritance, Overriding and Constructor chaining

Objective:

To implements the concepts of Inheritance, overriding and constructor chaining.

Scope:

Inheritance enables specialized classes (i.e., subclasses) to access members of a general class (i.e., a superclass) which can be controlled in many ways using techniques like constructor chaining or overriding.

Example 1: This program demonstrates inheritance with four classes i.e. TestCircleRectangle, SimpleGeometricObject, CircleFromSimpleGeometricObject and RectangleFromSimpleGeometricObject respectively. TestCircleRectangle is the main class with the main method. SimpleGeometricObject is the superclass of CircleFromSimpleGeometricObject and RectangleFromSimpleGeometricObject.

An object each for CircleFromSimpleGeometricObject and RectangleFromSimpleGeometricObject is made in the main method of TestCircleRectangle class. Both of these objects are then used to call their members as well as members of the base class SimpleGeometricObject.

```
public class TestCircleRectangle {
    public static void main(String[] args) {
        CircleFromSimpleGeometricObject circle =
            new CircleFromSimpleGeometricObject(1);
        System.out.println("A circle " + circle.toString());
        System.out.println("The color is " + circle.getColor());
        System.out.println("The radius is " + circle.getRadius());
        System.out.println("The area is " + circle.getArea());
        System.out.println("The diameter is " + circle.getDiameter());

        RectangleFromSimpleGeometricObject rectangle =
            new RectangleFromSimpleGeometricObject(2, 4);
        System.out.println("\nA rectangle " + rectangle.toString());
        System.out.println("The area is " + rectangle.getArea());
        System.out.println("The perimeter is " +
            rectangle.getPerimeter());
    }
}
```



```

class SimpleGeometricObject {
private String color = "white";
private boolean filled;
private java.util.Date dateCreated;

/** Construct a default geometric object */
public SimpleGeometricObject() {
dateCreated = new java.util.Date();
}
/** Construct a geometric object with the specified color
 * and filled value */
public SimpleGeometricObject(String color, boolean filled) {
dateCreated = new java.util.Date();
this.color = color;
this.filled = filled;
}

/** Return color */
public String getColor() {
return color;
}

/** Set a new color */
public void setColor(String color) {
this.color = color;
}
}

```

```

/** Return filled. Since filled is boolean,
its getter method is named isFilled */
public boolean isFilled() {
    return filled;
}

/** Set a new filled */
public void setFilled(boolean filled) {
    this.filled = filled;
}

/** Get dateCreated */
public java.util.Date getDateCreated() {
    return dateCreated;
}

/** Return a string representation of this object */
public String toString() {
    return "created on " + dateCreated + "\ncolor: " + color +
    " and filled: " + filled;
}
}

```

```

class CircleFromSimpleGeometricObject
    extends SimpleGeometricObject
{
    private double radius;

    public CircleFromSimpleGeometricObject() {
    }

    public CircleFromSimpleGeometricObject(double radius) {
        this.radius = radius;
    }

    public CircleFromSimpleGeometricObject(double radius,
        String color, boolean filled) {
        this.radius = radius;
        setColor(color);
        setFilled(filled);
    }

    /** Return radius */
    public double getRadius() {
        return radius;
    }

    /** Set a new radius */
    public void setRadius(double radius) {
        this.radius = radius;
    }
}

```



```
public double getArea() {  
    return radius * radius * Math.PI;  
}  
  
/** Return diameter */  
public double getDiameter() {  
    return 2 * radius;  
}  
  
/** Return perimeter */  
public double getPerimeter() {  
    return 2 * radius * Math.PI;  
}  
  
/** Print the circle info */  
public void printCircle() {  
    System.out.println("The circle is created " + getDateCreated() +  
        " and the radius is " + radius);  
}  
}
```

```

class RectangleFromSimpleGeometricObject
    extends SimpleGeometricObject {
    private double width;
    private double height;

    public RectangleFromSimpleGeometricObject() {
    }

    public RectangleFromSimpleGeometricObject(
        double width, double height) {
        this.width = width;
        this.height = height;
    }

    public RectangleFromSimpleGeometricObject(
        double width, double height, String color, boolean filled) {
        this.width = width;
        this.height = height;
        setColor(color);
        setFilled(filled);
    }

    /** Return width */
    public double getWidth() {
        return width;
    }

    /** Set a new width */
    public void setWidth(double width) {
        this.width = width;
    }
}

```

```

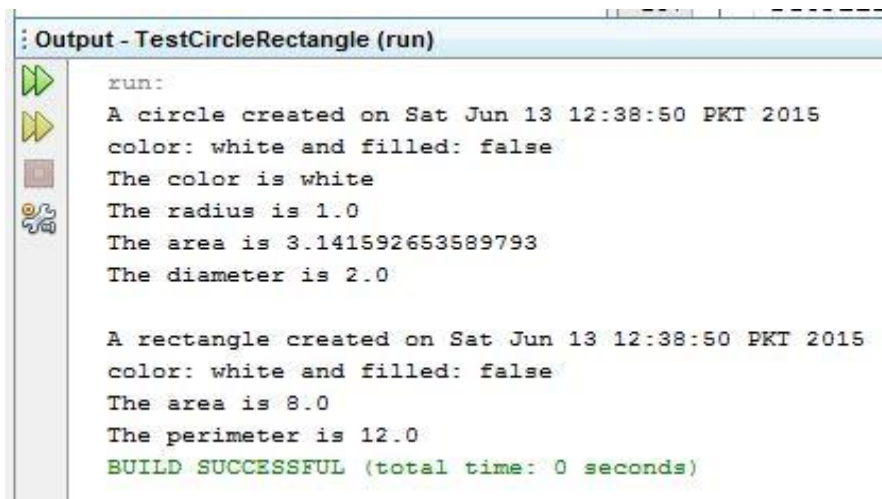
/** Return height */
public double getHeight() {
    return height;
}

/** Set a new height */
public void setHeight(double height) {
    this.height = height;
}

/** Return area */
public double getArea() {
    return width * height;
}

/** Return perimeter */
public double getPerimeter() {
    return 2 * (width + height);
}
}

```



```

Output - TestCircleRectangle (run)
run:
A circle created on Sat Jun 13 12:38:50 PKT 2015
color: white and filled: false
The color is white
The radius is 1.0
The area is 3.141592653589793
The diameter is 2.0

A rectangle created on Sat Jun 13 12:38:50 PKT 2015
color: white and filled: false
The area is 8.0
The perimeter is 12.0
BUILD SUCCESSFUL (total time: 0 seconds)

```

Example 2: This program demonstrates overriding in inheritance. To override a method, the method must be defined in the subclass using the same signature and the same return type as in its superclass. Overriding means to provide a new implementation for a method in the subclass. In the following program the method `name()` of the `Employee` class overrides the method `name()` of the `Person` class

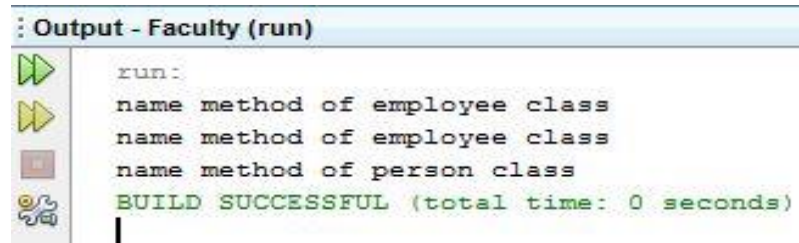
```

public class Faculty extends Employee {
    public static void main(String[] args) {
        Faculty obj_Faculty= new Faculty();
        obj_Faculty.name();
        Employee obj_Employee= new Employee();
        obj_Employee.name();
        Person obj_Person= new Person();
        obj_Person.name();
    }
}

class Employee extends Person {
    public void name() {
        System.out.println("name method of employee class");
    }
}

class Person {
    public void name() {
        System.out.println("name method of person class");
    }
}

```



```

: Output - Faculty (run)
run:
name method of employee class
name method of employee class
name method of person class
BUILD SUCCESSFUL (total time: 0 seconds)

```

Example 3: Whenever an object of a class is created the constructor is called. In case of inheritance whenever a constructor of a subclass is called, for example object creation, the constructor of the superclass executes first and then the subclass constructor executes. This phenomenon is called constructor chaining. The following program demonstrates constructor chaining.

```

public class Faculty extends Employee {
    public static void main(String[] args) {
        new Faculty();
    }

    public Faculty() {
        System.out.println("(4) Performs Faculty's tasks");
    }
}

class Employee extends Person {
    public Employee() {
        this("(2) Invoke Employee's overloaded constructor");
        System.out.println("(3) Performs Employee's tasks ");
    }

    public Employee(String s) {
        System.out.println(s);
    }
}

class Person {
    public Person() {
        System.out.println("(1) Performs Person's tasks");
    }
}

```

Output - Faculty (run)

```

run:
(1) Performs Person's tasks
(2) Invoke Employee's overloaded constructor
(3) Performs Employee's tasks
(4) Performs Faculty's tasks
BUILD SUCCESSFUL (total time: 0 seconds)

```

Exercises:



Exercise 1:

Consider the program from Example1. Create a `toString()` method in both subclasses `RectangleFromSimpleGeometricObject` and `CircleFromSimpleGeometricObject` such that these two methods in subclasses overrides the `toString()` method in the superclass `SimpleGeometricObject`. Call the `toString()` method in the `main` method of `TestCircleRectangle` class from both objects of the subclasses.

Exercise 2:

Create three classes `Food`, `Fruit` and `Apple` in such a way that class `Apple` extends `Fruit` class and `Fruit` class extends class `Food`. Create a constructor in each class containing only a statement to print current class name. Call all the three constructors using constructor chaining. For reference use Example 3.

Homework:

Add a new method `drawShape()` in `SimpleGeometricObject` (i.e., superclass) Class and overrides this method into subclass i.e., `RectangleFromSimpleGeometricObject` and `CircleFromGeometricObject` to draw a specific shape according to subclass specification.



Lab No. 11
Polymorphism -I

Lab No. 11: Polymorphism-I

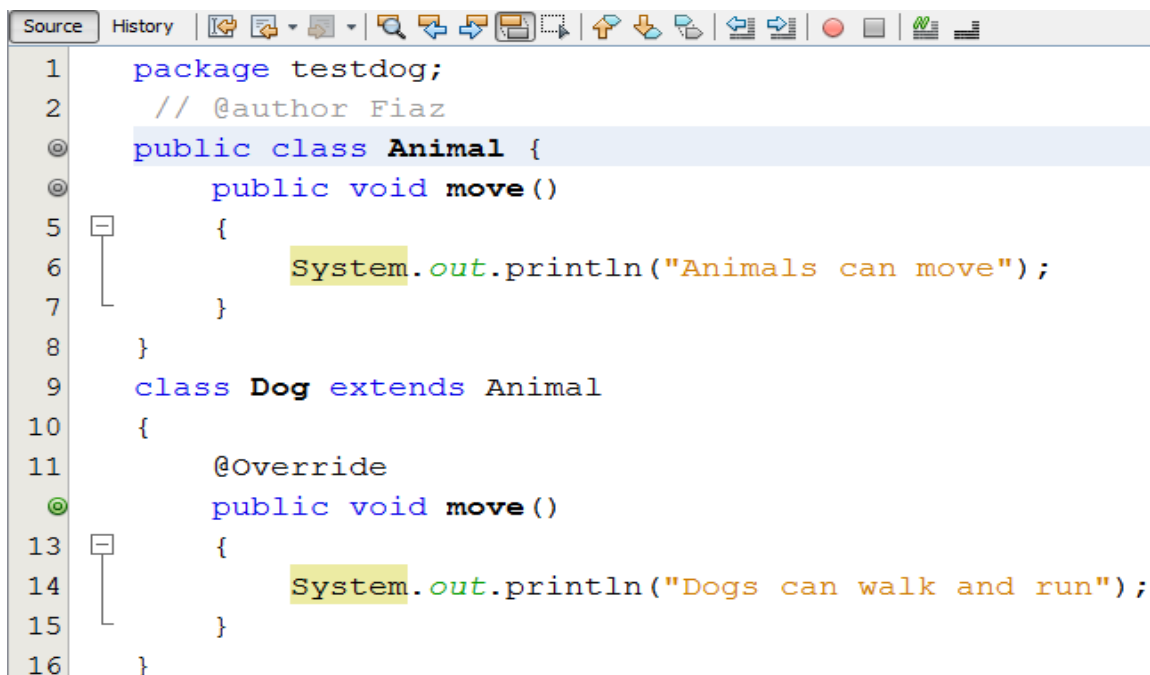
Objective:

In these labs we will discuss and implement method overriding and polymorphism, which are one of the key features of Object oriented programming (OOPs). Moreover the importance of these concepts would be seen in problem solving in Java.

Scope:

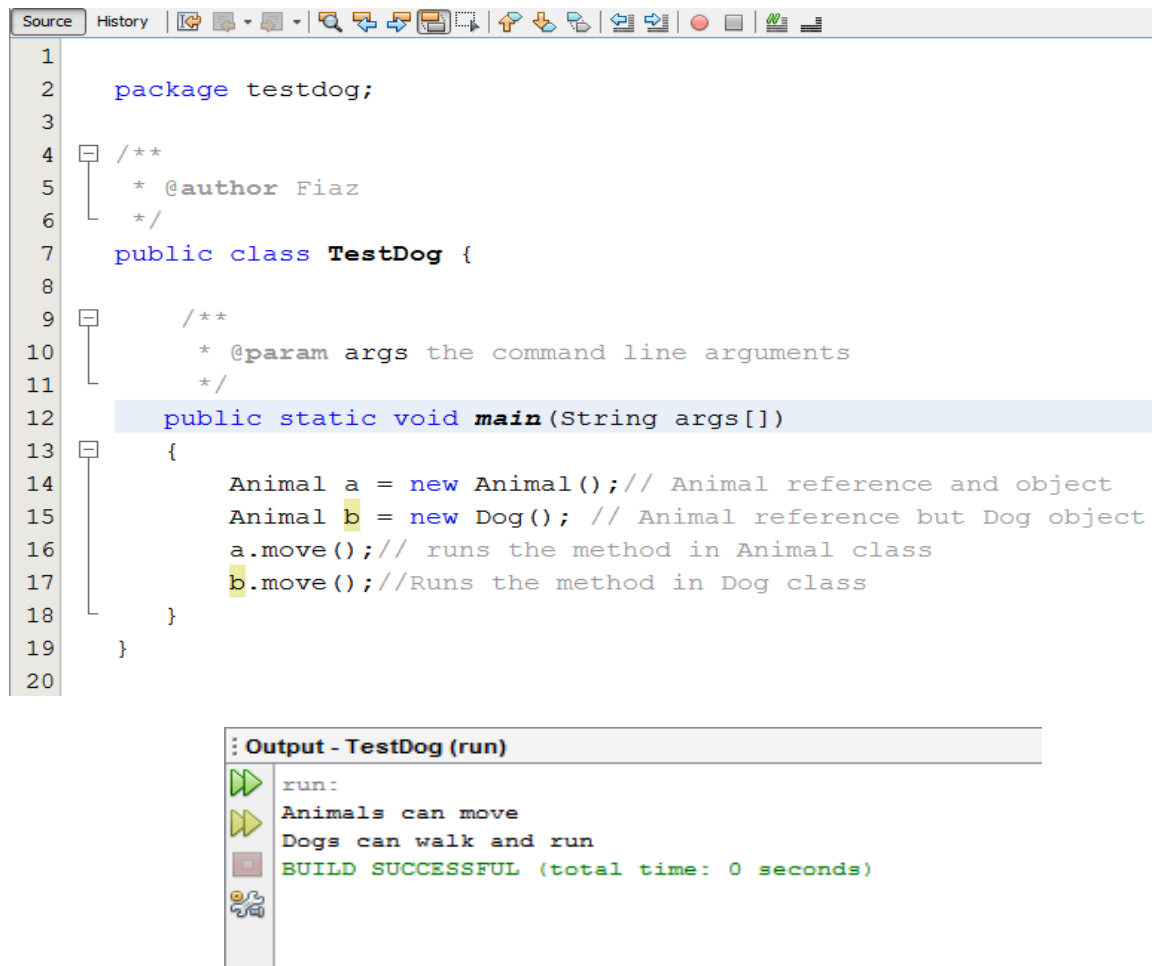
In inheritance we have seen the concept of super classes and sub classes. If a class inherits a method from its super class, then there is a chance to override the method provided that it is not marked final. The benefit of overriding is: ability to define a behavior that's specific to the sub class type. Which means a sub-class can implement a parent class method based on its requirement. In object oriented terms, overriding means to override the functionality of any existing method.

Example 1: Let us look at an example.



```
1 package testdog;
2 // @author Fiaz
3 public class Animal {
4     public void move ()
5     {
6         System.out.println("Animals can move");
7     }
8 }
9 class Dog extends Animal
10 {
11     @Override
12     public void move ()
13     {
14         System.out.println("Dogs can walk and run");
15     }
16 }
```





The screenshot shows an IDE with a source editor and an output console. The source editor contains the following Java code:

```
1
2 package testdog;
3
4 /**
5  * @author Fiaz
6  */
7 public class TestDog {
8
9     /**
10    * @param args the command line arguments
11    */
12    public static void main(String args[])
13    {
14        Animal a = new Animal(); // Animal reference and object
15        Animal b = new Dog(); // Animal reference but Dog object
16        a.move(); // runs the method in Animal class
17        b.move(); //Runs the method in Dog class
18    }
19 }
20
```

The output console, titled "Output - TestDog (run)", shows the following output:

```
run:
Animals can move
Dogs can walk and run
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example 2: Consider the following example

```
Source History
1 package testdog;
2 // @author Fiaz
3 public class Animal {
4     public void move ()
5     {
6         System.out.println("Animals can move");
7     }
8 }
9 class Dog extends Animal
10 {
11     @Override
12     public void move ()
13     {
14         System.out.println("Dogs can walk and run");
15     }
16     public void bark ()
17     {
18         System.out.println("Dogs can bark");
19     }
20 }
```

```
Source History
1 package testdog;
2
3 /**
4  * @author Fiaz
5  */
6 public class TestDog {
7
8     /**
9      * @param args the command line arguments
10     */
11     public static void main(String args[])
12     {
13         Animal a = new Animal(); // Animal reference and object
14         Animal b = new Dog(); // Animal reference but Dog object
15         a.move(); // runs the method in Animal class
16         b.move(); // Runs the method in Dog class
17         b.bark();
18     }
19 }
20
```

```
Output - TestDog (run)
run:
Animals can move
Dogs can walk and run
Exception in thread "main" java.lang.RuntimeException: Uncompilable source code - Erroneous sym type: testdog.Animal.bark
    at testdog.TestDog.main(TestDog.java:17)
Java Result: 1
BUILD SUCCESSFUL (total time: 2 seconds)
```

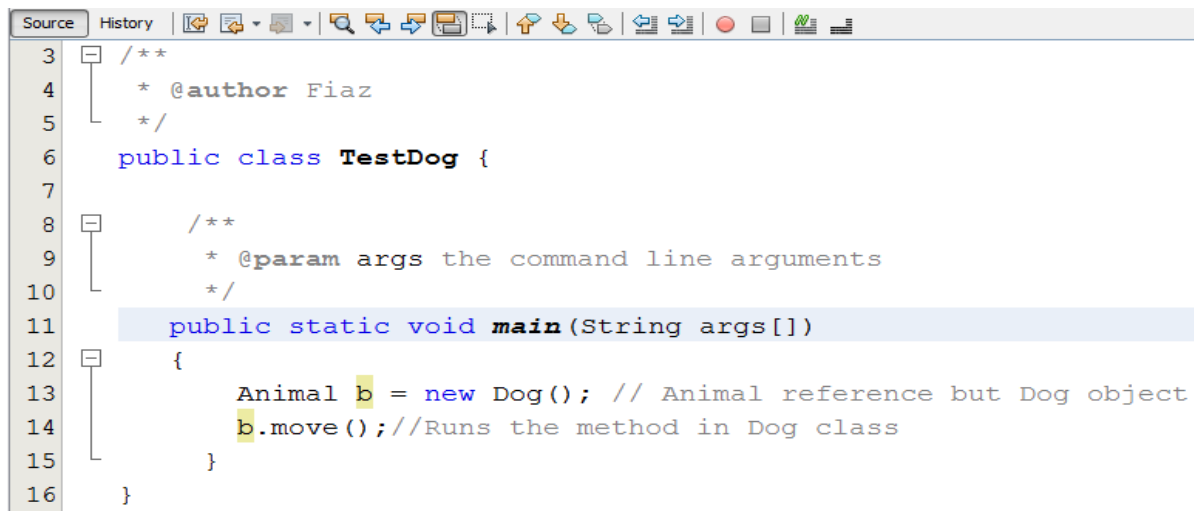
This program will throw a **compile time error** since **b's** reference type **Animal** doesn't have a method by the name of **bark**.

Using the super keyword:

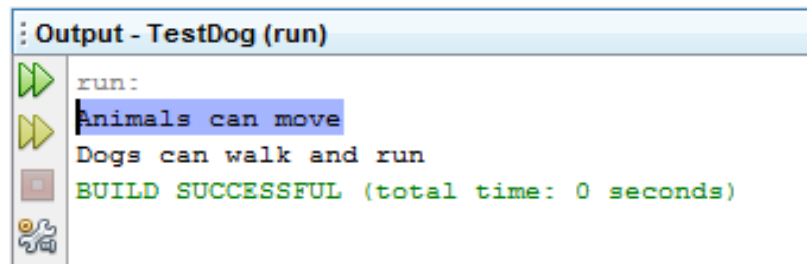
When invoking a superclass version of an overridden method the **super** keyword is used. Consider the following example:

```
Source History
1 package testdog;
2 // @author Fiaz
3 public class Animal {
4     public void move ()
5     {
6         System.out.println("Animals can move");
7     }
8 }
9 class Dog extends Animal
10 {
11     @Override
12     public void move ()
13     {
14         super.move(); // invokes the super class method move
15         System.out.println("Dogs can walk and run");
16     }
17 }
```





```
3  /**
4   * @author Fiaz
5   */
6  public class TestDog {
7
8      /**
9       * @param args the command line arguments
10      */
11     public static void main(String args[])
12     {
13         Animal b = new Dog(); // Animal reference but Dog object
14         b.move(); //Runs the method in Dog class
15     }
16 }
```



```
Output - TestDog (run)
run:
Animals can move
Dogs can walk and run
BUILD SUCCESSFUL (total time: 0 seconds)
```

Lab No. 12

Polymorphism -II

Lab No. 12: Polymorphism-II

Objective:

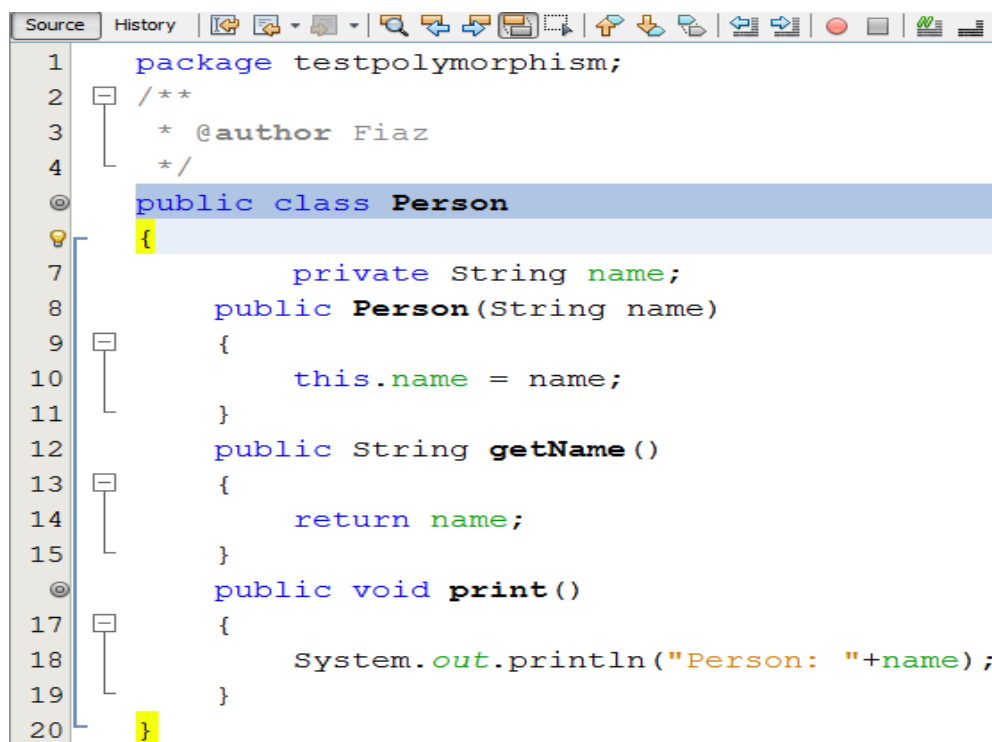
To implement the concept of Polymorphism

Scope:

Polymorphism is the ability of an object to take on many forms, it is a Greek word where poly means many and morphism means form. The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object.

It is important to know that the only possible way to access an object is through a reference variable. A reference variable can be of only one type. Once declared the type of a reference variable cannot be changed. The reference variable can be reassigned to other objects provided that it is not declared final. The type of the reference variable would determine the methods that it can invoke on the object. A reference variable can refer to any object of its declared type or any subtype of its declared type. A reference variable can be declared as a class or interface type

Example 1: consider the following classes:

A screenshot of an IDE window showing a Java source file. The code defines a package 'testpolymorphism' and a class 'Person'. The class has a private String attribute 'name', a constructor 'Person(String name)' that initializes 'name', a 'getName()' method that returns 'name', and a 'print()' method that prints the name. The code is as follows:

```
1 package testpolymorphism;
2 /**
3  * @author Fiaz
4  */
5 public class Person
6 {
7     private String name;
8     public Person(String name)
9     {
10         this.name = name;
11     }
12     public String getName()
13     {
14         return name;
15     }
16     public void print()
17     {
18         System.out.println("Person: "+name);
19     }
20 }
```

```
Source History
1 package testpolymorphism;
2 public class Instructor extends Person
3 {
4     private double salary;
5     public Instructor(String name, double salary){
6         super(name);
7         this.salary = salary;
8     }
9     public double getSalary() {
10         return salary;
11     }
12 @Override
13 public void print() {
14     System.out.println("Instructor: "+getName()+ ", "+salary);
15 }
16 class Student extends Person {
17     private double gpa;
18     public Student(String name, double gpa) {
19         super(name);
20         this.gpa = gpa;
21     }
22     public double getGPA() {
23         return gpa;
24     }
25 @Override
26 public void print() {
27     System.out.println("Student: "+getName()+ ", "+gpa);
28 }
29 }
```

Now because of inheritance, it is possible to write a single method that can accept an object of any of the above classes as parameter and call the print method polymorphically like:

```

1  package testpolymorphism;
2  /**
3   * @author Fiaz
4   */
5  public class TestPolymorphism {
6
7      /**
8       * @param args the command line arguments
9       */
10     public static void main(String[] args) {
11         // TODO code application logic here
12         Person p = new Person("Aamir");
13         Instructor i = new Instructor("Fiaz", 9000);
14         Student s = new Student("Shahid", 3.5);
15         describe(p);
16         describe(i);
17         describe(s);
18     }
19     public static void describe(Person p) {
20         p.print();
21     }
22 }
23

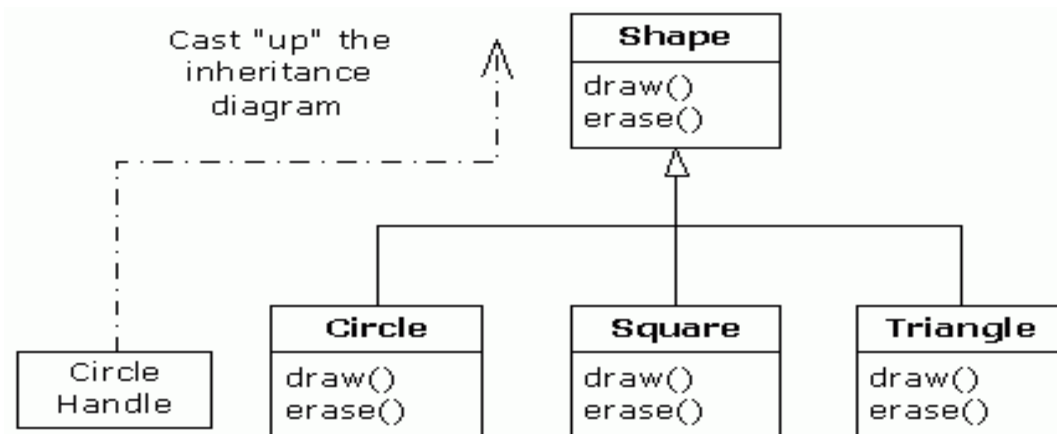
```

```

:: Output - TestPolymorphism (run)
run:
Person: Aamir
Instructor: Fiaz, 9000.0
Student: Shahid, 3.5
BUILD SUCCESSFUL (total time: 0 seconds)

```

Example 2: Consider the following inheritance diagram:

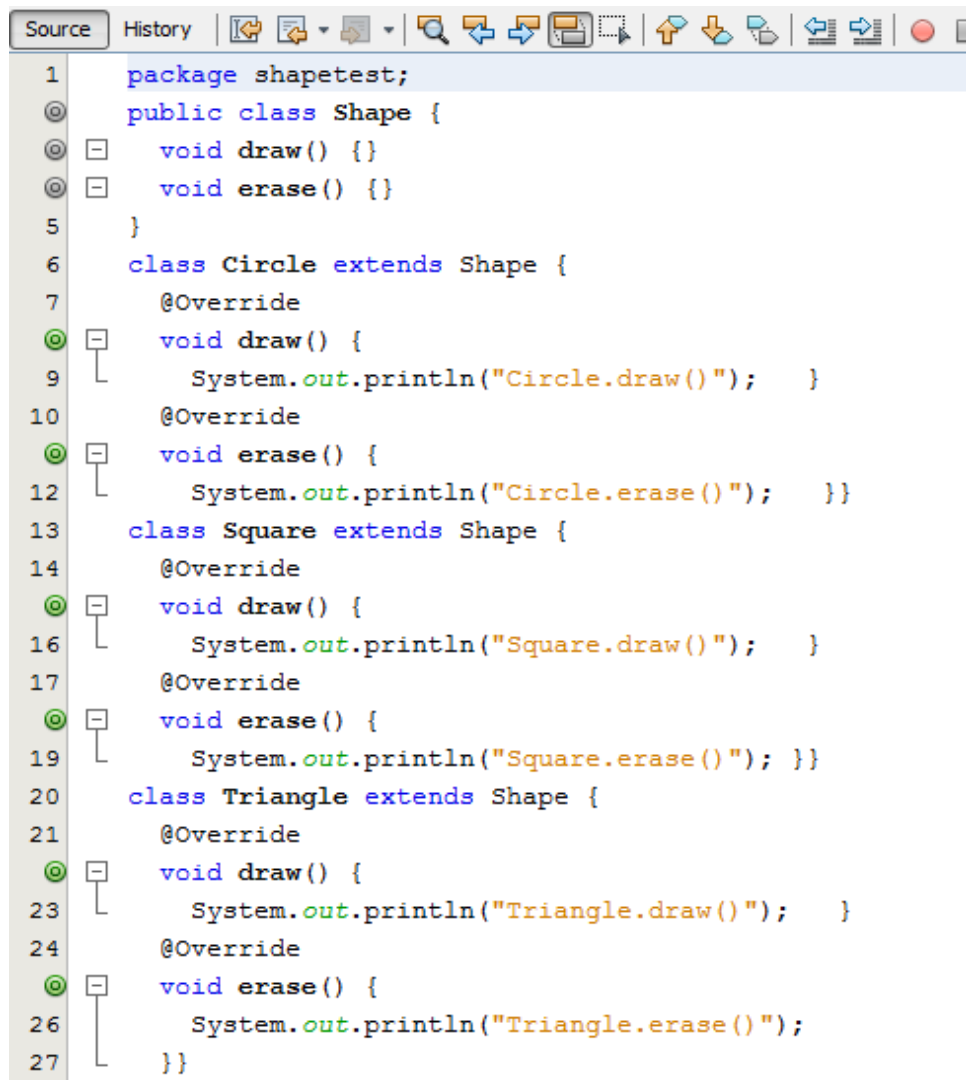


- ❖ Suppose we have the following:

```
Shape s = new Circle();  
s.draw();
```

- ❖ The question is which draw() method will be called?
- ❖ Because of polymorphism, the draw() of the actual object (not necessarily that of the reference type) will be called. Thus, it is the draw() method of the Circle object that is called.

Here is the demonstration of this idea

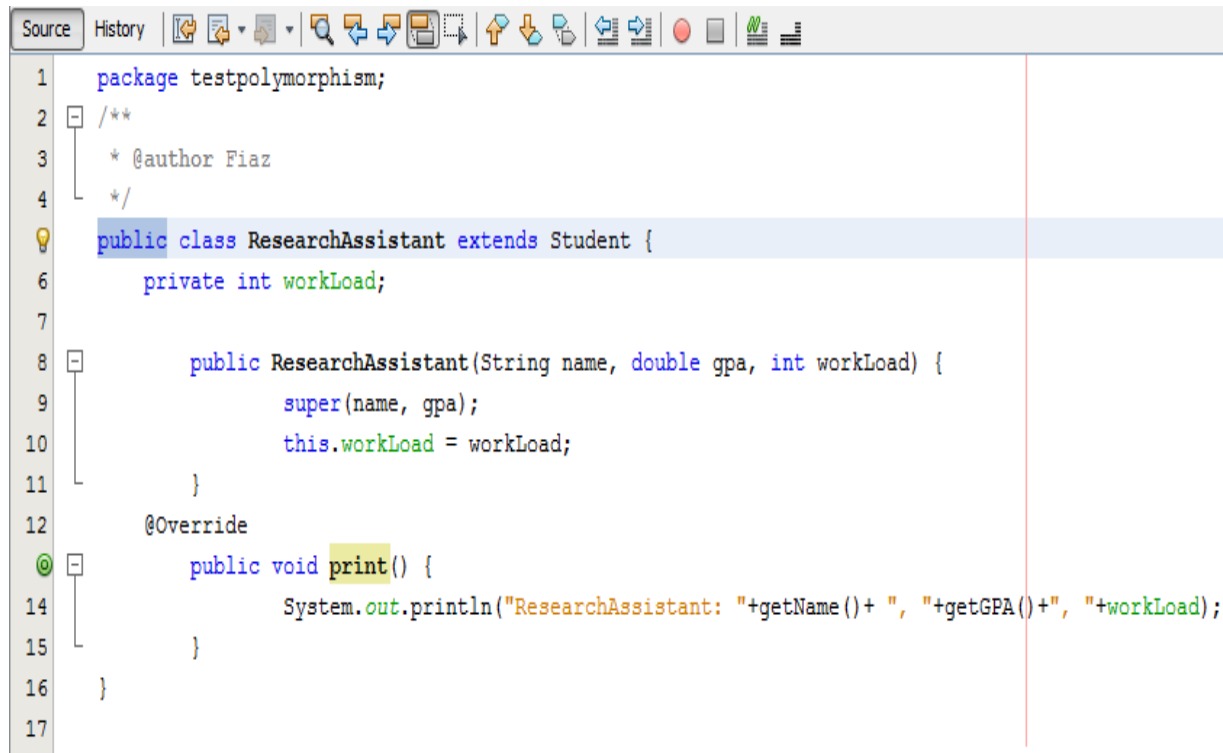
A screenshot of an IDE window showing Java code. The window has a title bar with 'Source' and 'History' tabs. The code is as follows:

```
1 package shapetest;  
2 public class Shape {  
3     void draw() {}  
4     void erase() {}  
5 }  
6 class Circle extends Shape {  
7     @Override  
8     void draw() {  
9         System.out.println("Circle.draw()");    }  
10    @Override  
11    void erase() {  
12        System.out.println("Circle.erase()");    }}  
13 class Square extends Shape {  
14    @Override  
15    void draw() {  
16        System.out.println("Square.draw()");    }  
17    @Override  
18    void erase() {  
19        System.out.println("Square.erase()"); }}  
20 class Triangle extends Shape {  
21    @Override  
22    void draw() {  
23        System.out.println("Triangle.draw()");    }  
24    @Override  
25    void erase() {  
26        System.out.println("Triangle.erase()");  
27    }}
```

```
Source History
1 package testpolymorphism;
2 public class Instructor extends Person
3 {
4     private double salary;
5     public Instructor(String name, double salary){
6         super(name);
7         this.salary = salary;
8     }
9     public double getSalary() {
10         return salary;
11     }
12 @Override
13     public void print() {
14         System.out.println("Instructor: "+getName()+" ", "+salary);
15     }
16 class Student extends Person {
17     private double gpa;
18     public Student(String name, double gpa) {
19         super(name);
20         this.gpa = gpa;
21     }
22     public double getGPA() {
23         return gpa;
24     }
25 @Override
26     public void print() {
27         System.out.println("Student: "+getName()+" ", "+gpa);
28     }
29 }
```

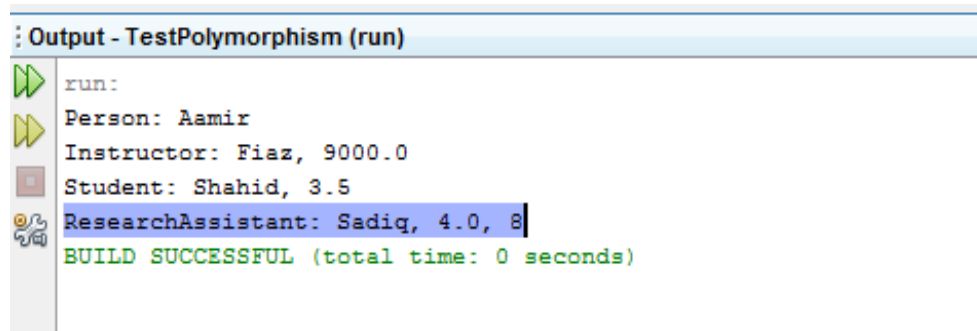
```
Output - ShapeTest (run)
run:
Triangle.draw()
Square.draw()
Square.draw()
Square.draw()
Circle.draw()
Triangle.draw()
Triangle.draw()
Circle.draw()
Circle.draw()
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example 3: Suppose we define a **subclass**, say **ResearchAssistant** for the class **Student** in **Example 1** as follows:



```
1 package testpolymorphism;
2 /**
3  * @author Fiaz
4  */
5 public class ResearchAssistant extends Student {
6     private int workLoad;
7
8     public ResearchAssistant(String name, double gpa, int workLoad) {
9         super(name, gpa);
10        this.workLoad = workLoad;
11    }
12    @Override
13    public void print() {
14        System.out.println("ResearchAssistant: "+getName()+ ", "+getGPA()+", "+workLoad);
15    }
16 }
17
```

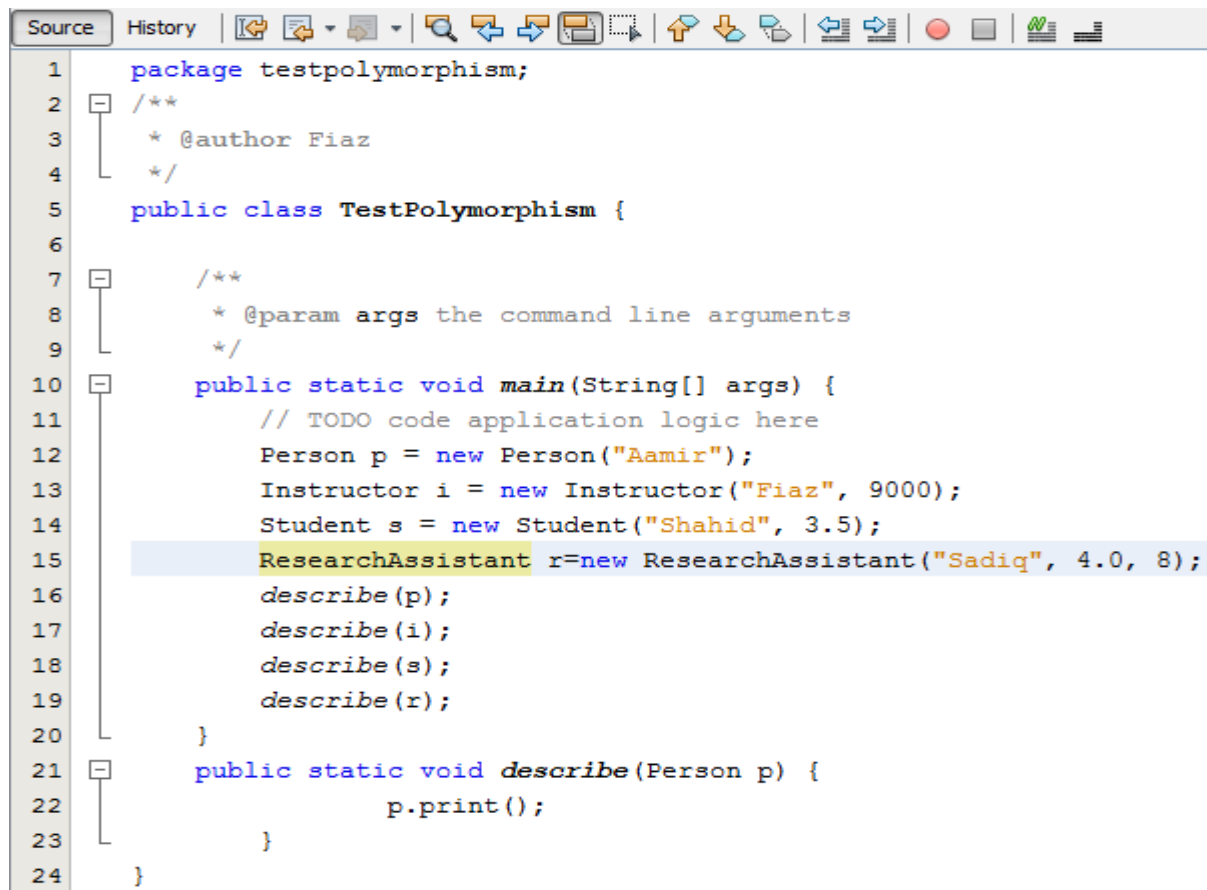
It is now possible to call the **describe()** with an object of type **ResearchAssistant** without any change to the method as shown below. The **describe()** method is said to be **extensible** – meaning the functionality can be added to a code simply by inheriting a new data type from the base class.



```
Output - TestPolymorphism (run)
run:
Person: Aamir
Instructor: Fiaz, 9000.0
Student: Shahid, 3.5
ResearchAssistant: Sadiq, 4.0, 8
BUILD SUCCESSFUL (total time: 0 seconds)
```

Whereas the addition of ResearchAssistant class and its calling is shown below





```
1 package testpolymorphism;
2 /**
3  * @author Fiaz
4  */
5 public class TestPolymorphism {
6
7     /**
8      * @param args the command line arguments
9      */
10    public static void main(String[] args) {
11        // TODO code application logic here
12        Person p = new Person("Aamir");
13        Instructor i = new Instructor("Fiaz", 9000);
14        Student s = new Student("Shahid", 3.5);
15        ResearchAssistant r=new ResearchAssistant("Sadiq", 4.0, 8);
16        describe(p);
17        describe(i);
18        describe(s);
19        describe(r);
20    }
21    public static void describe(Person p) {
22        p.print();
23    }
24 }
```

Exercises:

Exercise 1:

Consider the following class

Public class father

```
{
int age;
father(int x)
{
age=x;
}
public void Iam()
{
System.out.println("I AM FATHER and My AGE = %d", age);
}
}
```

Derive two sub classes son and daughter from the above class and override the method Iam() in each of the subclass and prints an appropriate message in each overridden method. You should also define a suitable constructor for each of these sub classes.

Now write a test class to check the functionality of above example. In main() method create three objects one for each father, son and daughter respectively and call their Iam() method non polymorphically.

Then create an array of father references of size ten and initialize this array randomly with the objects of daughter and son. Then traverse this array and call the Iam() method polymorphically and display the results.

Exercise 2:

ShapeHierarchy: Implement the Shape hierarchy shown in Fig. 1 givenbelow. Each TwoDimensionalShape should contain method getArea to calculate the area of the two-dimensional shape. Each ThreeDimensionalShape should have methods getArea and getVolume to calculate the surface area and volume, respectively, of the three-dimensional shape. Create a program that uses an array of Shape references to objects of each concrete class in the hierarchy. The program should print a text description of the object to which each array element refers. Also, in the loop that processes all the shapes in the array, determine whether each shape is a TwoDimensionalShape or a ThreeDimensionalShape. If it's a TwoDimensionalShape, display its area. If it's a ThreeDimensionalShape, display its area and volume.

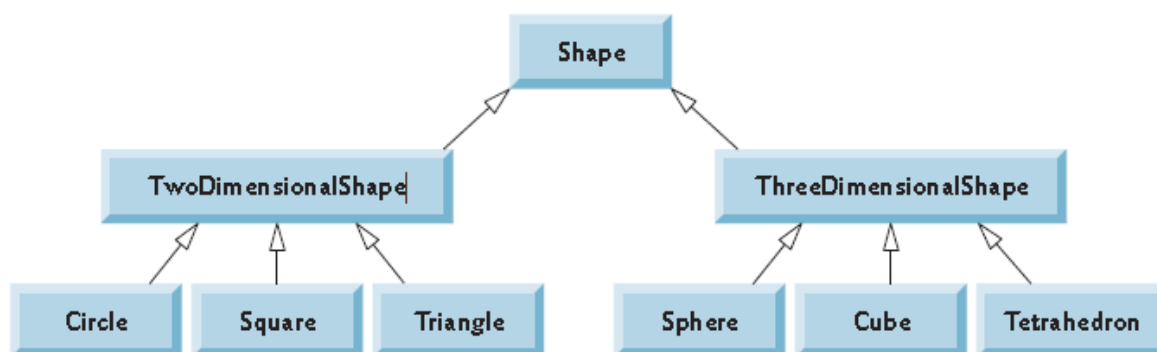


Figure 1: Shape hierarchy

Exercise 3:

Modify the exercise 1 and use a Random class to generate the random number between 1 to the length of the array. Then get the object of an array at indexed equal to the random number generated and call its functionality.

Homework:



1. Define a base class called **Worker** that has two derived classes: **Employee** and **Manager**. For workers, we only have simple attributes such as their names and working hours. For employees and managers, each has specific attributes unique to the class. Employees have attributes such as the name of the department that they are working with, and their salaries. Managers have attributes such as the number of managed employees and the number of projects that they're in charge. Also, define another class called **Project** that has attributes such as the name and the current progress (in %) of the project.
 - (1) Write the corresponding constructors that initialize the attributes for classes.
 - (2) Write the corresponding member functions to set and get the attributes for classes.
 - (3) Finally, test the classes in a main program. In the test program, you should dynamically allocate some employees and managers, print their corresponding information.
2. A company pays its employees on a weekly basis. The employees are of four types: Salaried employees are paid a fixed weekly salary regardless of the number of hours worked, hourly employees are paid by the hour and receive overtime pay (i.e., 1.5 times their hourly salary rate) for all hours worked in excess of 40 hours, commission employees are paid a percentage of their sales and base-salaried commission employees receive a base salary plus a percentage of their sales. For the current pay period, the company has decided to reward salaried-commission employees by adding 10% to their base salaries. The company wants to write an application that performs its payroll calculations polymorphically.

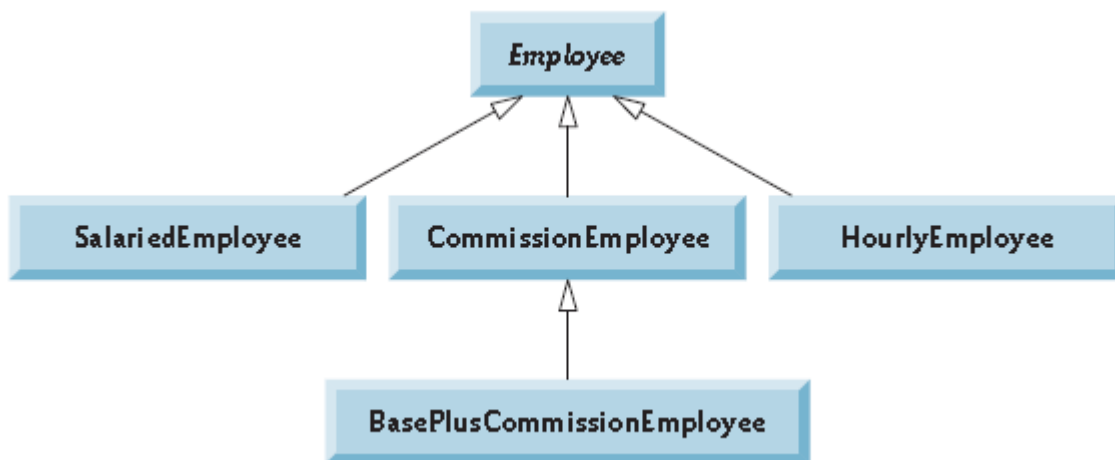


Figure 2: Employee hierarchy

Lab No. 13

Abstract classes, Interfaces, and Inner classes-I

Lab No. 13: Abstract classes, Interface and Inner classes - I

Objective:

To implement the concept of Abstract classes, Interface and Inner classes

Scope:

An abstract class is a class that is declared abstract—it may or may not include abstract methods. Abstract classes cannot be instantiated, but they can be sub classes. When an abstract class is sub-classes, the sub-class usually provides implementations for all of the abstract methods in its parent class.

Example 1: The Following program implement the concept of Abstract classes and method

```
public class Employee extends Person {
    private int empId;
    public Employee(String nm, String gen, int id) {
        super(nm, gen);
        this.empId=id;
    }
    @Override
    public void work() {
        if(empId == 0){
            System.out.println("Not working");
        }else{
            System.out.println("Working as employee!!");
        }
    }
    public static void main(String args[]){
        //coding in terms of abstract classes
        Person student = new Employee("Dove", "Female", 0);
        Person employee = new Employee("Pankaj", "Male", 123);
        student.work();
        employee.work();
        //using method implemented in abstract class - inheritance
        // employee.changeName("Pankaj Kumar");
        System.out.println(employee.toString());
    }
}
```



```

public abstract class Person {

    private String name;
    private String gender;

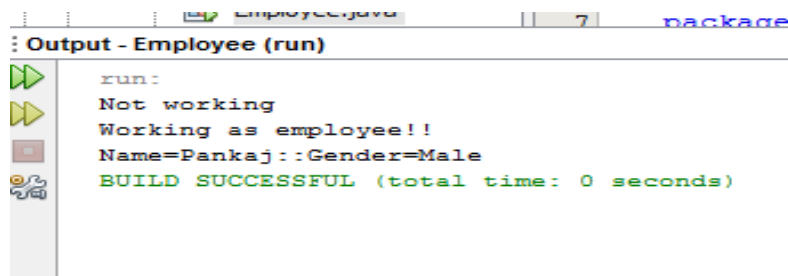
    public Person(String nm, String gen){
        this.name=nm;
        this.gender=gen;
    }

    //abstract method
    public abstract void work();

    @Override
    public String toString(){
        return "Name="+this.name+":Gender="+this.gender;
    }

}

```



Exercises:

Exercise 1:

Create an abstract class with at least one method, in its own package. Create a class in a separate package. Add a protected inner class that implements the interface. In a third package, inherit from your class and, inside a method, return an object of the protected inner class, up-casting to the abstract class during the return.

Homework:

Create an Abstract Class named Restaurant

Create a function/method that will print the name of the restaurant when called.

Create an abstract function/method named total price

Create an abstract function/method named menu items

Create an abstract function/method name location

Create a Class called McDonalds that extends Restaurant

Implement all abstract methods

Add logic so that the total price method/function will give the total price of the meal including a 6% tax

Add a method that returns a Boolean named hasPlayPlace, which returns true when this location has a play place

Create a Constructor that will set the name of the Mcdonalds, location, and hasPlayPlace



Lab No. 14

Abstract classes, Interfaces, and Inner classes-II

Lab No. 14: Abstract classes, Interface and Inner classes-II

Objective:

To implement the concept of Abstract classes, Interface and Inner classes

Scope:

An interface is a collection of abstract methods. A class implements an interface, thereby inheriting the abstract methods of the interface. An interface is not a class. Writing an interface is similar to writing a class, but they are two different concepts. A class describes the attributes and behaviors of an object. An interface contains behaviors that a class implements.

Example 1: The Following program implements the concept of Interfaces

```
interface Animal {  
  
    public void eat();  
    public void travel();  
}  
|  
  
public class MammalInt implements Animal{  
  
    public void eat(){  
        System.out.println("Mammal eats");  
    }  
  
    public void travel(){  
        System.out.println("Mammal travels");  
    }  
  
    public int noOfLegs(){  
        return 0;  
    }  
  
    public static void main(String args[]){  
        MammalInt m = new MammalInt();  
        m.eat();  
        m.travel();  
    }  
}
```



```
Output - MammalInt (run)
run:
Mammal eats
Mammal travels
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example 2: The Following program implements the concept of Interfaces

```
package intrface;
interface A{
    int x=10;
    void show();
}

interface B{
    int y=20;
    void display();
}

class C implements A,B{

    public void show(){
        System.out.println("I am in A interface and having X = "+x);
    }

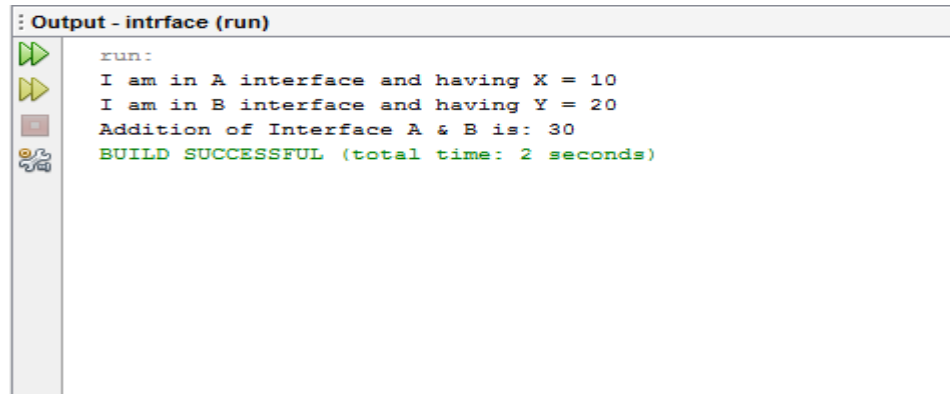
    public void display(){
        System.out.println("I am in B interface and having Y = "+y);
    }

    void add_X_Y(){
        System.out.println("Addition of Interface A & B is: "+(x+y));
    }
}
```

```

public class Interface {
    public static void main(String[] args) {
        C ob=new C();
        ob.show();
        ob.display();
        ob.add_X_Y();
    }
}

```



```

Output - intrface (run)
run:
I am in A interface and having X = 10
I am in B interface and having Y = 20
Addition of Interface A & B is: 30
BUILD SUCCESSFUL (total time: 2 seconds)

```

Exercises:

Exercise 1:

Create three interfaces, each with two methods. Inherit a new interface from the three, adding a new method. Create a class by implementing the new interface and also inheriting from a concrete class. Now write four methods, each of which takes one of the four interfaces as an argument. In main(), create an object of your class and pass it to each of the methods.

Homework:

1. Consider the four core interfaces, Set, List, Queue, and Map. For each of the following four assignments, specify which of the four core interfaces is best-suited, and explain how to use it to implement the assignment.
 1. Whimsical Toys Inc (WTI) needs to record the names of all its employees. Every month, an employee will be chosen at random from these records to receive a free toy.
 2. WTI has decided that each new product will be named after an employee but only first names will be used, and each name will be used only once. Prepare a list of unique first names.
 3. WTI decides that it only wants to use the most popular names for its toys. Count up the number of employees who have each first name.
 4. WTI acquires season tickets for the local lacrosse team, to be shared by employees.
 5. Create a waiting list for this popular sport.



Lab No. 15

Abstract classes, Interfaces, and Inner classes-III

Lab No. 15: Abstract classes, Interface and Inner classes-III

Objective:

To implements the concepts of Inner classes.

Scope:

Inner classes nest within other classes. A normal class is a direct member of a package. Static member class is a static member of a class. Like any other static method, a static member class has access to all static methods of the parent, or top-level, class. Like a static member class, a member class is also defined as a member of a class. Unlike the static variety, the member class is instance specific and has access to any and all methods and members; even the parent's this reference. Local classes are declared within a block of code and are visible only within that block, just as any other method variable.

Example 1: The Following program implements the concept of Inner classes

```
*/public class DataStructure {  
    // Create an array  
  
    private final static int SIZE = 15;  
    private int[] arrayOfInts = new int[SIZE];  
  
    public DataStructure() {  
        // fill the array with ascending integer values  
        for (int i = 0; i < SIZE; i++) {  
            arrayOfInts[i] = i;  
        }  
    }  
  
    public void printEven() {  
  
        // Print out values of even indices of the array  
        DataStructureIterator iterator = new EvenIterator();  
        while (iterator.hasNext()) {  
            System.out.print(iterator.next() + " ");  
        }  
        System.out.println();  
    }  
  
    interface DataStructureIterator extends java.util.Iterator<Integer> {  
  
    }  
}
```



```

public class EvenIterator implements DataStructureIterator {

    /**
     * @param args the command line arguments
     */
    // Start stepping through the array from the beginning
    private int nextIndex = 0;
    private String[] arrayOfInts;

    public boolean hasNext() {

        // Check if the current element is the last in the array
        return (nextIndex <= SIZE - 1);
    }

    public Integer next() {

        // Record a value of an even index of the array
        Integer retValue = Integer.valueOf(arrayOfInts[nextIndex]);

        // Get the next even element
        nextIndex += 2;
        return retValue;
    }

    public static void main(String s[]) {

        // Fill the array with integer values and print out only
        // values of even indices
        DataStructure ds = new DataStructure();
        ds.printEven();
    }
}

```

```

Output - MammalInt (run)
run:
0 2 4 6 8 10
BUILD SUCCESSFUL (total time: 19 seconds)

```

Example 2: Following is the implementation of Anonymous class

```
public class HelloWorldAnonymousClasses {
    interface HelloWorld {
        public void greet();
        public void greetSomeone(String someone);
    }
    public void sayHello() {
        class EnglishGreeting implements HelloWorld {
            String name = "world";
            public void greet() {greetSomeone("world");}
            public void greetSomeone(String someone) {
                name = someone;
                System.out.println("Hello " + name);}}
        HelloWorld englishGreeting = new EnglishGreeting();
        HelloWorld frenchGreeting = new HelloWorld() {
            String name = "tout le monde";
            public void greet() {
                greetSomeone("tout le monde");}
            public void greetSomeone(String someone) {
                name = someone;
                System.out.println("Salut " + name);}};
        HelloWorld spanishGreeting = new HelloWorld() {
            String name = "mundo";
            public void greet() {
                greetSomeone("mundo"); }
            public void greetSomeone(String someone) {
                name = someone;
                System.out.println("Hola, " + name); }};
        englishGreeting.greet();
        frenchGreeting.greetSomeone("Fred");
        spanishGreeting.greet();
    }
    public static void main(String... args) {
        HelloWorldAnonymousClasses myApp = new HelloWorldAnonymousClasses();
        myApp.sayHello();
    }
}
```

```
Output - HelloWorldAnonymousClasses (run)

run:
Hello world
Salut Fred
Hola, mundo
BUILD SUCCESSFUL (total time: 0 seconds)
```



Exercises:

Exercise 1:

Create an interface with at least one method, in its own package. Create a class in a separate package. Add a protected inner class that implements the interface. In a third package, inherit from your class and, inside a method, return an object of the protected inner class, up-casting to the interface during the return. .

Exercise 2:

Create an interface with at least one method, and implement that interface by defining an inner class within a method, which returns a reference to your interface. .

Homework:

1. Define a class structure (abstract classes, regular classes, methods, private fields, interfaces) which implement pizzas, pizza sub-types, crust types, topping sets and (most importantly) the comparator objects representing different people's preferences about pizzas;

Define a list of pizzas to be compared and ranked, preferably not a fixed list, but one which you can change "externally" (see below);

Define the individual people/comparator objects to be used to rank the pizzas;

Run the ranking algorithm and print out the sorted list of pizzas with their scores (it might be good to print out individual people's rank lists, too, so that you can tell that the algorithm is working properly).

You should make your own design decisions about class structure, although of course I will be available for help and discussion, and also design your own graphical or command-line interface. But please try to take the following issues into account:



Lab No. 16

Static members and Functions

Lab No. 16: Static members and Functions

Objective:

To implements the concepts of static members and functions.

Scope:

After introducing static data members as you may recall, a static member is not duplicated for each object of the class; rather a single data item is shared by all objects of a class. For better understanding some examples are given below.

Example 1: This program demonstrates the use of static member shared by some objects.

```
class Static_Test {

    public static int static_x = 100;           //static identifier static_x
    public int instance_y;                     //non-static identifier instance_y

    Static_Test() {                           //non-parameterized constructor
        static_x++;
    }

    public void get(int j) {                   // A class method
        instance_y = j;
    }

    public void show() {
        System.out.println(static_x + "\t" + instance_y);
    }
}

public class Stat_1 {
    public static void main(String[] args) {
        System.out.println("X\tY");
        System.out.println("-----");
        Static_Test ob1 = new Static_Test();
        ob1.get(30);
        ob1.show();                           //static_x is shared by all objects
        Static_Test ob2 = new Static_Test();
        ob2.get(50);
        ob2.show();
        Static_Test ob3 = new Static_Test();
        ob3.get(81);
    }
}
```



```

        ob3.show();

        System.out.println("Array of objects sharing the static data");
        Static_Test[] ob = new Static_Test[100];    //Array of objects sharing
        for(int i=0; i<5; i++){                    //the static data member
            ob[i]=new Static_Test();
            ob[i].instance_y=(int) (30.0*Math.random());
            ob[i].show();
        }
    }
}

```

Output - stat_1 (run) %






```

run:
X      Y
-----
101    30
102    50
103    81
Array of objects sharing the static data
104    17
105    1
106    1
107    7
108    27
BUILD SUCCESSFUL (total time: 0 seconds)

```

Example 2: This program demonstrates the use of static member and static function in a class.

```
class static_demo_A {  
  
    public static int static_x = 3;           //static identifier initialization  
    public static int sq(int a) {  
        static_x = a;  
        return (static_x * static_x);  
    }  
    public void show() {  
        System.out.println(static_x);  
    }  
}  
  
class demo_B {  
    public int test_y;  
    demo_B(int a) {  
        test_y = a;  
    }  
    public void show() {                     //No need to create an object  
        test_y = static_demo_A.sq(test_y);  //for static member funtion  
        System.out.println(test_y);  
    }  
}  
  
public class Stat_2 {  
    public static void main(String[] args) {  
        static_demo_A ob1 = new static_demo_A();  
        System.out.println("Current output status become");  
        ob1.show();  
        demo_B ob2 = new demo_B(7);  
        ob2.show();    ob1.show();  
    }  
}
```

```
Output - stat_2 (run)

run:
Current output status become
3
49
7
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example 3: This program demonstrates the Printer sharing among various faculty members with the use of static member in a class.

```
class Printing {
    public static int static_user_id; //static user ID generation
    int pages;

    Printing() {
        static_user_id++;
    }

    public void get_printing(int j) {
        pages += j;
    }
}

public class Stat_3 {
    public static void main(String[] args) {

        Printing[] ob = new Printing[100]; //Array of objects
        int[] kid = new int[100]; //Array of ID's

        for (int i = 0; i < 5; i++) {
            ob[i] = new Printing();
            kid[i] = Printing.static_user_id;
            ob[i].pages = (int) (50.0 * Math.random());
            System.out.println("User " + kid[i] + " has taken initially "
                + ob[i].pages + " pages");
        }

        // Now Taking more pages and add to the user accounts
        System.out.println("\nNow the current status of each user after"
            + " getting more Pages\n ");
    }
}
```



```

        for (int i = 0; i < 5; i++) {
            int j = (int) (50.0 * Math.random());
            System.out.println("User " + kid[i] + " has taken " + j +
                               " Pages more");
            ob[i].get_printing(j);
            System.out.println("After updation the status become for user "
                               + kid[i] + " has printed " + ob[i].pages + " Pages");
        }
    }
}

```

Output:

```

Output - stat_3 (run)

run:
User 1 has taken initially 1 pages
User 2 has taken initially 38 pages
User 3 has taken initially 33 pages
User 4 has taken initially 37 pages
User 5 has taken initially 30 pages

Now the current status of each user after getting more Pages

User 1 has taken 3 Pages more
After updation the status become for user 1 has printed 4 Pages
User 2 has taken 16 Pages more
After updation the status become for user 2 has printed 54 Pages
User 3 has taken 24 Pages more
After updation the status become for user 3 has printed 57 Pages
User 4 has taken 44 Pages more
After updation the status become for user 4 has printed 81 Pages
User 5 has taken 10 Pages more
After updation the status become for user 5 has printed 40 Pages
BUILD SUCCESSFUL (total time: 0 seconds)

```

Exercises:

Exercise 1:

Create a SavingsAccount class and use a static data member to contain the annualInterest for each of the savors. Each member of the class contains a private data member savingbalance indicating the amount the saver currently has deposited. Provide a monthlyInterest() member function that calculate the monthly interest by multiplying the balance by annualInterest by 12;



this interest should be added to the savingbalance. Provide a static member function `modifyInterest()` that sets the static `annualInterest` to a new value.

Write a driver program to test class `SavingAccount` and instantiate two objects `saver1` & `saver2` with balance of \$30000 and \$50000 respectively. Set annual interest 5% then calculate the monthly interest and show the new balance for each saver.

Exercise 2:

Write a static method `equalize()` that takes two arrays of integers as arguments and returns true if they contain the same number of elements and all corresponding pairs of elements are equal, and false otherwise.

Exercise 3:

Write a static method `Triangle_Test()` that takes three double values as arguments and returns true if they could be the sides of a triangle (none of them is greater than or equal to the sum of the other two).

Homework

Write a program in JAVA to show a delegate report regarding the printer usage of three departments. Report consists of total pages has been taken by each department and total pages taken by each individual faculty member. At the end of each month how many pages in total has been taken from the printer.

Report should look like as:

Department of Computer Science

Total # of Pages taken:????

Faculty member 1 has taken:????

Faculty member 2 has taken:???? And so on

Department of Mathematics

Total # of Pages taken:????

Faculty member 1 has taken:????

Faculty member 2 has taken:???? And so on



Lab No. 17

File Handling

Lab No. 17: File Handling

Objective:

To implement the concepts Reading from text file using input output stream.

Scope:

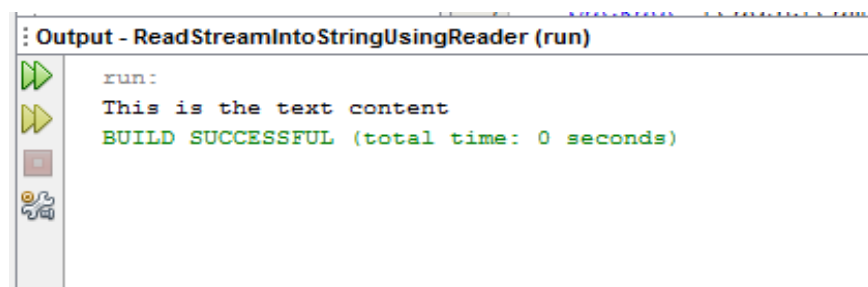
Standard streams are pre-connected input and output communication channels between a computer program and its environment when it begins execution. The three I/O connections are called standard input, standard output & standard error. We can use input stream in that scenario to read/write content from file.

Example 1: There will be another program which demonstrates the use InputStreamReader to read standard input stream from file with buffer reader

```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;

public class ReadStreamIntoStringUsingReader {

    public static void main(String[] args) throws FileNotFoundException, IOException {
        InputStream in = new FileInputStream(new File("D:\\ab.txt"));
        BufferedReader reader = new BufferedReader(new InputStreamReader(in));
        StringBuilder out = new StringBuilder();
        String line;
        while ((line = reader.readLine()) != null) {
            out.append(line);
        }
        System.out.println(out.toString()); //Prints the string content read from input
        reader.close();
    }
}
```



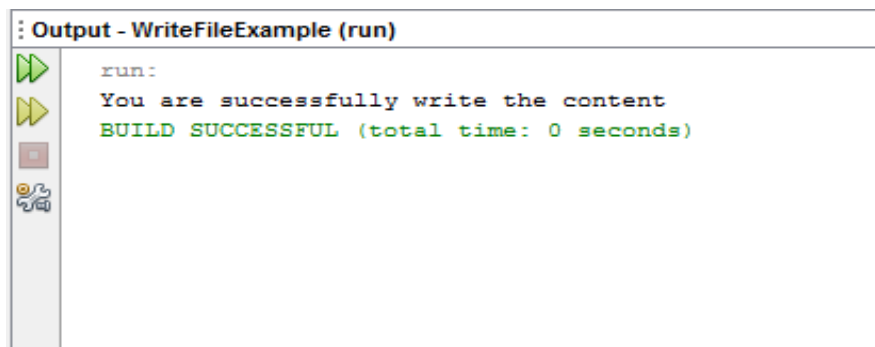
```
Output - ReadStreamIntoStringUsingReader (run)

run:
This is the text content
BUILD SUCCESSFUL (total time: 0 seconds)
```



Example 2: This program demonstrates How to write to file in Java File Output stream.

```
import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
public class WriteFileExample {
public static void main(String[] args) {
    FileOutputStream fop = null;
    File file;
    String content = "This is the text content";
    try {
        file = new File("D:\\ab.txt");
        fop = new FileOutputStream(file);
        // if file doesnt exists, then create it
        if (!file.exists()) {
            file.createNewFile();
        } // get the content in bytes
        byte[] contentInBytes = content.getBytes();
        fop.write(contentInBytes);
        fop.flush();
        fop.close();
        System.out.println("You are successfully write the content");
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        try {
            if (fop != null) {
                fop.close();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```



```
Output - WriteFileExample (run)
run:
You are successfully write the content
BUILD SUCCESSFUL (total time: 0 seconds)
```

Exercises:

Exercise 1: Open a text file so that you can read the file one line at a time. Read each line as a String and place that String object into a Linked List. Print all of the lines in the Linked List in reverse order.

Exercise 2: Modify Exercise 1 so that the name of the file you read is provided as a command-line argument.

Exercise 3: Open a text file so you can write text into it. Write the lines in the Array List, along with line numbers (do not attempt to use the Line Number classes), out to the file.

Exercise 4: Modify Exercise 2 to force all the lines in the Array List to uppercase and send the results to output file.

Homework:

1. Text 1= “Today is Thursday and it is a bright sunny day. I am the student of software engineering”
Text 2= “we need help”?
Write a program that should create a text file with a name OOPs in D:\ drive and store the above mention text of text 1 in file then add above mention text of Text 2 in the same file in reverse order.
2. Write a Java application that will be able to add, subtract, multiply, divide, compare, convert to floating point, reduce to its lowest terms, and find absolute value for rational numbers.

Your program should be written in Object Oriented Programming style. It should read a file called input.txt that contains a pair of rational numbers to a line.

(NOTE: the line may contain incorrect input). It should parse each line to retrieve two rational numbers and perform operations on them. Rational numbers will be separated by “and”. The results must be written to a file called results.txt.



Lab No. 18

Exception Handling-I

Lab No. 18: Exception Handling-I

Objective:

Introduction to Exception Handling. The Exception Hierarchy in java.

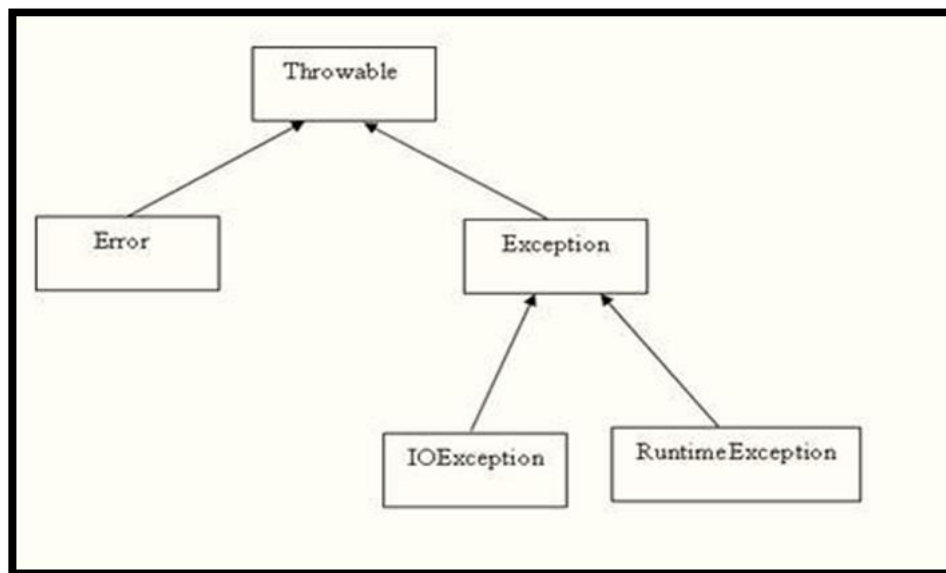
Scope:

An Exception can be anything which interrupts the normal flow of the program. When an exception occurs program processing gets terminated and doesn't continue further. In such cases we get a system generated error message. The good thing about exceptions is that they can be handled. Exceptions are conditions within the code. A developer can handle such conditions and take necessary corrective actions.

Exception Hierarchy:

All exception classes are subtypes of the java.lang.Exception class. The exception class is a subclass of the Throwable class. Other than the exception class there is another subclass called Error which is derived from the Throwable class.

Errors are not normally trapped from the Java programs. These conditions normally happen in case of severe failures, which are not handled by the java programs. Errors are generated to indicate errors generated by the runtime environment. Example : JVM is out of Memory. Normally programs cannot recover from errors. The Exception class has two main subclasses: IOException class and RuntimeException Class.



Example 1: The following programs handle an exception:

```
class Example1 {  
    public static void main(String args[]) {  
        int num1, num2;  
        try {  
            // Try block to handle code that may cause exception  
            num1 = 0;  
            num2 = 62 / num1;  
            System.out.println("Try block message");  
        } catch (ArithmeticException e) {  
            // This block is to catch divide-by-zero error  
            System.out.println("Error: Don't divide a number by zero");  
        }  
        System.out.println("I'm out of try-catch block in Java.");  
    }  
}
```

```
Error: Don't divide a number by zero  
I'm out of try-catch block in Java.
```

Example 2: Implementation of multiple Catch blocks:



```
class Example2{
    public static void main(String args[]){
        try{
            int a[]=new int[7];
            a[4]=30/0;
            System.out.println("First print statement in try block");
        }
        catch(ArithmeticException e){
            System.out.println("Warning: ArithmeticException");
        }
        catch(ArrayIndexOutOfBoundsException e){
            System.out.println("Warning: ArrayIndexOutOfBoundsException");
        }
        catch(Exception e){
            System.out.println("Warning: Some Other exception");
        }
        System.out.println("Out of try-catch block...");
    }
}
```

```
Warning: ArithmeticException
Out of try-catch block...
```



Example 3: Implementation of Nested try catch:

```
class Nest{  
    public static void main(String args[]){  
        //Parent try block  
        try{  
            //Child try block1  
            try{  
                System.out.println("Inside block1");  
                int b =45/0;  
                System.out.println(b);  
            }  
            catch(ArithmeticException e1){  
                System.out.println("Exception: e1");  
            }  
            //Child try block2  
            try{  
                System.out.println("Inside block2");  
                int b =45/0;  
                System.out.println(b);  
            }  
            catch(ArrayIndexOutOfBoundsException e2){  
                System.out.println("Exception: e2");  
            }  
        }  
    }  
}
```



```

        System.out.println("Just other statement");
    }
    catch(ArithmeticException e3){
        System.out.println("Arithmetic Exception");
        System.out.println("Inside parent try catch block");
    }
    catch(ArrayIndexOutOfBoundsException e4){
        System.out.println("ArrayIndexOutOfBoundsException");
        System.out.println("Inside parent try catch block");
    }
    catch(Exception e5){
        System.out.println("Exception");
        System.out.println("Inside parent try catch block");
    }
    System.out.println("Next statement..");
}
}

```

```

Inside block1
Exception: e1
Inside block2
Arithmetic Exception
Inside parent try catch block
Next statement..

```

Homework:

1. Write a program that prompts the user to read two integers and displays their sum. Your program should prompt the user to read the number again if the input is incorrect.
2. Write a program that meets the following requirements:
 - Create an array with 100 randomly chosen integers.
 - Prompts the user to enter the index of the array, and then displays the corresponding element value. If the specified index is out of bounds, display the message Out of Bounds.
3. Write a program that causes the JVM to throw an OutOfMemoryError and catches and handles this error.



Lab No. 19

Exception Handling-II

Lab No. 19: Exception Handling-II

Objective:

Introduction to Exception Handling. The Exception Hierarchy in java.

Scope:

An Exception can be anything which interrupts the normal flow of the program. When an exception occurs program processing gets terminated and doesn't continue further. In such cases we get a system generated error message. The good thing about exceptions is that they can be handled. Exceptions are conditions within the code. A developer can handle such conditions and take necessary corrective actions.

Example 1: Implementation throwing an exception

```
import java.io.*;
class M{
    void method()throws IOException{
        throw new IOException("device error");
    }
}
public class Testthrows2{
    public static void main(String args[]){
        try{
            M m=new M();
            m.method();
        }catch(Exception e){System.out.println("exception handled");}

        System.out.println("normal flow...");
    }
}
```

```
Output:exception handled
        normal flow...
```



Example 2: Implementation throwing an IOException with normal flow.

```
import java.io.*;
class M{
    void method()throws IOException{
        System.out.println("device operation performed");
    }
}
class Testthrows3{
    public static void main(String args[])throws IOException{//declare exception
        M m=new M();
        m.method();

        System.out.println("normal flow...");
    }
}
```

```
Output:device operation performed
       normal flow...
```

Example 3: Implementation throwing a runtime Exception.

```
import java.io.*;
class M{
    void method()throws IOException{
        throw new IOException("device error");
    }
}
class Testthrows4{
    public static void main(String args[])throws IOException{//declare exception
        M m=new M();
        m.method();

        System.out.println("normal flow...");
    }
}
```



Output: Runtime Exception

Example 4: To demonstrate using our user-defined exception, the following CheckingAccount class contains a withdraw() method that throws an InsufficientFundsException.

```
5
7 // File Name InsufficientFundsException.java
8
9
10 public class InsufficientFundsException extends Exception
11 {
12     private double amount;
13     public InsufficientFundsException(double amount)
14     {
15         this.amount = amount;
16     }
17     public double getAmount()
18     {
19         return amount;
20     }
21 }
```

```

public class CheckingAccount
{
    private double balance;
    private int number;
    public CheckingAccount(int number)
    {
        this.number = number;
    }
    public void deposit(double amount)
    {
        balance += amount;
    }
    public void withdraw(double amount) throws InsufficientFundsException
    {
        if(amount <= balance)
        {
            balance -= amount;
        }
        else
        {
            double needs = amount - balance;
            throw new InsufficientFundsException(needs);
        }
    }
    public double getBalance()
    {
        return balance;
    }
    public int getNumber()
    {
        return number;
    }
}

```

Example 5: The following BankDemo program demonstrates invoking the deposit() and withdraw() methods of CheckingAccount.

```

/**
 *
 * @author Ali Computers
 */
// File Name BankDemo.java
public class BankDemo {

    public static void main(String[] args) {
        CheckingAccount c = new CheckingAccount(101);
        System.out.println("Depositing $500...");
        c.deposit(500.00);
        try {
            System.out.println("\nWithdrawing $100...");
            c.withdraw(100.00);
            System.out.println("\nWithdrawing $600...");
            c.withdraw(600.00);
        }
        catch (InsufficientFundsException e) {
            System.out.println("Sorry, but you are short $"
                + e.getAmount());
            e.printStackTrace();
        }
    }
}

```

```

Depositing $500...

Withdrawing $100...

Withdrawing $600...
Sorry, but you are short $200.0
InsufficientFundsException
    at CheckingAccount.withdraw(CheckingAccount.java:25)
    at BankDemo.main(BankDemo.java:13)

```

Example 6: Java finally example where exception doesn't occur.

```
class TestFinallyBlock{
    public static void main(String args[]){
        try{
            int data=25/5;
            System.out.println(data);
        }
        catch(NullPointerException e){System.out.println(e);}
        finally{System.out.println("finally block is always executed");}
        System.out.println("rest of the code...");
    }
}
```

finally block is always executed
rest of the code...

Example 7: Java finally example where exception occurs and not handled.

```
class TestFinallyBlock1{
    public static void main(String args[]){
        try{
            int data=25/0;
            System.out.println(data);
        }
        catch(NullPointerException e){System.out.println(e);}
        finally{System.out.println("finally block is always executed");}
        System.out.println("rest of the code...");
    }
}
```

```
Output:finally block is always executed
       Exception in thread main java.lang.ArithmeticException:/ by zero
```

Example 8: Java finally example where exception occurs and handled.

```
public class TestFinallyBlock2{
    public static void main(String args[]){
        try{
            int data=25/0;
            System.out.println(data);
        }
        catch(ArithmeticException e){System.out.println(e);}
        finally{System.out.println("finally block is always executed");}
        System.out.println("rest of the code...");
    }
}
```

```
Output:Exception in thread main java.lang.ArithmeticException:/ by zero
       finally block is always executed
       rest of the code...
```

Homework:

1. List the various exceptional conditions that have occurred in programs throughout this text so far. List as many additional exceptional conditions as you can. For each of these, describe briefly how a program typically would handle the exception by using the exception-handling techniques discussed in this chapter. Typical exceptions include division by zero and array index out of bounds.
2. Write a program that demonstrates how various exceptions are caught with catch (Exception exception) This time, define classes ExceptionA (which inherits from class Exception) and ExceptionB (which inherits from class ExceptionA). In your program, create try blocks that throw exceptions of types ExceptionA, ExceptionB, NullPointerException and IOException. All exceptions should be caught with catch blocks specifying type Exception. (*Catching Exceptions Using Class Exception*)



3. Create a directory) Write a program that prompts the user to enter a directory name and creates a directory using the File's mkdirs method. The program displays the message "Directory created successfully" if a directory is created or "Directory already exists" if the directory already exists



Lab No. 20

GUI Programming-I

Lab No. 20: GUI Programming - I

Objective:

To create user interfaces using frame, panels, and simple GUI components.

Scope:

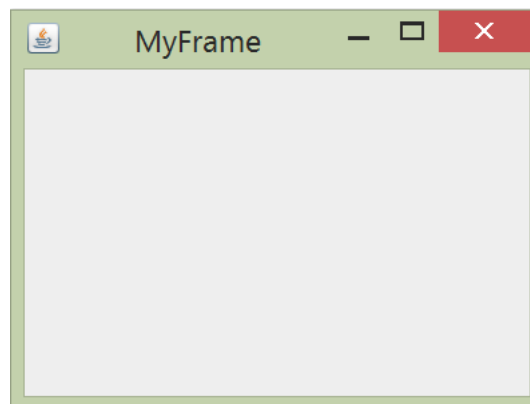
This lab will serve two purposes. First, it presents the basics of Java GUI programming. Second, it uses GUI to demonstrate OOP. Specifically, this lab introduces the framework of the Java GUI API and discusses GUI components and their relationships, containers and layout managers, colors, fonts, borders, image icons, and tool tips. It also introduces some of the most frequently used GUI components.

Example 1: Creating a frame

```
package lab_manual_gui;
import javax.swing.*;

public class MyFrame {

    public static void main(String[] args) {
        JFrame frame = new JFrame("MyFrame"); // Create a frame
        frame.setSize(400, 300); // Set the frame size
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Center a frame
        frame.setVisible(true); // Display the frame
    }
}
```



Example 2: Adding components to a frame

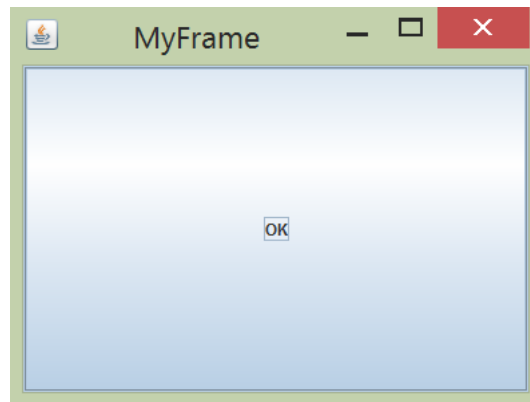
```
package lab_manual_gui;
import javax.swing.*;

public class MyFrame {

    public static void main(String[] args) {
        JFrame frame = new JFrame("MyFrame"); // Create a frame

        //Add a button to a frame
        JButton btn = new JButton("OK");
        frame.add(btn);

        frame.setSize(400, 300); // Set the frame size
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Center a frame
        frame.setVisible(true); // Display the frame
    }
}
```



Layout Manager

Each container contains a layout manager, which is an object responsible for laying out the GUI components in the container.

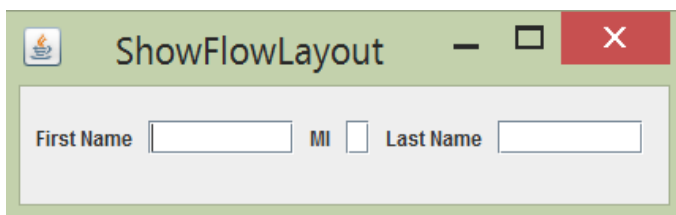
Example 3: Flow Layout

```
import javax.swing.JLabel;
import javax.swing.JTextField;
import javax.swing.JFrame;
import java.awt.FlowLayout;

public class ShowFlowLayout extends JFrame {

    public ShowFlowLayout() {
        // Set FlowLayout, aligned left with horizontal gap 10
        // and vertical gap 20 between components
        setLayout(new FlowLayout(FlowLayout.LEFT, 10, 20));
        // Add labels and text fields to the frame
        add(new JLabel("First Name"));
        add(new JTextField(8));
        add(new JLabel("MI"));
        add(new JTextField(1));
        add(new JLabel("Last Name"));
        add(new JTextField(8));
    }

    public static void main(String[] args) {
        ShowFlowLayout frame = new ShowFlowLayout();
        frame.setTitle("ShowFlowLayout");
        frame.setSize(200, 200);
        frame.setLocationRelativeTo(null); // Center the frame
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```



Example 4: Grid Layout

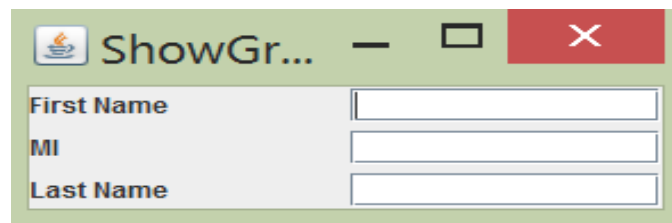
```
import javax.swing.JLabel;
import javax.swing.JTextField;
import javax.swing.JFrame;
import java.awt.GridLayout;

public class ShowGridLayout extends JFrame {

    public ShowGridLayout() {
        // Set GridLayout, 3 rows, 2 columns, and gaps 5 between
        // components horizontally and vertically
        setLayout(new GridLayout(3, 2, 5, 5));

        // Add labels and text fields to the frame
        add(new JLabel("First Name"));
        add(new JTextField(8));
        add(new JLabel("MI"));
        add(new JTextField(1));
        add(new JLabel("Last Name"));
        add(new JTextField(8));
    }

    public static void main(String[] args) {
        ShowGridLayout frame = new ShowGridLayout();
        frame.setTitle("ShowGridLayout");
        frame.setSize(300, 125);
        frame.setLocationRelativeTo(null); // Center the frame
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```



Example 5: Border Layout

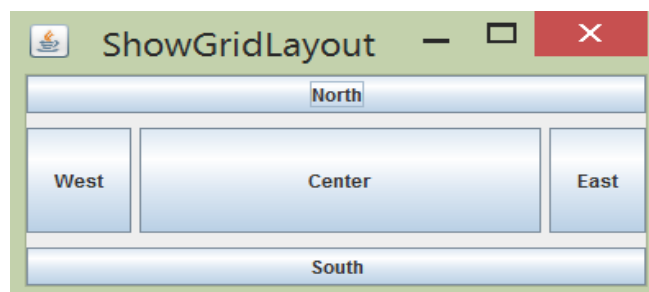
```
import javax.swing.JButton;
import javax.swing.JFrame;
import java.awt.BorderLayout;

public class ShowBorderLayout extends JFrame {

    public ShowBorderLayout() {
        // Set BorderLayout with horizontal gap 5 and vertical gap 10
        setLayout(new BorderLayout(5, 10));

        // Add buttons to the frame
        add(new JButton("East"), BorderLayout.EAST);
        add(new JButton("South"), BorderLayout.SOUTH);
        add(new JButton("West"), BorderLayout.WEST);
        add(new JButton("North"), BorderLayout.NORTH);
        add(new JButton("Center"), BorderLayout.CENTER);
    }

    public static void main(String[] args) {
        ShowBorderLayout frame = new ShowBorderLayout();
        frame.setTitle("ShowGridLayout");
        frame.setSize(400, 200);
        frame.setLocationRelativeTo(null); // Center the frame
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```



Example 6: Using Panels and sub-containers

A container can be placed inside another container. Panels can be used as sub-containers to group GUI components to achieve the desired layout.

```
import java.awt.*;
import javax.swing.*;

public class TestPanels extends JFrame {

    public TestPanels() {
        // Create panel p1 for the buttons and set GridLayout
        JPanel p1 = new JPanel();
        p1.setLayout(new GridLayout(4, 3));

        // Add buttons to the panel
        for (int i = 1; i <= 9; i++) {
            p1.add(new JButton("" + i));
        }

        p1.add(new JButton("" + 0));
        p1.add(new JButton("Start"));
        p1.add(new JButton("Stop"));
    }
}
```



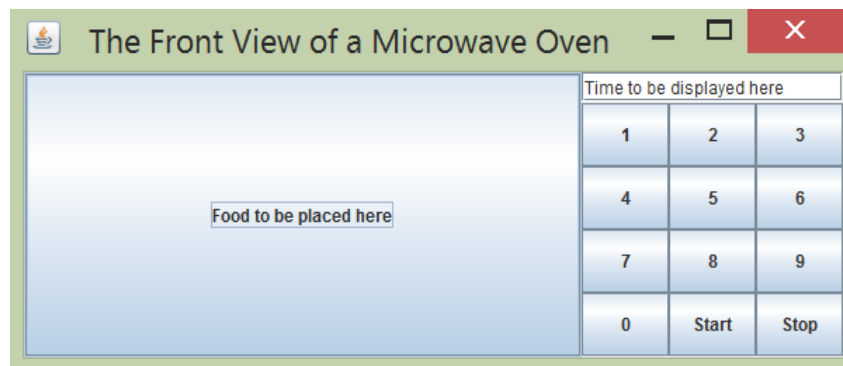
```

// Create panel p2 to hold a text field and p1
JPanel p2 = new JPanel(new BorderLayout());
p2.add(new JTextField("Time to be displayed here"),
        BorderLayout.NORTH);
p2.add(p1, BorderLayout.CENTER);

// add contents into the frame
add(p2, BorderLayout.EAST);
add(new JButton("Food to be placed here"),
        BorderLayout.CENTER);
}

/**
 * Main method
 */
public static void main(String[] args) {
    TestPanels frame = new TestPanels();
    frame.setTitle("The Front View of a Microwave Oven");
    frame.setSize(400, 250);
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);
}
}

```



Example 7: Colors & Fonts

```
import java.awt.*;
import javax.swing.*;
import javax.swing.border.*;

public class TestSwingCommonFeatures extends JFrame {

    public TestSwingCommonFeatures() {
        // Create a panel to group three buttons
        JPanel p1 = new JPanel(new FlowLayout(FlowLayout.LEFT, 2, 2));
        JButton jbtLeft = new JButton("Left");
        JButton jbtCenter = new JButton("Center");
        JButton jbtRight = new JButton("Right");
        jbtLeft.setBackground(Color.WHITE);
        jbtCenter.setForeground(Color.GREEN);
        jbtRight.setToolTipText("This is the Right button");
        p1.add(jbtLeft);
        p1.add(jbtCenter);
        p1.add(jbtRight);
        p1.setBorder(new TitledBorder("Three Buttons"));
        p1.setCursor(new Cursor(Cursor.CROSSHAIR_CURSOR));
    }
}
```



```

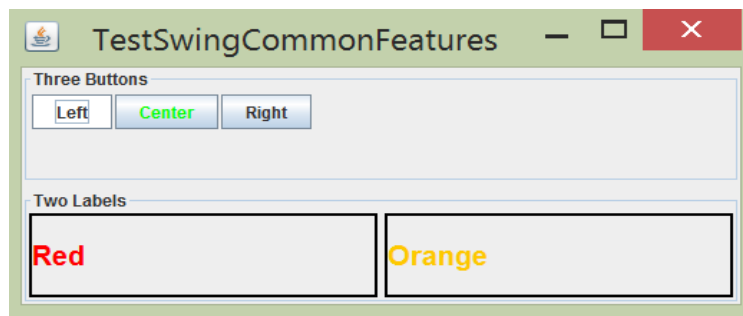
// Create a font and a line border
Font largeFont = new Font("TimesRoman", Font.BOLD, 20);
Border lineBorder = new LineBorder(Color.BLACK, 2);

// Create a panel to group two labels
JPanel p2 = new JPanel(new GridLayout(1, 2, 5, 5));
JLabel jlblRed = new JLabel("Red");
JLabel jlblOrange = new JLabel("Orange");
jlblRed.setForeground(Color.RED);
jlblOrange.setForeground(Color.ORANGE);
jlblRed.setFont(largeFont);
jlblOrange.setFont(largeFont);
jlblRed.setBorder(lineBorder);
jlblOrange.setBorder(lineBorder);
p2.add(jlblRed);
p2.add(jlblOrange);
p2.setBorder(new TitledBorder("Two Labels"));

// Add two panels to the frame
setLayout(new GridLayout(2, 1, 5, 5));
add(p1);
add(p2);
}

public static void main(String[] args) {
    // Create a frame and set its properties
    JFrame frame = new TestSwingCommonFeatures();
    frame.setTitle("TestSwingCommonFeatures");
    frame.setSize(300, 150);
    frame.setLocationRelativeTo(null); // Center the frame
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);
}
}

```



Example 8: Image Icon

```
import javax.swing.*.*;
import java.awt.*.*;

public class TestImageIcon extends JFrame {
    private ImageIcon usIcon = new ImageIcon("image/us.gif");
    private ImageIcon myIcon = new ImageIcon("image/my.jpg");
    private ImageIcon frIcon = new ImageIcon("image/fr.gif");
    private ImageIcon ukIcon = new ImageIcon("image/uk.gif");

    public TestImageIcon() {
        setLayout(new GridLayout(1, 4, 5, 5));
        add(new JLabel(usIcon));
        add(new JLabel(myIcon));
        add(new JButton(frIcon));
        add(new JButton(ukIcon));
    }

    /** Main method */
    public static void main(String[] args) {
        TestImageIcon frame = new TestImageIcon();
        frame.setTitle("TestImageIcon");
        frame.setSize(500, 125);
        frame.setLocationRelativeTo(null); // Center the frame
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```



Exercises:

Exercise 1:

(Use the FlowLayout manager) Write a program that meets the following requirements (see Figure):



- Create a frame and set its layout to FlowLayout.
- Create two panels and add them to the frame.
- Each panel contains three buttons. The panel uses FlowLayout.



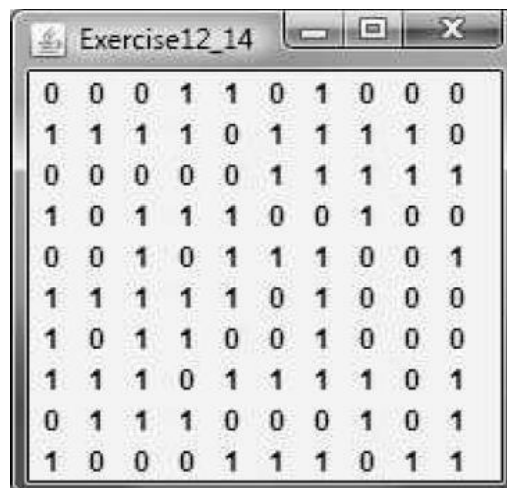
(Use the BorderLayout manager) Rewrite the preceding program to create the same user interface, but instead of using FlowLayout for the frame, use BorderLayout. Place one panel in the south of the frame and the other in the center.

(Use the GridLayout manager) Rewrite Programming Exercise 1 to add six buttons into a frame. Use a GridLayout of two rows and three columns for the frame.

(Use JPanel to group buttons) Rewrite Programming Exercise 1 to create the same user interface. Instead of creating buttons and panels separately, define a class that extends the JPanel class. Place three buttons in your panel class, and create two panels from the user-defined panel class.

Exercise 2:

(Display random 0 or 1) Write a program that displays a 10-by-10 square matrix, as shown in Figure. Each element in the matrix is 0 or 1, randomly generated. Display each number centered in a label.



Lab No. 21

GUI Programming-II

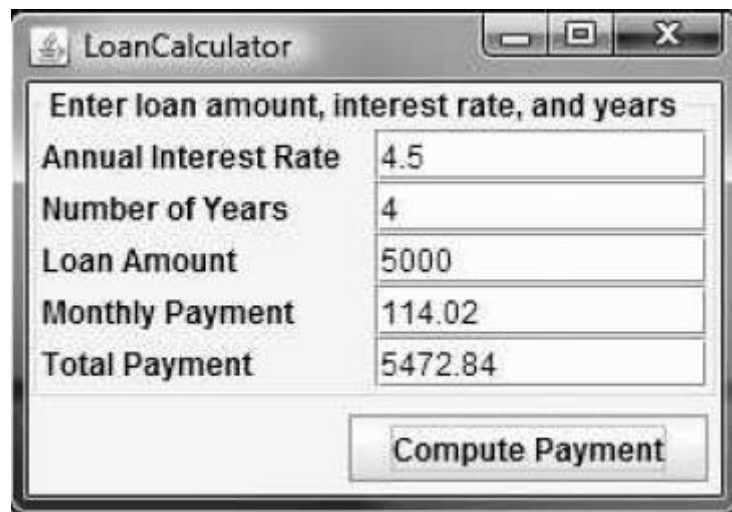
Lab No. 21: GUI Programming - II

Objective:

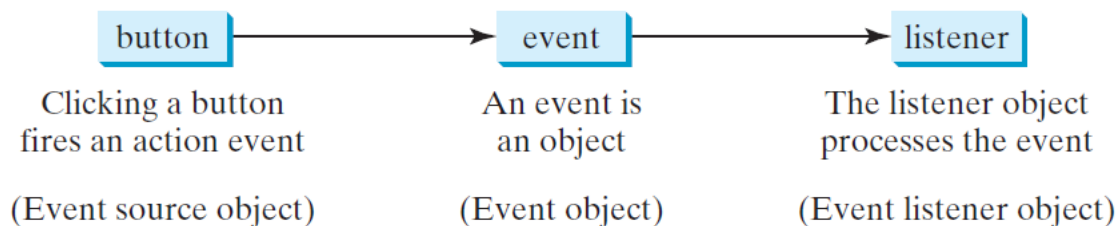
To get a taste of event-driven programming.

Scope:

Suppose you want to write a GUI program that lets the user enter a loan amount, annual interest rate, and number of years and click the Compute Payment button to obtain the monthly payment and total payment, as shown in Figure. How do you accomplish the task? You have to use event-driven programming to write the code to respond to the button-clicking event.



To respond to a button click, you need to write the code to process the button-clicking action. The button is an event source object—where the action originates. You need to create an object capable of handling the action event on a button. This object is called an event listener.



Example 1: Handling a simple event

```
import javax.swing.*;
import java.awt.event.*;

public class HandleEvent extends JFrame {

    public HandleEvent() {
        // Create two buttons
        JButton jbtOK = new JButton("OK");
        JButton jbtCancel = new JButton("Cancel");

        // Create a panel to hold buttons
        JPanel panel = new JPanel();
        panel.add(jbtOK);
        panel.add(jbtCancel);

        add(panel); // Add panel to the frame

        // Register listeners
        OKListenerClass listener1 = new OKListenerClass();
        CancelListenerClass listener2 = new CancelListenerClass();
        jbtOK.addActionListener(listener1);
        jbtCancel.addActionListener(listener2);
    }
}
```



```

public static void main(String[] args) {
    JFrame frame = new HandleEvent();
    frame.setTitle("Handle Event");
    frame.setSize(200, 150);
    frame.setLocation(200, 100);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);
}
}

```

```

class OKListenerClass implements ActionListener {

    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("OK button clicked");
    }
}

```

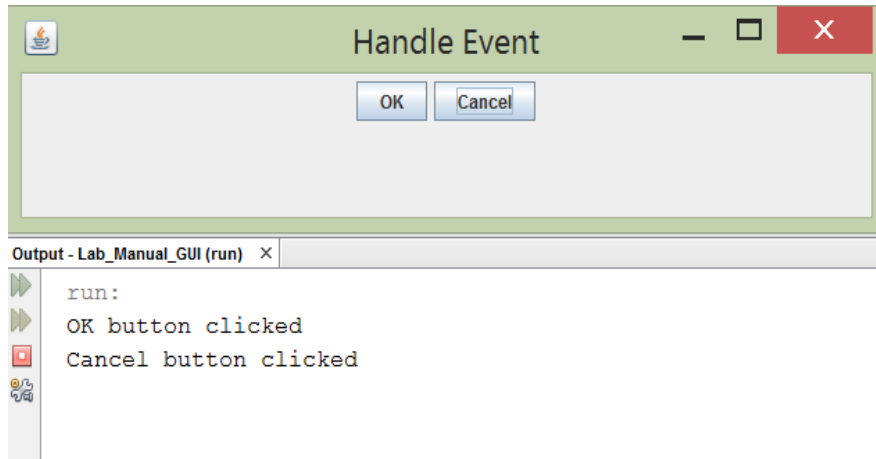
```

class CancelListenerClass implements ActionListener {

    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Cancel button clicked");
    }
}

```





Example 2: Handling a simple event (Alternate way using inner anonymous classes)

```
import javax.swing.*;
import java.awt.event.*;

public class HandleEvent extends JFrame {

    public HandleEvent() {
        // Create two buttons
        JButton jbtOK = new JButton("OK");
        JButton jbtCancel = new JButton("Cancel");

        // Create a panel to hold buttons
        JPanel panel = new JPanel();
        panel.add(jbtOK);
        panel.add(jbtCancel);

        add(panel); // Add panel to the frame

        // Register listeners
        jbtOK.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                jbtOKActionPerformed(e);
            }
        });
        jbtCancel.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                jbtCancelActionPerformed(e);
            }
        });
    }

    public void jbtOKActionPerformed(ActionEvent evt){
        System.out.println("OK button clicked");
    }

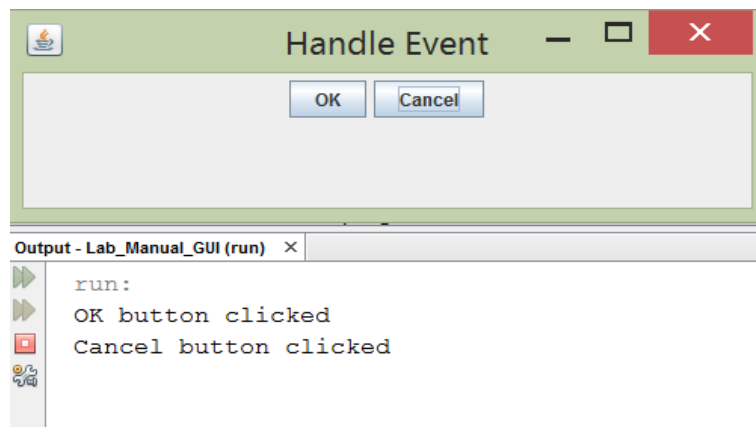
    public void jbtCancelActionPerformed(ActionEvent evt){
        System.out.println("Cancel button clicked");
    }
}
```



```

public static void main(String[] args) {
    JFrame frame = new HandleEvent();
    frame.setTitle("Handle Event");
    frame.setSize(200, 150);
    frame.setLocation(200, 100);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);
}
}

```



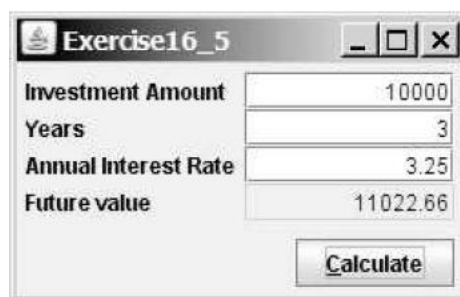
Exercises:

Exercise 1:

(Create an investment-value calculator) Write a program that calculates the future value of an investment at a given interest rate for a specified number of years. The formula for the calculation is:

$$\text{futureValue} = \text{investmentAmount} * (1 + \text{monthlyInterestRate})^{\text{years} * 12}$$

Use text fields for the investment amount, number of years, and annual interest rate. Display the future amount in a text field when the user clicks the Calculate button, as shown in Figure.



Lab No. 22

JDBC-I

Lab No. 22: JDBC-I

Objective:

To implement the concepts of JDBC, that enables Java programs to execute SQL statements. Since nearly all relational database management systems (DBMSs) support SQL. JDBC makes it possible to write a single database application that can run on different platforms and interact with different DBMSs.

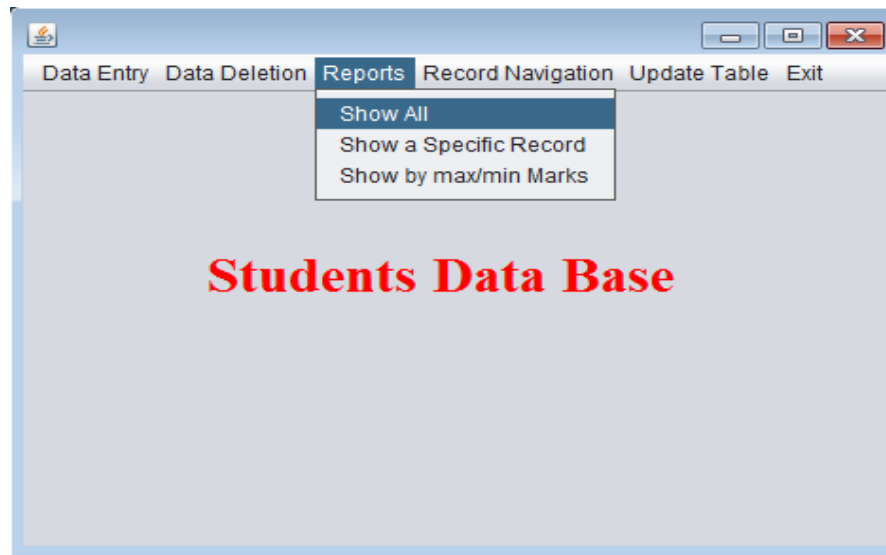
Scope:

JAVA interacts with database using a common database application programming interface called as JDBC. The JDBC API provides set of interfaces and there are different implementations respective to different databases. So the developer will be interacting with JDBC API rather than getting in to complications of the database. JDBC allows us to write Java code, and leave the platform (database) specific code to the driver. JAVA was designed to be platform independent and JDBC takes JAVA one step ahead making java database code database independent. That means if we use JDBC for connectivity for SQL Server we can run the same code without changing for ORACLE.

Example 1: This program demonstrates to how data retrieved from the table by applying JDBC objects.

```
private void jMenuItem5ActionPerformed(java.awt.event.ActionEvent evt) {  
    try{  
        String url="jdbc:odbc:MYDSN";  
        Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");  
        Connection c=DriverManager.getConnection(url);  
        Statement st=c.createStatement();  
        ResultSet rs=st.executeQuery("select * from data");  
  
        System.out.println("S_ID\tS_Name\tOOP_Marks\tGrade");  
        while(rs.next()){  
            System.out.println(rs.getInt(1)+"\t"+rs.getString(2)+"\t"+  
                rs.getInt(3)+"\t"+rs.getString(4));  
        }  
        c.close();  
    }catch(Exception ee){  
        System.out.println(ee);  
    }  
}
```





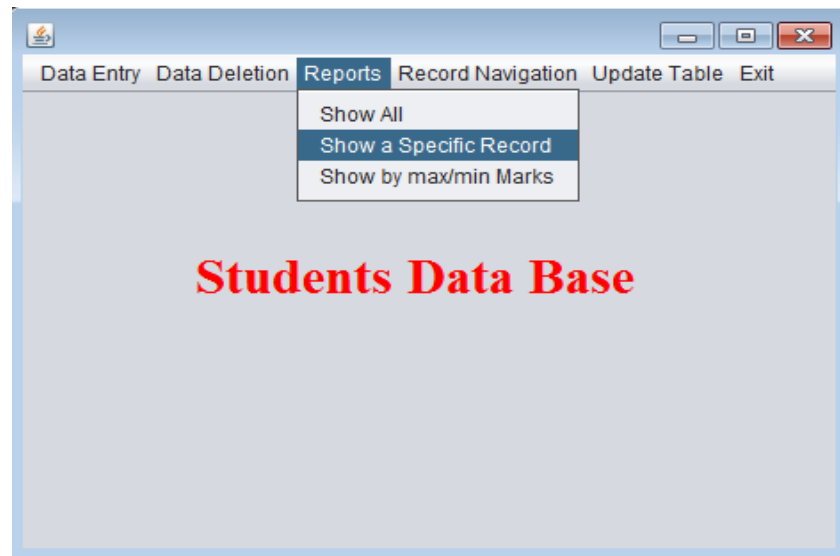
Output - Data_Base_99 (run)

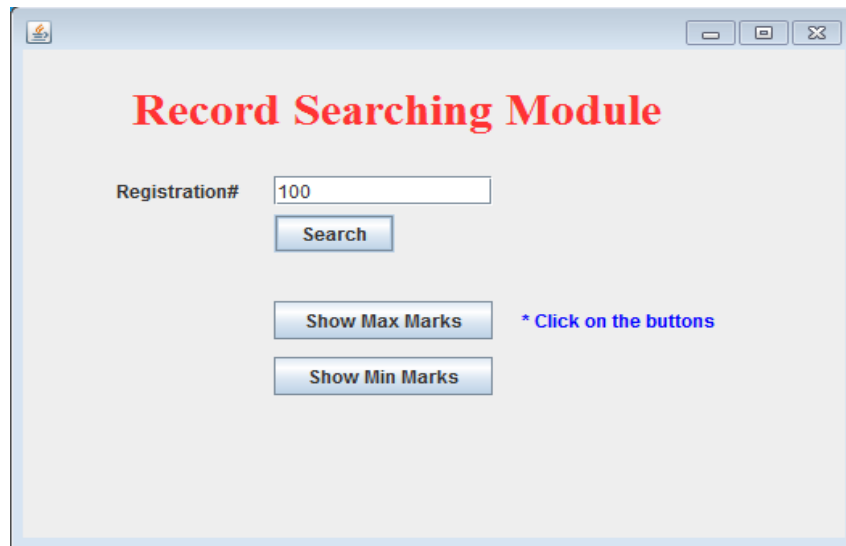
S_ID	S_Name	OOP_Marks	Grade
10	MOHAMMAD	83	B
20	UMAR	91	A
30	BALOGH	70	C
40	ADIL	85	B
50	TARIQ	40	F
99	KHAN	45	F

Example 2: To search the specific record in the table following query will be executed-

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    try{
        String url="jdbc:odbc:MYDSN";
        Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
        Connection c=DriverManager.getConnection(url);
        Statement st=c.createStatement();
        int kid = Integer.parseInt(text1.getText());
        String s = "select * from data where stud_id="+kid;
        ResultSet rs=st.executeQuery(s);

        String str="ID          Name          Marks          Grade\n";
        str+="\n";
        while(rs.next()){
            str+=rs.getInt(1)+" "+rs.getString(2)+" "+rs.getInt(3)+" "
            + " "+rs.getString(4);
        }
        JOptionPane.showMessageDialog(null,str,"Search by Registration#",
        JOptionPane.INFORMATION_MESSAGE);
        c.close();
    }catch(Exception ee){
        System.out.println(ee);
    }
    text1.setText("");
}
```

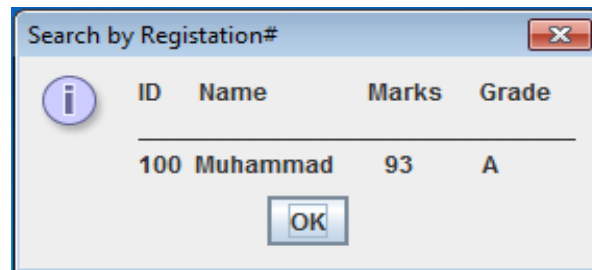




Record Searching Module

Registration#

* Click on the buttons

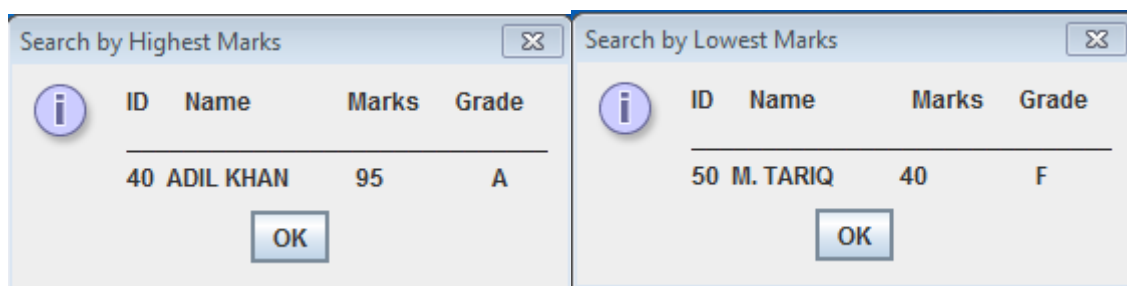


ID	Name	Marks	Grade
100	Muhammad	93	A

Example 3: Search by Highest and the lowest marks with details by applying the sub-query

```
ResultSet rs=st.executeQuery("select * from data where marks="+
                               "(select MAX(marks)from data)");

ResultSet rs=st.executeQuery("select * from data where marks="+
                               "(select MIN(marks)from data)");
```



ID	Name	Marks	Grade
40	ADIL KHAN	95	A

ID	Name	Marks	Grade
50	M. TARIQ	40	F

Exercises:

Exercise 1:

Create the following tables:

1) client_master

columnname	datatype	size
client_no	varchar	25
name	varchar2	20
address1	varchar2	30
address2	varchar2	30
city	varchar2	15
state	varchar2	15
pincode	number	06
bal_due	number	10,2

2) Product_master

Columnname	datatype	size
Product_no	varchar2	25
Description	varchar2	30
Profit_percent	number	9,2
Unit_measure	varchar2	10
Qty_on_hand	number	06
Reoder_lvl	number	05
Sell_price	number	06
Cost_price	number	06

On the basis of above two tables apply queries accordingly:

- i) Find out the names of all the clients.
- ii) Retrieve the list of names and cities of all the clients.
- iii) List the various products available from the product_master table.
- iv) List all the clients who are located in D.I. Khan.
- v) Display the information for client no 001 and 007.

Homework:

- i) Find the products with description as '1.44 drive' and '1.22 Drive'.
- ii) Find all the products whose selling price is greater than 7000.
- iii) Find the list of all clients who stay in city 'D.I. Khan' or city 'Abbottabad'.
- iv) Find the product whose selling price is greater than 3000 and less than or equal to 9000.
- v) List the name, city and state of clients those are not in the state of 'KPK'.



Lab No. 23

JDBC-II

Lab No. 23: JDBC-II

Objective:

To implement the concepts of JDBC, that enables Java programs to execute SQL statements. Since nearly all relational database management systems (DBMSs) support SQL. JDBC makes it possible to write a single database application that can run on different platforms and interact with different DBMSs.

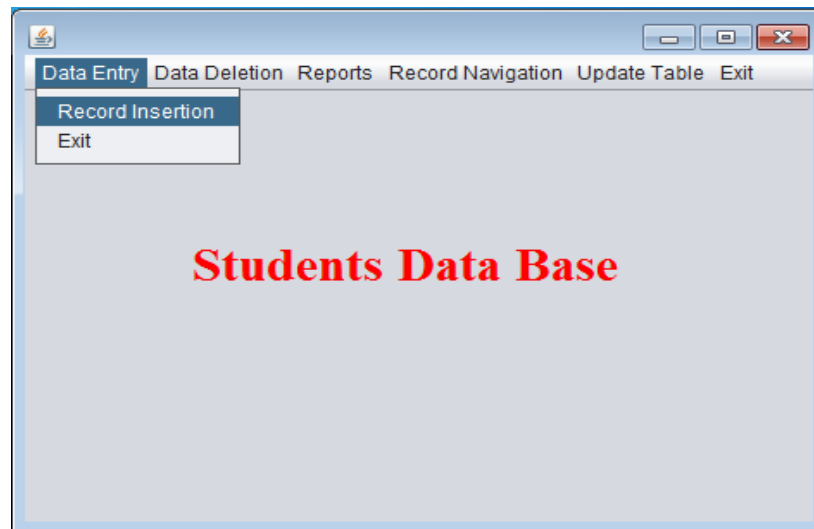
Scope:

JAVA interacts with database using a common database application programming interface called as JDBC. The JDBC API provides set of interfaces and there are different implementations respective to different databases. So the developer will be interacting with JDBC API rather than getting in to complications of the database. JDBC allows us to write Java code, and leave the platform (database) specific code to the driver. JAVA was designed to be platform independent and JDBC takes JAVA one step ahead making java database code database independent. That means if we use JDBC for connectivity for SQL Server we can run the same code without changing for ORACLE.

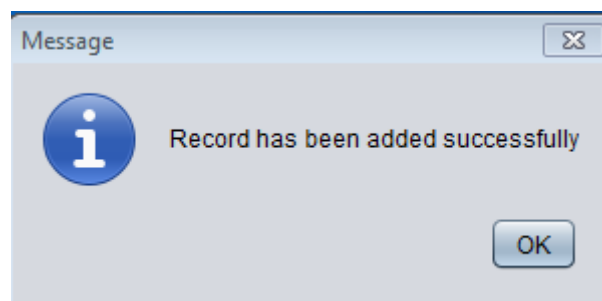
Example 1: This program demonstrates that user may enter the data into the database.

```
try{
    String url="jdbc:odbc:MYDSN";
    Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
    Connection c=DriverManager.getConnection(url);
    Statement st=c.createStatement();
    int kid = Integer.parseInt(text1.getText());
    String kname = text2.getText();
    int kmarks = Integer.parseInt(text3.getText());
    if(kmarks>=90)
        gr="A";
    else if(kmarks>=80)
        gr="B";
    else if(kmarks>=70)
        gr="C";
    else if(kmarks>=60)
        gr="D";
    else
        gr="F";
    String sql = "INSERT INTO data VALUES ('"+kid+"', '"+kname+"', '"+
        kmarks+"', '"+gr+"') ";
    st.executeUpdate(sql);
    JOptionPane.showMessageDialog(null,"Record has been added "
        + "successfully");
    c.close();
}catch(Exception ee){
    System.out.println(ee.getMessage());
}
```





The screenshot shows a Java Swing window titled "Data Entry Module". The window contains three text input fields and a "Save" button. The first field is labeled "Student Registration#" and contains the value "583". To the right of this field is a blue asterisk followed by the text "* enter a valid registration#". The second field is labeled "Student Name" and contains the value "M. USMAN". The third field is labeled "Marks in OOP" and contains the value "83". The "Save" button is located below the "Marks in OOP" field.



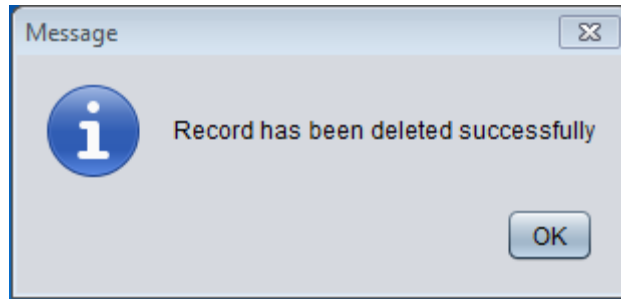
Example 2: This program demonstrates the Record Deletion.

```
try{
String url="jdbc:odbc:MYDSN";
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection c=DriverManager.getConnection(url);
Statement st=c.createStatement();

int kid = Integer.parseInt(text1.getText());

String sql = "DELETE from data where stud_id="+kid;
st.executeUpdate(sql);
JOptionPane.showMessageDialog(null, "Record deleted successfully");
c.close();
}catch(Exception ee){
    System.out.println(ee);
}
```





Exercises:

Exercise 1:

On the basis of above two tables apply queries accordingly:

1. Change the selling price of '1.44 floppy drive to Rs.1150.00
2. Delete the record with client 0001 from the client master table.
3. Change the city of client_no'0005' to Abbottabad.
4. Change the bal_due of client_no '0001, to 5000.
5. Find the products whose selling price is more than 3500 and also find the new selling price as original selling price *15.
6. Find out the clients who stay in a city whose second letter is **b**.
7. Find out the name of all clients having 'a' as the second letter in their names.
8. List the products in sorted order of their description.
9. Count the total number of orders
10. Calculate the average price of all the products.
11. Calculate the minimum price of products.
12. Determine the maximum and minimum prices. Rename the title as 'max_price' and min_price respectively.
13. Count the number of products having price greater than or equal to 3500.

Homework:

(Group by)

1. Print the description and total quantity sold for each product.
2. Find the value of each product sold.
3. Calculate the average quantity sold for each client that has a maximum order value of 15000.
4. Find out the products which has been sold to Baloch.



5. Find the names of clients who have 'CD Drive'.
6. Find the products and their quantities for the orders placed by 'Tariq' and 'Baloch'.
7. Select product_no, total qty_ordered for each product.
8. Select product_no, product description and qty ordered for each product.
9. Display the order number and day on which clients placed their order.
10. Display the month and Date when the order must be delivered.



Lab No. 24

JDBC-III

Lab No. 24: JDBC-III

Objective:

To implement the concepts of JDBC, that enables Java programs to execute SQL statements. Since nearly all relational database management systems (DBMSs) support SQL. JDBC makes it possible to write a single database application that can run on different platforms and interact with different DBMSs.

Scope:

JAVA interacts with database using a common database application programming interface called as JDBC. The JDBC API provides set of interfaces and there are different implementations respective to different databases. So the developer will be interacting with JDBC API rather than getting in to complications of the database. JDBC allows us to write Java code, and leave the platform (database) specific code to the driver. JAVA was designed to be platform independent and JDBC takes JAVA one step ahead making java database code database independent. That means if we use JDBC for connectivity for SQL Server we can run the same code without changing for ORACLE.

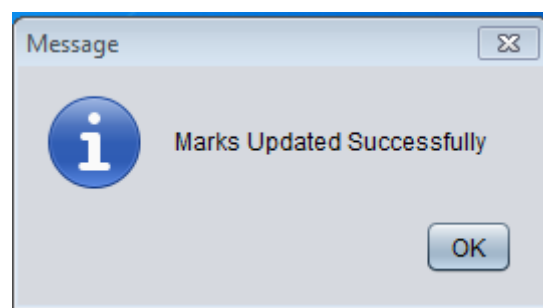
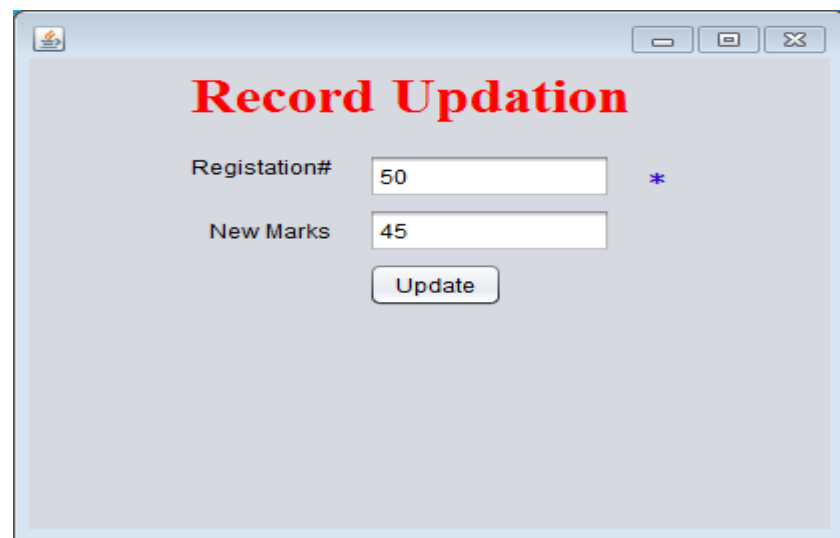
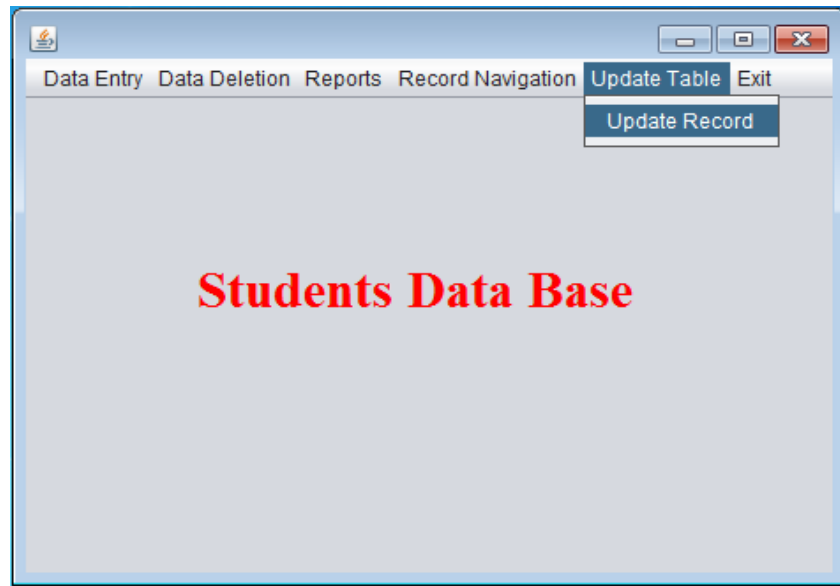
Example 1: This program demonstrates Record updating.

```
try{
    String url="jdbc:odbc:MYDSN";
    Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
    Connection c=DriverManager.getConnection(url);
    Statement st=c.createStatement();

    int kid = Integer.parseInt(text1.getText());
    int kmarks = Integer.parseInt(text3.getText());

    if(kmarks>=90)
        gr="A";
    else if(kmarks>=80)
        gr="B";
    else if(kmarks>=70)
        gr="C";
    else if(kmarks>=60)
        gr="D";
    else
        gr="F";
    String sql = "UPDATE data set marks="+kmarks+",grade='"+gr+"' "
                + " where stud_id="+kid;
    st.executeUpdate(sql);
    JOptionPane.showMessageDialog(null, sql);
    c.close();
}catch(Exception ee){ System.out.println(ee); }
JOptionPane.showMessageDialog(null,"Marks Updated Successfully");
```





Example 2: This program demonstrates the Navigation of Records.

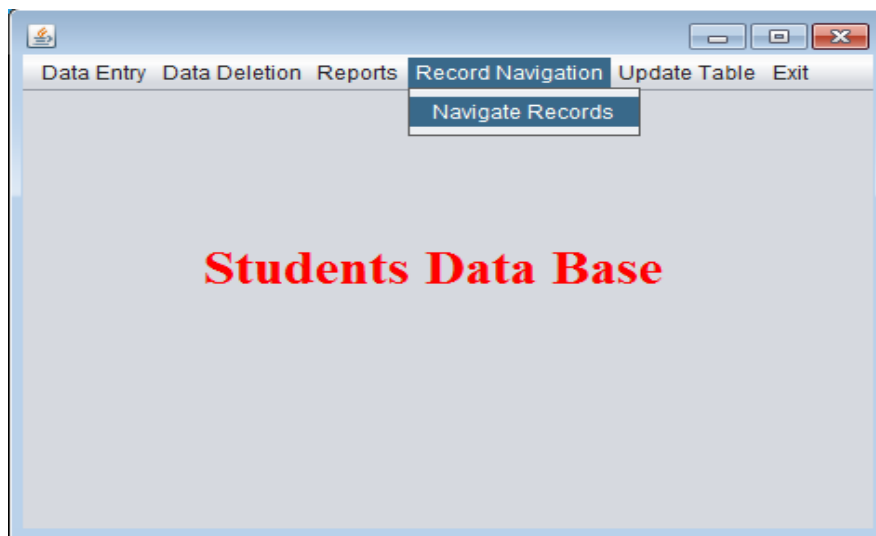
```
try{
String url="jdbc:odbc:MYDSN";
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection c=DriverManager.getConnection(url);
Statement st=c.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                                ResultSet.CONCUR_READ_ONLY);
ResultSet rs=st.executeQuery("select * from data");
rs.first();
    text1.setText(""+rs.getInt(1));
    text2.setText(""+rs.getString(2));
    text3.setText(""+rs.getInt(3));
    text4.setText(""+rs.getString(4));
c.close();
}catch(Exception ee){
    System.out.println(ee);
}

try{
String url="jdbc:odbc:MYDSN";
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection c=DriverManager.getConnection(url);
Statement st=c.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                                ResultSet.CONCUR_READ_ONLY);
ResultSet rs=st.executeQuery("select * from data");
rs.last();
    text1.setText(""+rs.getInt(1));
    text2.setText(""+rs.getString(2));
    text3.setText(""+rs.getInt(3));
    text4.setText(""+rs.getString(4));
c.close();
}catch(Exception ee){
    System.out.println(ee);
}
```

```

try{
String url="jdbc:odbc:MYDSN";
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection c=DriverManager.getConnection(url);
Statement st=c.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                                ResultSet.CONCUR_READ_ONLY);
ResultSet rs=st.executeQuery("select * from data");
rs.afterLast();
while(rs.previous()){
    text1.setText(""+rs.getInt(1));
    text2.setText(""+rs.getString(2));
    text3.setText(""+rs.getInt(3));
    text4.setText(""+rs.getString(4));
}
c.close();
}catch(Exception ee){
    System.out.println(ee);
}

```



Exercises:

Exercise 1:

(Sub-Query)

On the basis of above two tables apply queries accordingly:

1. Find the product_no and description of non- moving products.
2. Find the customer name, address, city and pincode for the client who has placed order no “0583”
3. Find the client names who have placed order before the month of Dec 2015.
4. Find out if product “1.44 Drive” is ordered by only client and print the client_no name to whom it was sold.
5. Find the names of client who have placed orders worth Rs.10000 or more.
6. Select the orders placed by ‘Baloch’.

Homework:

1. Select the names of persons who are in Mr. Mudassar’s department and who have also worked on an inventory control system.
2. Select all the clients and the salesman in the city of Lahore.
3. Select salesman name in “Lahore” who has at least one client located at “D.I.Khan”
4. Select the product_no, description, qty_on-hand, cost_price of on_moving items in the product_master table.



Lab No. 25

Multithreading-I

Lab No. 25: Multithreading-I

Objective:

To implement the concepts of Multithreading

Scope:

Multithreading is a widespread programming and execution model that allows multiple threads to exist within the context of a single process. These threads share the process's resources, but are able to execute independently. The threaded programming model provides developers with a useful abstraction of concurrent execution. Multithreading can also be applied to a single process to enable parallel execution on a multiprocessing system.

Example 1: This program demonstrates how to implement threads in Java.

```
package threading;

class A extends Thread{
    String name;
    A(String n){
        name=n;
    }
    public void run(){
        try{
            for(int i=1; i<=5; i++) {
                System.out.println("Greetings from "+name+" "+i);
                Thread.sleep(3000);
            }
        }catch(InterruptedException e){
            System.out.println(e.getMessage());
        }
    }
}

//end of A

class B extends Thread{
    String name;
    B(String n){
        name=n;
    }
    public void run(){
        try{
            for(int i=1; i<=5; i++) {
                System.out.println("Greetings from "+name+"....."+i);
                Thread.sleep(1500);
            }
        }
    }
}
```



```

        }catch (InterruptedException e){
            System.out.println(e.getMessage());
        }try
    } //run
} //end of B
//Main Program
public class Threading {
    public static void main(String[] args) {
        A t1 = new A("Mohammad Umar...");
        B t2 = new B("Mohammad Tariq...");
        t1.start();
        t2.start();
    }
}

```

Output - Threading (run)

```

run:
Greetings from Mohammad Umar...    1
Greetings from Mohammad Tariq.....1
Greetings from Mohammad Tariq.....2
Greetings from Mohammad Umar...    2
Greetings from Mohammad Tariq.....3
Greetings from Mohammad Tariq.....4
Greetings from Mohammad Umar...    3
Greetings from Mohammad Tariq.....5
Greetings from Mohammad Umar...    4
Greetings from Mohammad Umar...    5
BUILD SUCCESSFUL (total time: 15 seconds)

```



Example 2: This program demonstrates how to implement threads in Java

```
package threading;

class A extends Thread{
    String name;
    A(String j){
        name=j;
    }
    public void run(){
        try{
            for(int i=1; i<=5; i++){
                System.out.println(name+"\t"+i);
                System.out.println("Current Thread is executing..." +
                                   Thread.currentThread().getName());
                Thread.sleep(1000);
            }
        }catch(InterruptedException e){
            System.out.println(e.getMessage());
        }
    }
}

//A
//Main Program
public class Threading {
    public static void main(String[] args) {
        A t1 = new A("Thread ONE.....");
        A t2 = new A("Thread TWO.....");
        A t3 = new A("Thread THREE...");
        t1.start();

        try{
            t1.join(); //t1 executed till end, then next thread will start
        }catch(Exception e){
            System.out.println(e);
        }

        t2.start();    t3.start();
        System.out.println(t1.isAlive()); //System.out.println(t1.getName());
        long start=System.currentTimeMillis();
        for(long i=0; i<100; i++){
            try{    Thread.sleep(100);
            }catch(Exception e){
                System.out.println(e);
            }
        }
        long end=System.currentTimeMillis();
        System.out.println("Start Time =" +start);
        System.out.println("End Time   =" +end);
        System.out.println("Difference =" +(end-start)/1000);
    }
}
```



Output - Threading (run)

```
run:
Thread ONE..... 1
Current Thread is executing...Thread-0
Thread ONE..... 2
Current Thread is executing...Thread-0
Thread ONE..... 3
Current Thread is executing...Thread-0
Thread ONE..... 4
Current Thread is executing...Thread-0
Thread ONE..... 5
Current Thread is executing...Thread-0
false
Thread THREE... 1
Current Thread is executing...Thread-2
Thread TWO..... 1
Current Thread is executing...Thread-1
Thread THREE... 2
Current Thread is executing...Thread-2
Thread TWO..... 2
Current Thread is executing...Thread-1
Thread THREE... 3
Current Thread is executing...Thread-2
Thread TWO..... 3
Current Thread is executing...Thread-1
Thread THREE... 4
Current Thread is executing...Thread-2
Thread TWO..... 4
Current Thread is executing...Thread-1
Thread TWO..... 5
Current Thread is executing...Thread-1
Thread THREE... 5
Current Thread is executing...Thread-2
Start Time =1436851750399
End Time   =1436851761321
Difference =10
BUILD SUCCESSFUL (total time: 17 seconds)
```



Exercises:

Exercise 1:

Create an example of a “busy wait.” One thread sleeps for awhile and then sets a flag to **true**. The second thread watches that flag inside a while loop (this is the “busy wait”) and when the flag becomes **true**, sets it back to false and reports the change to the console. Note how much wasted time the program spends inside the “busy wait” and create a second version of the program that uses **wait()** instead of the “busy wait.”

Homework

Exercise 1

Write a program to create two threads. In this class we have one constructor used to start the threads and run it. Check whether these two threads are run or not.

Exercise 2

Create a multithreaded program by using Runnable interface and then create, initialize and start three Thread objects from your class. The threads will execute concurrently and display the following String array elements.

```
String course [ ] = {“Java”, “J2EE”, “Spring”, “Struts”};
```



Lab No. 26

Multithreading-II

Lab No. 26: Multithreading-II

Objective:

To implement the concepts of Multithreading

Scope:

Multithreading is a widespread programming and execution model that allows multiple threads to exist within the context of a single process. These threads share the process's resources, but are able to execute independently. The threaded programming model provides developers with a useful abstraction of concurrent execution. Multithreading can also be applied to a single process to enable parallel execution on a multiprocessing system.

Example 1: This program demonstrates how to implement threads in Java.

```
package threading;

class A extends Thread{
    String name;
    A(String n){
        name=n;
    }
    public void run(){
        int f=1;
        try{
            for(int i=1; i<=9; i++) {
                f=f*i;
                System.out.println("Factorial Calculation :"+name+"..."+"i+"      "+f);
                Thread.sleep(1000);
            }
        }catch (InterruptedException e){
            System.out.println(e.getMessage());
        }
    }
}

//end of A

class B extends Thread{
    String name;
    B(String n){
        name=n;
    }
    public void run(){
        int x,sum=0;
        try{
```



Example 2: This program demonstrates how to implement threads in Java

```
package threading;
class Counter implements Runnable {
    public int counter = 0;
    private final int rounds = 10000;
    public void run () {
        int id;
        String name = Thread.currentThread().getName();
        if (name.equals("thread1"))
            id = 1;
        else
            id = 2;

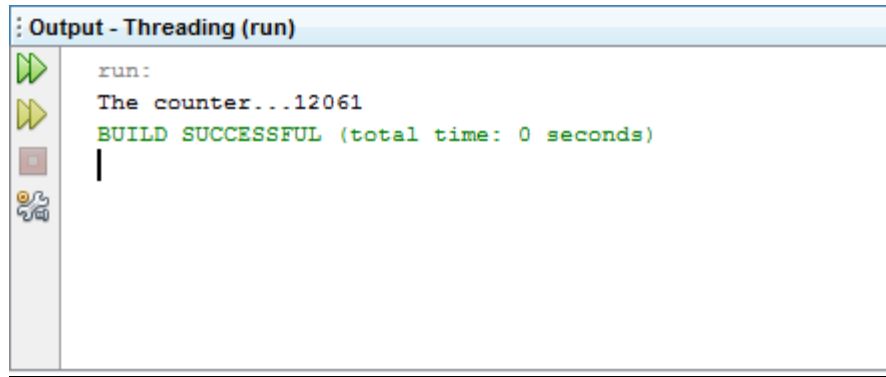
        for(int i = 0; i < rounds; i++) {
            int tmp = counter;
            counter = tmp + 1;           //System.out.println(counter);
        }
    }
}

//Main Program
public class Threading {
    public static void main(String[] args) {
        try {
            Counter ob = new Counter ();
            // Create two threads that run our run () method.
            Thread t1 = new Thread (ob, "thread1");
            Thread t2 = new Thread (ob, "thread2");

            t1.start(); t2.start();
            // Wait for the threads to finish.
            t1.join(); t2.join();
            System.out.println("The counter..." + ob.counter); //Print the counter

        } catch (InterruptedException e) {
            System.out.println ("Interrupted!");
        }
    }
}
```





```
run:
The counter...12061
BUILD SUCCESSFUL (total time: 0 seconds)
```

Exercises:

Exercise 1:

Write a program that runs 5 threads, each thread randomizes a number between 1 and 10. The main thread waits for all the others to finish, calculates the sum of the numbers which were randomized and prints that sum. You will need to implement a *Runnable* class that randomizes a number and store it in a member field. When all the threads have done, your main program can go over all the objects and check the stored values in each object.

Exercise 2:

Modify the program in (Ex. 1) so that instead of each object keeping its own score, you will use one collection to store all the results in.

Homework:

1. Write a program for inventory problem to illustrate the usage of synchronized keyword.

Note: The output should be similar as mentioned below:

Thread1Thread[test thread,5,main]

Thread2Thread[test thread,5,main]

Quantity ordered :13

Quantity on hand :487

Total quantify taken away by way of order :13

Quantity ordered :91

Quantity on hand :396

Total quantify taken away by way of order :104

Object Oriented Programming using JAVA



Lab No. 27

Special Lab Task-I

Lab No. 27: Special lab Task – I (Linked List Implementation)

Objective:

The special lab is designed for the students with some general menu driven programs for better understanding of Java that will be more beneficial for the students and to meet the objective of the course.

Scope:

This will be helpful for the students to adapt the mini-project level approach.

Exercise 1:

You are supposed to write a complete Inventory system in Java by applying Linked List for James Timber Store. James Timber owns a large piece of forest. He runs a business cutting down the trees and sells them. He asks you to develop a program to track the trees in the forest. Each tree when cut (called timber) has got unique identification code and other information for each of his timber as below.

- Zone : a character representing the location zone of the tree
- TimberID : integer
- Kind : a 30 character description of the timber
- Weight : a float number containing the weight of the timber
- Height : a float number containing the height of the timber
- Quantity : integer
- Price : integer

Perform the following tasks

- a) Declare the nodes for the linked list using all details written above. Make sure you declare a object reference in the node so that it can be used to link the other nodes.
- b) Create a Main Menu to allow the user to choose options.

Main Menu

- [1]. Add a new timber records
- [2]. Display all records for a given Zone
- [3]. Display a particular record given the Kind
- [4]. Analysis of record level
- [5]. Sales update Report



Please enter your choice:???

- c) **Add New---** is used to allow the user to enter timber record details. No duplication of TimberID is allowed.
- d) **Display Records**—will display zone wise records, for example there are four zone in the forest like A, B, C, D.
- e) **Display Particular record** of the given kind
- f) **Analysis of record level**—Display all records where the quantity in hand is lower than 100 in the stock.
- g) **Sales update**—ask the user to enter the TimberID to be sold. Search the TimberID in the linked list and when found ask the user Quantity to be sold. If the quantity to be sold is greater than quantity on hand, please display an error message, otherwise update the node accordingly and display the sales report.



Lab No. 28

Special Lab Task-II

Lab No. 28: Special Lab Task –II (Queue Implementation)

Objective:

To be able to form linked data structure by self-referential classes and to create and manipulate dynamic data structures such as Linked List, Queues and Stack and to understand various important applications of linked data structures.

Scope:

Queue has many applications in computer systems, most computers have only a single processor, so only one user can be server at a time. Entries for the other users are placed in a queue. Each entry gradually advances to the front of the queue as user receives service. The entry at the front of the queue is the next to receive service process will continue till all the jobs are finished.

Exercise:

1) Define a class called PastVisits that contains the following fields:

DateVisited: a string of 10 characters in form of 'dd/mm/yyyy'

MedicinePrescribed: a string of 100 characters

Symptoms: a string of 100 characters

Define a dynamic Queue called PatientRecords which is capable of holding the following information:

Name: a string of up to 30 characters

Visits: an integer recording the numbers of visits to the clinic

PastVisitsInfo: array that can store 100 of the above PastVistits class

NextPatient: a pointer pointing to the next PatientRec class

Define a class called PatientsQueue which stores the following:

NumOfPatient: an integer recording the number of patients in the queue

FirstInQueue: an address pointing to the first PatientRec class



LastInQueue: an address pointing to the last PatientRec class

- a) The doctor wishes to know the name of the next patient to visit him. Write a method called `DisplayFirstPatient()` that takes in a parameter which points to a `PatientQueue` and prints out the name of the patient at the head of the queue.
- b) The nurse wishes to know the Patient's names in the queue who visited the clinic on a certain date. Write a method called `FindPatients()` which takes in a parameter which points to `PatientQueue` and a second parameter `RequiredDate` which is a string of 10 characters. The function should print out the names of all patients in the queue on that date.



Lab No. 29

Special Lab Task-III

Lab No. 29: Special Lab Task –III

Objective:

The special lab is designed for the students with some general menu driven programs for better understanding of Java that will be more beneficial for the students and to meet the objective of the course.

Scope:

This will be helpful for the students to adapt the mini-project level approach.

Exercise 1:

You should develop test cases (of screen captures) to show successful running of each of the option in this program.

Define a structure called **LineType** with following field:

Line : that can store a maximum 80 characters

Declare an array called **Page** that can store up to 25 **LineType** above

Develop a program in Java to manipulate **Page** above to allow the user to perform the following operations.

Main Menu

- [1]. Enter Lines of Text
- [2]. Retrieve all Lines
- [3]. Retrieve a particular line of Text
- [4]. Delete a particular line of Text

Choice:

If option **1** is selected

Enter Line #: 1

Enter line (up to max. 80 char): ***This is first line***

Enter Line #: 2

Enter line (up to max. 80 char): ***This is second line***

If option **2** is selected

Line #: 1 This is first line

Line #: 2 This is second line

If option **3** is selected

Enter Line #: 2

This is second line

If option **4** is selected

Enter Line # to Delete: 1

Line 1 Deleted Successfully

Exercise 2:

Develop a program that allows the user to perform the following operations

Main Menu

- 1) Convert Day Integer to Day String
 - 2) Convert Month Integer to Month String
 - 3) Convert Date Integer to Date String
 - 4) Change Date
- Choice:???

Each of the above operations is to be implemented in terms of a function taking in appropriate parameter

Whereby:



If option 1 is chosen

Enter Day Integer: **3**

Third

If option 2 is chosen

Enter Month Integer: **12**

December

If option 3 is chosen

Enter Date Integer (Day followed by Months): **2 11**

Second of November

If option 4 is chosen

Enter Original Date Integer (Day followed by Month): **2 11**

Enter no. of Days to Add: 3



Lab No. 30

Special Lab Task-IV

Lab No. 30: Special lab Task - IV

Objective:

The special lab is designed for the students with some general menu driven programs for better understanding of Java that will be more beneficial for the students and to meet the objective of the course.

Scope:

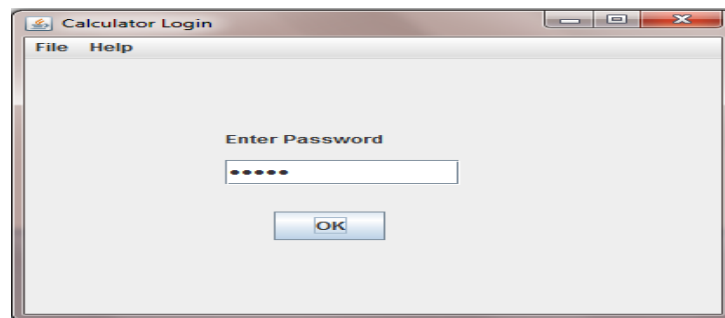
This will be helpful for the students to adapt the mini-project level approach.

Exercises:

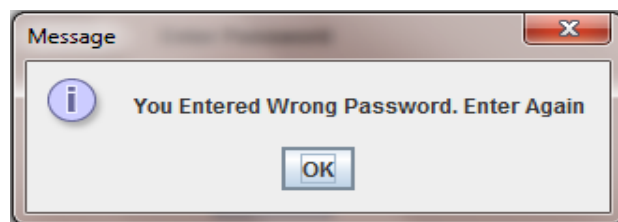
Designing Simple Calculator in JAVA with Login Form

- Design a login form with password field, If password is invalid, display the proper message
- If user enters valid password, the functional calculator should be displayed
- Apply Exception Handling where applicable

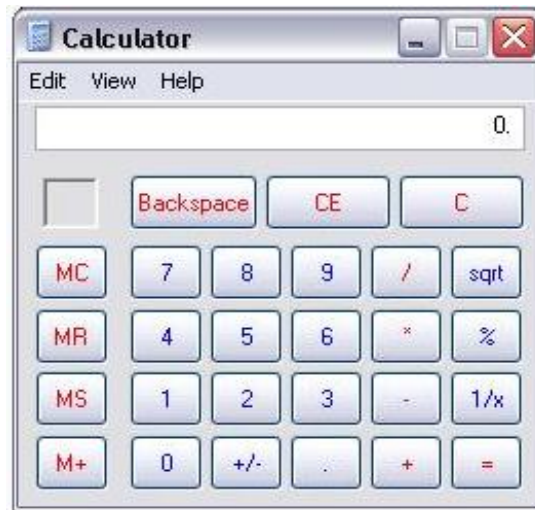
Example:



If password is invalid then



If password is valid then



References

1. Java The complete reference, by Herbert Schildt, 8th edition
2. OOP using Java, from Problem solving to Program Design by D.S. Malik
3. Java How to Program, by Deitel & Deitel, 9th edition
4. Beginning Java 2 by Ivor Horton